



UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

INGENIERÍA INFORMÁTICA EN INGENIERÍA DEL
SOFTWARE

TRABAJO FIN DE GRADO

**Estudio de viabilidad de una Nariz
Electrónica como herramienta de apoyo para
los Agentes Caninos**

Raúl Martín-Romo Sánchez
Febrero, 2026



UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

INGENIERÍA INFORMÁTICA EN INGENIERÍA DEL
SOFTWARE

TRABAJO FIN DE GRADO

**Estudio de viabilidad de una Nariz
Electrónica como herramienta de apoyo para
los Agentes Caninos**

Autor: Raúl Martín-Romo Sánchez

Tutor: Pablo García Rodríguez

Co-Tutor: Jesús Lozano Rogado

Agradecimientos

Me gustaría expresar mi más sincero agradecimiento a aquellas personas que han influido en el desarrollo de este trabajo de una manera u otra. En primer lugar, a mis tutores, Pablo García Rodríguez y Jesús Lozano Rogado, por su orientación, apoyo y asesoramiento indispensable para lograr este trabajo, y a Cristina Brugera Salas por su orientación acerca del funcionamiento de la nariz electrónica y su atención cuando tenía algún problema con la misma.

También me gustaría agradecer a mi familia por su apoyo incondicional durante toda mi etapa universitaria, y a mis amigos por su compañía en este viaje y ayudarme cuando lo necesitaba. En concreto, en relación con este proyecto, quiero agradecer a mis compañeros y amigos Alejandro Barrena, Pablo Natera y Pablo Fernández por su aportación de ideas, resolver mis dudas, revisar esta memoria para buscar errores y posibles mejoras, y haber aguantado el ruido de la nariz electrónica durante tantos días.

Por último pero no por ello menos importante, quiero dar las gracias a los miembros del Servicio Cinológico de la Guardia Civil por su disposición para ayudarme en todo lo que he necesitado, proporcionar las muestras necesarias, por permitirme conocer de primera mano el trabajo que realizan los agentes caninos en su día a día y por lo bien que me han tratado cuando he tenido que ir a la comandancia.

Dedicatoria

Quiero dedicar este trabajo a esas personas que, pese a que pueda llegar a parecerles un poco raro, me han animado y apoyado siempre en mi objetivo de unirme al cuerpo cinológico. Estas personas son mis padres, mi hermana, mis abuelos y mis amigos, todas personas muy importantes en mi vida y a las que aprecio muchísimo. Muchas gracias por aguantarme todo este tiempo, espero poder contar con todos vosotros en los años venideros.

Resumen

Se aborda el estudio de viabilidad de una nariz electrónica como herramienta de apoyo a agentes caninos en tareas de detección de sustancias, investigando sus fundamentos tecnológicos basados en sensores MOS y aplicaciones actuales. Se construye un *dataset* de muestras de hachís y sustancias negativas, entrenándose redes neuronales en Keras y PyTorch con regularización Dropout, evaluadas con y sin aumento de datos.

Se desarrollan una aplicación móvil con comunicación Bluetooth y modelos exportados a TensorFlow Lite. Ambos modelos alcanzan precisiones elevadas (96 % y 94.67 %), sin sobreajuste. En pruebas comparativas con un agente canino especializado (Zape) de la Guardia Civil se evalúan aciertos, velocidad y precisión. Zape logra 100 % de aciertos en 25 segundos máximo, mientras que la *e-nose* obtiene 50 % de aciertos en más de 155 segundos, demostrando limitaciones ante corrientes de aire y detección en presencia de personas.

Se concluye que la *e-nose* constituye una herramienta complementaria viable en situaciones específicas donde agentes caninos no estén disponibles. Se proponen mejoras futuras: especializar modelos por contexto de uso, desarrollar detectores multisustancia aprovechando la arquitectura modular, mejorar la tolerancia ambiental del dispositivo e implementar interfaces más intuitivas.

Palabras clave: Nariz Electrónica, Agentes Caninos, Sensores, Red Neuronal Artificial, Aplicación Móvil , Hachís, Sustancia objetivo.

Abstract

This work addresses the feasibility study of an electronic nose as a support tool for canine agents in substance detection tasks, investigating its technological foundations based on MOS sensors and current applications. A *dataset* of hashish samples and negative substances is constructed, training neural networks in Keras and PyTorch with Dropout regularization, evaluated with and without data augmentation.

A mobile application with Bluetooth communication is developed and models are exported to TensorFlow Lite. Both models achieve high accuracy rates (96 % and 94.67 %), without overfitting. In comparative tests with a specialized canine agent (Zape) from the Civil Guard, successes, speed, and accuracy are evaluated. Zape achieves 100 % success rate in a maximum of 25 seconds, while the *e-nose* obtains 50 % success rate in more than 155 seconds, demonstrating limitations when facing air currents and detection in the presence of people.

It is concluded that the *e-nose* constitutes a viable complementary tool in specific situations where canine agents are not available. Future improvements are proposed: specialize models by use context, develop multi-substance detectors leveraging the modular architecture, improve environmental tolerance of the device, and implement more intuitive interfaces.

Keywords: Electronic Nose, Canine Agents, Sensors, Artificial Neural Network, Mobile Application, Hashish, Target substance.

Índice general

1. Introducción	1
1.1. Motivación	2
2. Alcance y objetivos	3
2.1. Objetivos	3
2.1.1. Objetivo general	3
2.1.2. Objetivos específicos	4
2.2. Alcance	5
3. Estado del Arte	6
3.1. Narices electrónicas (<i>e-noses</i>)	6
3.1.1. Antecedentes y evolución de la nariz electrónica	7
3.1.2. Funcionamiento	7
3.1.3. Tipos de sensores	11
3.1.4. Procesamiento de datos (Quimiometría)	15
3.1.5. Aplicaciones	16
3.1.6. Desafíos y limitaciones	17

3.2. Agentes caninos	18
4. Metodología y herramientas	20
4.1. Definición de la metodología aplicada	20
4.2. Planificación	23
4.2.1. Planificación inicial	23
4.2.2. Planificación final	25
4.3. Herramientas utilizadas	29
4.3.1. Herramientas hardware	30
4.3.2. Herramientas software	32
5. Implementación, desarrollo y pruebas comparativas	49
5.1. Implementación y desarrollo	49
5.1.1. Construcción del <i>dataset</i>	50
5.1.2. Entrenamiento de la red neuronal	70
5.1.3. Desarrollo de la aplicación móvil	78
5.1.4. Funcionamiento de la aplicación	84
5.2. Pruebas comparativas	91
5.2.1. Prueba 0: Cajas vacías	93
5.2.2. Prueba 1: Paquetería	93
5.2.3. Prueba 2: Interior de vehículos	93
5.2.4. Prueba 3: Personas	94
5.2.5. Prueba 4: Equipajes	94

6. Resultados	95
6.1. Resultados de redes neuronales	95
6.2. Resultados de pruebas comparativas	99
7. Conclusiones y futuras mejoras	103
7.1. Conclusiones	103
7.2. Futuras mejoras	105
7.2.1. Mejora del <i>dataset</i>	105
7.2.2. Desarrollo de nuevos modelos	105
7.2.3. Especialización	106
7.2.4. Cambiar la terminal en la interfaz de usuario	106
Anexos	107
A. Aromas de varias sustancias	108
B. Aroma de hachís	121
C. Configuración de la aplicación para la toma de muestras	131
C.1. Configuración de ajustes	131
C.1.1. Terminal	132
C.1.2. Receive	134
C.1.3. Send	134
C.1.4. Misc	135
C.2. Conexión con la nariz electrónica	137

C.3. Configuración de botones	138
D. Entrenamiento de modelos experimentales	140
D.1. Entrenamiento	140
D.1.1. Keras	140
D.1.2. PyTorch	149
D.2. Resultados	161
D.3. Pseudocódigos	167
D.3.1. Keras	167
D.3.2. PyTorch	169
E. Pseudocódigos de Sniffer	172
E.1. Pantalla principal	172
E.2. Pantalla de análisis	175
E.3. Pantalla de ajustes	180
E.4. Barra superior	183
E.5. Navegación	185
E.6. Inicialización de la aplicación	188
Bibliografía	194

Índice de tablas

3.2. Materiales comunes en sensores MOS y sus aplicaciones.	14
4.1. Planificación inicial de sprints.	24
4.3. Comparativa de herramientas para limpieza y ampliación del <i>dataset</i>	36
4.5. Comparativa de lenguajes de programación para entrenamiento de ANN. .	39
4.7. Comparativa de entornos de ejecución para entrenamiento de ANN.	41
4.9. Comparativa de frameworks para entrenamiento de ANN.	44
4.11. Comparativa de herramientas para desarrollo de aplicación Bluetooth. . . .	47
A.2. Registro de mediciones de sustancias y condiciones de muestreo.	120
B.2. Registro de mediciones de hachís y condiciones de muestreo.	130
C.2. Configuración recomendada de botones macros para la nariz electrónica. .	139

Índice de figuras

3.1. Ejemplo de un estudio de detección de <i>Monilinia laxa</i> en nectarinas mediante PCA.	10
3.2. Funcionamiento general de una <i>e-nose</i>	11
3.3. Presentación esquemática de un sensor MOX.	15
4.1. Tablero Kanban del segundo sprint.	23
4.2. Diagrama de Gantt de la planificación inicial.	25
4.3. Seguimiento de las tareas del proyecto.	26
4.4. Diagrama de Gantt de la planificación final.	29
4.5. Carcasa de la nariz electrónica.	30
4.6. Placa PCB de la nariz electrónica.	31
4.7. Interfaz de la aplicación móvil para el control de la toma de muestras. . . .	32
5.1. Disposición del equipo para la muestra EXP_M_5-1.	52
5.2. Disposición del equipo para la muestra EXP_B_34.	53
5.3. Muestra de hachís usada de la toma 1 a la 4.	55
5.4. Muestra de hachís usada de la toma 17 a la 20.	55

5.5. Muestra de hachís usada de la toma 33 a la 38.	56
5.6. Estructura de ficheros de la aplicación móvil	81
5.7. Pantalla principal de la aplicación móvil con la lista de dispositivos Bluetooth disponibles	85
5.8. Botón de búsqueda de dispositivos Bluetooth en la aplicación Sniffer. . . .	86
5.9. Selección del modelo de red neuronal en la aplicación Sniffer.	86
5.10. Pantalla de análisis de sustancias en la aplicación Sniffer.	88
5.11. Botones de la barra superior en la pantalla de análisis de sustancias. . . .	89
5.12. Pantalla de ajustes de la aplicación móvil	90
5.13. Botón de volver en la barra superior de la pantalla de ajustes	90
5.14. Diagrama de flujo del funcionamiento de la aplicación móvil	91
5.15. Imagen de Zape, agente canino de la Guardia Civil	92
6.1. Curvas de aprendizaje del modelo ANN sin aumento de datos	96
6.2. Matriz de confusión del modelo ANN sin aumento de datos	96
6.3. Curvas de aprendizaje del modelo ANN con aumento de datos	97
6.4. Matriz de confusión del modelo ANN con aumento de datos	98
6.5. Tabla comparativa de resultados entre Zape y la nariz electrónica	99
6.6. Comparativa de tiempos entre Zape y la nariz electrónica	101
6.7. Comparativa de precisión entre Zape y la nariz electrónica	102
C.1. Aplicación Serial Bluetooth Terminal en la Play Store.	131
C.2. Configuración de la pestaña Terminal.	133

C.3. Configuración de la pestaña Receive.	134
C.4. Configuración de la pestaña Send.	135
C.5. Configuración de la pestaña Misc.	136
C.6. Lista de dispositivos Bluetooth disponibles.	137
C.7. Pantalla de configuración de un botón macro.	138
D.1. Curvas de precisión (Accuracy) y pérdida (Loss) durante el entrenamiento y validación del modelo en Keras.	162
D.2. Matriz de confusión del modelo entrenado en Keras.	162
D.3. Curvas de precisión (Accuracy) y pérdida (Loss) durante el entrenamiento y validación del modelo en Keras con datos ampliados.	163
D.4. Matriz de confusión del modelo entrenado en Keras con datos ampliados. .	163
D.5. Curvas de precisión (Accuracy) y pérdida (Loss) durante el entrenamiento y validación del modelo en PyTorch.	164
D.6. Matriz de confusión del modelo entrenado en PyTorch.	165
D.7. Curvas de precisión (Accuracy) y pérdida (Loss) durante el entrenamiento y validación del modelo en PyTorch con datos ampliados.	165
D.8. Matriz de confusión del modelo entrenado en PyTorch con datos ampliados.	166

Capítulo 1

Introducción

El olfato humano es uno de los sentidos químicos fundamentales y desempeña un papel clave en la detección del entorno, la seguridad alimentaria, la memoria y la emoción [1]. A diferencia de la visión y la audición, que procesan estímulos físicos (luz y sonido), el olfato traduce señales químicas volátiles presentes en el aire en experiencias perceptivas complejas. Esta capacidad permite identificar alimentos en buen o mal estado, reconocer peligros (como humo o gases), y desencadenar respuestas afectivas y evocaciones memorísticas profundas.

Para replicar esta capacidad sensorial, se han desarrollado dispositivos conocidos como narices electrónicas (*e-noses*) [2, 3]. Una nariz electrónica es un dispositivo que se considera parte de la rama de investigación llamada biomímesis, ya que imita el sentido del olfato humano mediante el uso de sensores químicos para detectar y reconocer olores y compuestos volátiles en el aire. Estos dispositivos están diseñados para identificar patrones específicos de gases y vapores, permitiendo su aplicación en diversas áreas como la seguridad, la industria alimentaria, la medicina y el control ambiental [4].

El olfato canino, en especial, destaca por su extraordinaria sensibilidad y discriminación [5]. En comparación con los humanos, los perros poseen un epitelio olfatorio mucho más extenso y un número de receptores olfatorios significativamente mayor, lo que



amplifica su capacidad para detectar concentraciones ínfimas de compuestos volátiles. Además, su morfología nasal favorece una mayor captura y análisis de olores. Estas características, combinadas con entrenamiento y aprendizaje asociativo, permiten a los perros distinguir mezclas complejas y realizar tareas de detección en ámbitos como seguridad, búsqueda y rescate, medicina y control ambiental [6].

Por ende, este proyecto plantea estudiar la viabilidad de una nariz electrónica como herramienta de apoyo para los agentes caninos en tareas de detección de sustancias. Se explorará si la tecnología de narices electrónicas puede complementar y potenciar las capacidades olfativas de los perros, mejorando la eficiencia y precisión en la identificación de sustancias específicas en diversos contextos operativos.

1.1. Motivación

Desde una perspectiva personal, este trabajo nace de una profunda admiración por los cuerpos de seguridad y, en particular, por las unidades caninas.

Mi vínculo con los animales, especialmente con los perros, es cercano y afectivo. He convivido con ellos desde pequeño gracias a que mi abuelo siempre ha tenido varios perros en casa. Esa experiencia cotidiana, hecha de cuidado, respeto y aprendizaje, ha reforzado tanto mi cariño hacia los animales como mi interés por comprender y aprovechar científicamente sus capacidades en beneficio de la sociedad.

La motivación que guía este proyecto es, por tanto, doble: por un lado, aportar soluciones tecnológicas que apoyen el desempeño de las unidades caninas en tareas de seguridad, rescate y detección; por otro, honrar el papel de los perros como compañeros y servidores en labores críticas, integrando sensibilidad humana y rigor científico para lograr un impacto positivo.

Capítulo 2

Alcance y objetivos

En esta sección se describen los objetivos principales y específicos del proyecto, así como el alcance del mismo.

2.1. Objetivos

2.1.1. Objetivo general

El objetivo general de este proyecto es comprobar la viabilidad de una nariz electrónica para realizar un trabajo similar al de los agentes caninos de la Guardia Civil en la detección de sustancias. Con ello se pretende proporcionar una nueva herramienta tecnológica fácil de usar que pueda servir de apoyo en algunas tareas de control de sustancias, evitando una sobrecarga de trabajo para los agentes caninos y sus guías, así como proporcionando una segunda verificación si se considera oportuno.

Cabe destacar que el objetivo no es desarrollar un sistema que pueda sustituir a los agentes caninos, sino proporcionar una herramienta adicional que pueda ser útil como apoyo en ciertos escenarios específicos o como complemento a los agentes caninos cuando estos no estén disponibles.



2.1.2. Objetivos específicos

Con el fin de alcanzar el objetivo general, se han definido los siguientes objetivos específicos:

- **Creación de un *dataset*:** Recolectar y organizar un conjunto de datos espectrales representativo que incluya muestras con y sin la sustancia objetivo. Este proceso se divide en dos fases:
 - Llevar a cabo un plan de experimentación para recolectar datos espectrales de muestras con y sin la sustancia objetivo.
 - Preprocesar los datos recolectados para asegurar su calidad y adecuación para el entrenamiento de la red neuronal.
- **Desarrollo de un modelo de red neuronal:** Entrenar un modelo de red neuronal utilizando el *dataset* creado, con el objetivo de clasificar las muestras en función de la presencia o ausencia de la sustancia objetivo.
- **Creación de una aplicación móvil:** Desarrollar una aplicación móvil que permita la interacción con la nariz electrónica, facilitando la adquisición de datos y la visualización de resultados.
- **Evaluación del modelo:** Evaluar el rendimiento del modelo mediante gráficas y matrices de confusión.
- **Comparativa con los agentes caninos:** Realizar pruebas comparativas entre la nariz electrónica y los agentes caninos para evaluar la efectividad del sistema desarrollado en condiciones reales.



2.2. Alcance

El alcance de este proyecto se centra en el desarrollo de un sistema capaz de detectar la presencia de una sustancia determinada usando una nariz electrónica, similar a la función que desempeñan los agentes caninos de la Guardia Civil. La investigación no solo aborda la viabilidad tecnológica, sino que engloba todo el flujo de trabajo necesario para la implementación de una herramienta de apoyo operativa.

A continuación, se detallan las áreas fundamentales que comprende el alcance de este trabajo:

- **Investigación y Estado del Arte:** Análisis profundo de los fundamentos tecnológicos de las narices electrónicas basados en sensores de óxido metálico (MOS), así como del funcionamiento del servicio cinológico actual, identificando las sinergias posibles entre ambas disciplinas.
- **Desarrollo de un sistema basado en detección con narices electrónicas:** Creación de un sistema completo que incluye la recolección de datos, el desarrollo de modelos de aprendizaje automático y la implementación de una interfaz móvil para facilitar su uso en el exterior.
- **Validación y Comparativa:** Realización de pruebas exhaustivas para evaluar la efectividad del sistema desarrollado en comparación con los agentes caninos, asegurando que el sistema cumple con los requisitos operativos necesarios.

Sin embargo, quedan fuera de los objetivos la creación o mejora de una nariz electrónica, ya que se utilizará una ya existente. Además, el proyecto se centra exclusivamente en la detección de una sustancia específica, sin abordar la detección de múltiples sustancias. Esta sustancia ha sido seleccionada dentro de las opciones con las que cuenta la Guardia Civil.

Capítulo 3

Estado del Arte

3.1. Narices electrónicas (*e-noses*)

Anteriormente se ha descrito qué es una nariz electrónica, pero es importante definir qué características debe tener una nariz electrónica para ser considerada como tal.

Para que un dispositivo pueda considerarse una nariz electrónica debe cumplir varios requisitos fundamentales. En primer lugar, ha de ser un dispositivo capaz de detectar y diferenciar de forma simultánea múltiples compuestos volátiles; además, debe generar patrones de respuesta característicos que permitan su identificación mediante técnicas de análisis de datos.

Es imprescindible que ofrezca repetibilidad y estabilidad en sus mediciones a lo largo del tiempo y frente a distintas condiciones ambientales, y que pueda adaptarse a variaciones de temperatura, humedad y presión. Finalmente, la nariz electrónica no solo debe limitarse a proporcionar lecturas crudas de los sensores, debiendo en su lugar entregar una salida procesada (mediante preprocesado y reconocimiento de patrones) que interprete y clasifique las señales para producir identificaciones o decisiones útiles y robustas.



Las principales ventajas de las narices electrónicas recaen en su rapidez, portabilidad y gasto económico, en comparación con métodos tradicionales para ciertas aplicaciones como la detección de sustancias, o técnicas cromatográficas para controles de calidad [7, 8].

3.1.1. Antecedentes y evolución de la nariz electrónica

La base tecnológica de la nariz electrónica (*e-nose*) se encuentra en la investigación de los sensores de gas de semiconductores de óxido metálico (MOS), cuyo desarrollo se remonta a mediados del siglo XX. Un hito temprano fue la descripción de las propiedades sensibles al gas del óxido de germanio en 1953. Los primeros sensores de gas se fabricaron utilizando óxido de zinc (ZnO) [9] y, a lo largo del tiempo, la tecnología evolucionó para incorporar materiales como el dióxido de estaño (SnO_2), el dióxido de titanio (TiO_2) y el óxido de tungsteno (WO_3). De estos, el dióxido de estaño granular es el material más ampliamente estudiado y empleado en la actualidad [4, 10].

A pesar de que existen registros de sistemas de nariz electrónica reportados desde 1974 [2, 3], sus primeras aplicaciones prácticas se dieron en los laboratorios de control de calidad para el testeo de alimentos procesados [4]. La formalización y comercialización de esta tecnología se materializó a principios de los años 90 con la aparición de los sistemas de reconocimiento de aroma en el mercado. Un sistema pionero fue el Modular Sensor System (MOSES I) [11], marcando el inicio de la disponibilidad de estos instrumentos para una variedad de aplicaciones.

En la actualidad, esta tecnología ha evolucionado hasta convertirse en un instrumento capaz de reconocer y comparar olores complejos, un logro que la enmarca dentro del campo de la biomímesis al buscar imitar el olfato humano.

3.1.2. Funcionamiento

El funcionamiento de una nariz electrónica se basa en el principio de la biomímesis, buscando replicar el proceso del olfato humano mediante la combinación de hardware



(matriz de sensores) y software (sistema de reconocimiento de patrones). El objetivo fundamental es la detección de moléculas volátiles en el aire y su posterior comparación con muestras de referencia calibradas.

El sistema completo se divide en tres partes esenciales que trabajan de forma secuencial: la toma de muestra (equipo de muestreo), la matriz de sensores y el sistema de tratamiento de datos.

Ciclo de medición y detección

El proceso se inicia con la preparación de la muestra, donde los Compuestos Orgánicos Volátiles (COV) son concentrados por calentamiento en la fase de vapor sobre la muestra, lo que se conoce como muestra contaminada. Estos volátiles son introducidos en el sistema de detección a través de un ciclo de medición bien definido:

- **Fase de Adsorción (Exposición):** Los sensores se exponen directamente a la atmósfera de la muestra contaminada, permitiendo que las moléculas de olor se adsorban sobre la superficie de los sensores. Durante esta fase, los sensores registran sus respuestas continuamente.
- **Fase de Desorción (Línea Base):** A continuación, los sensores son expuestos a aire limpio. Esto permite que las moléculas de olor se desorban, lo que restablece la línea base del sensor para la siguiente medición y permite la posterior diferenciación de olores.

Los sensores de gas, principalmente los semiconductores de óxido metálico (MOS), miden las diferentes propiedades físico-químicas de los componentes del aroma. Esta interacción modifica las propiedades eléctricas del sensor, convirtiendo el olor en una señal eléctrica mensurable.



Generación de la huella olfativa

La *e-nose* requiere una matriz de múltiples sensores de gas para funcionar. Cada sensor posee una cierta especificidad para un grupo de compuestos químicos. Debido a que los olores suelen estar compuestos por más de una fuente, cada sensor de la matriz reacciona de forma única, generando un valor de conductividad individual.

El resultado de esta matriz de respuestas es una matriz de datos compleja, que constituye la “huella digital” o “huella olfativa” del olor percibido, esencial para la identificación y clasificación de los olores, ya que cada combinación de respuestas de los sensores corresponde a un aroma específico.

Procesamiento y reconocimiento de patrones

La etapa final y definitiva del sistema es el procesamiento de datos. El ordenador se encarga de procesar la matriz de datos mediante técnicas quimiométricas (estadísticas multivariadas y algoritmos de aprendizaje automático).

Métodos avanzados como el Análisis de Componentes Principales (PCA) [12], véase Figura 3.1, el Análisis de Función de Discriminantes (DFA) [13] o las Redes Neuronales Artificiales [14] son esenciales para la extracción de patrones significativos y la clasificación del olor. Es precisamente este procesamiento algorítmico lo que cumple el criterio de producir un resultado distinto a las respuestas de los sensores de gas, al permitir a la *e-nose* clasificar los diferentes elementos en función de su similitud aromática e Identificar si el olor se ajusta a un patrón conocido.

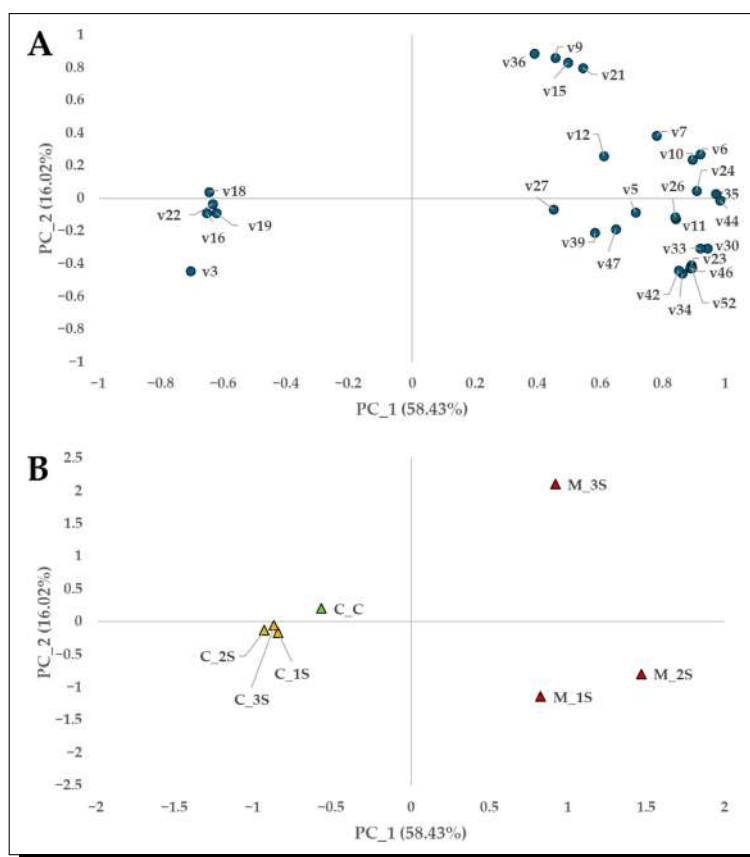
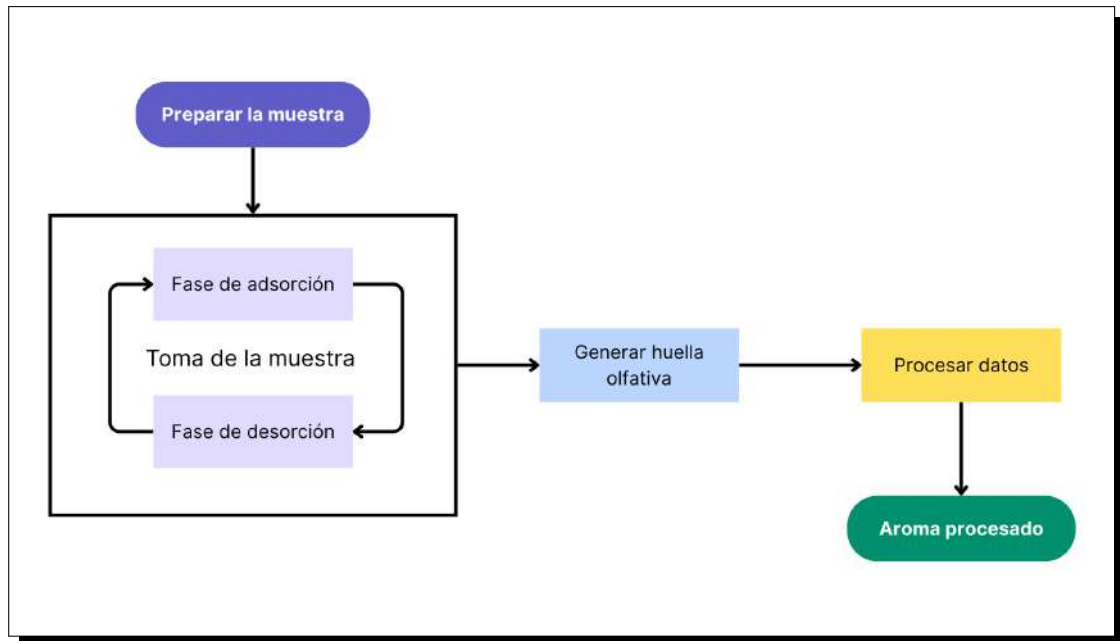


Figura 3.1: Ejemplo de un estudio de detección de *Monilinia laxa* en nectarinas mediante PCA.

De esta forma, la nariz electrónica es capaz de procesar la fracción volátil de forma global, al igual que la nariz humana, y proporciona una interpretación sobre la naturaleza del aroma.

En la Figura 3.2 se muestra un diagrama de flujo que ilustra el funcionamiento general de una nariz electrónica, desde la toma de muestra hasta la generación de resultados interpretativos mediante el procesamiento de datos.

Figura 3.2: Funcionamiento general de una *e-nose*.

3.1.3. Tipos de sensores

Para la detección de olores, las narices electrónicas emplean diversos tipos de sensores, cada uno con sus propias características y aplicaciones. Los tipos de sensores más comunes son:

- Semiconductores de óxido metálico (MOS o MOX).
- Transistores de campo eléctrico con semiconductores de óxido metálico (MOSFET).
- Conductores de polímero orgánico (CP).
- Cristales piezoeléctricos (BAW).
- Detectores de fotoionización, biosensores y nanotecnología.

La vida útil de los sensores MOSFET [15, 16] y CP [17, 18], típicamente de 1 a 3 años, se ve limitada por su alta dependencia de la temperatura y la humedad, a pesar de ofrecer una sensibilidad aceptable.



Los cristales piezoeléctricos (BAW) [19, 20], que funcionan como microbalanzas de alta precisión, ofrecen una gran sensibilidad a la masa de las moléculas adsorbidas, pero su rendimiento se ve comprometido por la humedad y la temperatura. Por otro lado, los detectores de fotoionización proporcionan una respuesta casi instantánea y son muy sensibles a compuestos orgánicos volátiles, aunque carecen de selectividad. En un intento por maximizar la especificidad, los biosensores integran elementos biológicos (como enzimas o anticuerpos), pero su fragilidad y corta vida útil limitan su aplicación práctica fuera del laboratorio. Finalmente, la nanotecnología, mediante el uso de materiales como nanotubos de carbono o grafeno [21, 22, 23], promete una sensibilidad sin precedentes y un consumo energético mínimo, si bien la reproducibilidad en su fabricación a gran escala sigue siendo un desafío.

La clase de sensores más utilizada son los sensores semiconductores de óxido metálico (MOS) [10]. De hecho, más del 35 % de los sistemas de nariz electrónica reportados desde 1974 se basan en la tecnología MOS [24]. Los sensores MOS tienen selectividad y reproducibilidad pobres, pero son poco dependientes de la humedad, lo que les da una vida útil de entre 3 y 5 años. Funcionan cambiando la conductividad eléctrica del semiconductor (como SnO_2 o TiO_2) cuando interacciona con los compuestos volátiles.

El principio radica en un proceso de quimisorción, que es la formación de enlaces químicos entre las moléculas de gas y el semiconductor. La quimisorción altera la concentración de los portadores de corriente en la superficie del semiconductor, lo que resulta en un cambio en su conductividad eléctrica. Finalmente, cuando el sensor se encuentra en una atmósfera de impurezas gaseosas reductoras, la barrera de energía disminuye. Esto, desde un punto de vista práctico, significa que la resistencia (R) disminuye cuando el sensor se expone a aire contaminado.

La resistencia del sensor puede expresarse mediante la fórmula:

$$R = R_Z \cdot \exp\left(\frac{W_G}{k_B T}\right) \quad (3.1)$$

Donde R_Z representa la resistencia interna al grano (Ω), W_G es la barrera energética (eV), k_B es la constante de Boltzmann ($eV \cdot K^{-1}$), y T es la temperatura absoluta (K).

Los sensores MOS se clasifican según la técnica utilizada para producir la capa sensible al gas. Por un lado están los sensores de película delgada (Thin-Film), cuyo espesor de la capa oscila entre 5nm y $2\mu m$. Por otra parte están los sensores de película gruesa (Thick-Film), cuyo espesor de la capa es de 10 a $300\mu m$. En comparación con los de película delgada, son menos susceptibles al envenenamiento, son más duraderos y exhiben una mayor sensibilidad.

Tipos de materiales MOS

Los sensores MOS originales estaban hechos de óxido de zinc (ZnO) y fueron seguidos por el uso de otros materiales. Los óxidos de metal que se han utilizado o propuesto se muestran en la Tabla 3.2.

Material	Aplicaciones y Notas Clave
Dióxido de Estaño (SnO_2)	El más ampliamente estudiado y utilizado. Se usa en sensores de película porosa y gruesa con una alta relación superficie-volumen.
Dióxido de Titanio (TiO_2)	Compuesto generalizado, no tóxico, barato y respetuoso con el medio ambiente. Junto con SnO_2 , puede dar lugar a sensores de alta selectividad.
Óxido de Zinc (ZnO)	Utilizado para la detección de gases, monitoreo ambiental, procesos industriales y análisis de respiración. Se utiliza para monitorear la calidad del aire interior.

Continúa en la siguiente página



Material	Aplicaciones y Notas Clave
Óxido de Tungsteno (WO_3)	Utilizado en detección de gas, monitoreo ambiental, procesos industriales y en el desarrollo de ventanas inteligentes.
Óxido de Indio (In_2O_3)	Se utiliza para monitorear y regular contaminantes como óxidos de nitrógeno (NO_x), óxidos de azufre (SO_x) y Compuestos Orgánicos Volátiles (COV).
Óxido Cúprico (CuO)	Ha sido utilizado para diversas aplicaciones como baterías, biosensores, células solares y sensores de gas.
Óxido Cuproso (Cu_2O)	Abordado menos en aplicaciones de detección de gas en comparación con CuO .

Tabla 3.2: Materiales comunes en sensores MOS y sus aplicaciones.

Estos son ejemplos comerciales, pero en el laboratorio de sensores de la Escuela de Ingenierías Industriales de Badajoz se están desarrollando sensores para usar en vez de los comerciales con una mejor capacidad de reacción a gases concretos [7].

En la Figura 3.3 se muestra un esquema de un sensor MOS típico. El sensor consta de un sustrato aislante (como alúmina o vidrio) sobre el cual se deposita una capa delgada de óxido metálico. Encima de esta capa se colocan electrodos metálicos para medir la resistencia eléctrica del óxido. Un calentador integrado mantiene la temperatura óptima para la operación del sensor, ya que la sensibilidad y selectividad dependen en gran medida de la temperatura de funcionamiento.

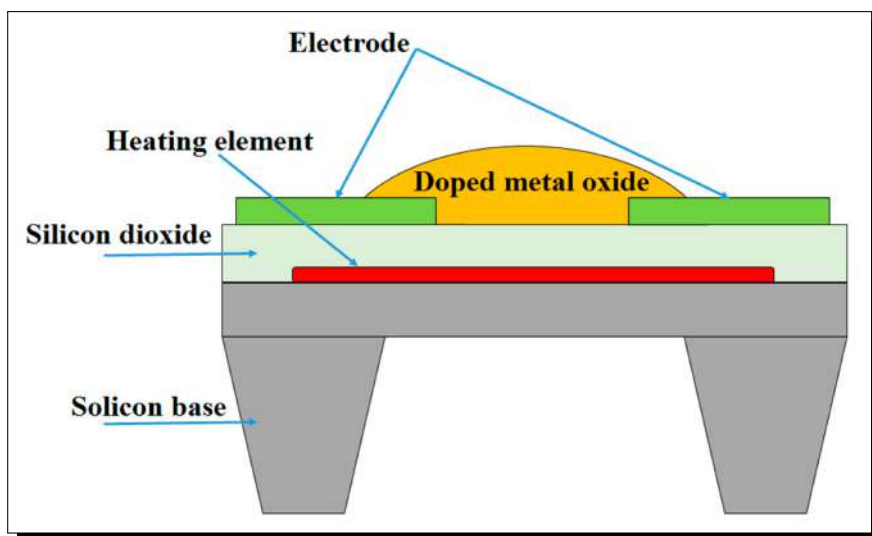


Figura 3.3: Presentación esquemática de un sensor MOX.

3.1.4. Procesamiento de datos (Quimiometría)

Una vez que la matriz de sensores de la nariz electrónica ha capturado las respuestas a los Compuestos Orgánicos Volátiles (COV), es indispensable someter los datos a un riguroso proceso de interpretación para obtener un resultado significativo. Cada sensor aporta un valor de conductividad, generando una matriz de datos compleja que debe ser analizada.

Para interpretar estos conjuntos de datos generados por la matriz de sensores, son esenciales las técnicas estadísticas y de aprendizaje automático multivariantes avanzadas. Este campo de estudio se conoce como quimiometría o sistema de reconocimiento de patrones. Mediante estas técnicas, el sistema transforma la simple respuesta eléctrica de los sensores en un resultado clasificable o identificable, cumpliendo así con uno de los criterios esenciales de una nariz electrónica.



Técnicas de Análisis de Datos

Las respuestas de las señales son analizadas mediante diversos métodos analíticos [25]. Los más reconocidos incluyen:

- **Análisis Gráfico:** Utilizado para la visualización básica de los datos (como los diagramas de barras y los diagramas polares).
- **Análisis Estadístico Multivariado:** Métodos como el Análisis de Componentes Principales (PCA) [12] se emplean comúnmente para visualizar y clasificar la estructura de los datos multivariados. Otras técnicas incluyen el Análisis de Función de Discriminantes (DFA) [13] y el Análisis de Clusters (CA).
- **Análisis de Redes:** Algoritmos avanzados de aprendizaje automático como las Redes Neuronales Artificiales [14] y las Máquinas Vectoriales de Soporte (SVM) [26, 27] han demostrado ser altamente eficaces en la extracción de patrones significativos y en la mejora de la precisión de clasificación y cuantificación.

La elección del método de análisis adecuado siempre dependerá de los datos disponibles y del tipo específico de resultado que se requiera para la aplicación.

3.1.5. Aplicaciones

Los sistemas de olfato electrónico han evolucionado desde su aplicación inicial en laboratorios de control de calidad para el análisis de alimentos [7] hasta convertirse en una técnica de análisis no destructiva y de gran aplicabilidad en diversos sectores como seguridad y defensa (detección de drogas y explosivos) [4], detección de enfermedades, control de la calidad del ambiente (por ejemplo, para detectar incendios) [8] o para diversos estudios científicos. Por ejemplo, se han llevado a cabo estudios que comparan la calidad de cervezas artesanales frente a cervezas industriales [28]. Además, las *e-noses* han demostrado su efectividad en aplicaciones agrícolas, como la detección no destructiva



de hongos en frutas frescas mediante el análisis de Compuestos Orgánicos Volátiles (VOCs) [29].

Las narices electrónicas pueden acoplarse a otros sistemas, como es el caso de los drones para captar olores en áreas de difícil acceso, dependiendo de la aplicación que se le quiera dar. Otro ejemplo es una nariz electrónica en un reloj de pulsera cuyo objetivo es medir constantemente la calidad del aire [30].

3.1.6. Desafíos y limitaciones

A pesar de sus notables ventajas en rapidez y portabilidad, las narices electrónicas aún enfrentan importantes desafíos técnicos y de estandarización que han limitado la expansión de su uso como herramienta de monitoreo.

Una de las principales limitaciones radica en los propios sensores de gas, especialmente aquellos de bajo coste. Se observa una baja selectividad, lo que significa que los sensores a menudo reaccionan a múltiples compuestos, un fenómeno conocido como sensibilidad cruzada. Esta falta de especificidad inherente a un solo sensor limita la consecución de un alto rendimiento en la detección. Para abordar esta limitación, la solución más efectiva es el enfoque multisensorial, que combina una matriz de sensores con algoritmos de aprendizaje automático para extraer patrones distintivos.

Otro problema crucial se relaciona con la dependencia ambiental, ya que la presencia de factores como la temperatura y el vapor de agua puede alterar las características del sensor [31]. Además, los sensores enfrentan problemas intrínsecos como la respuesta no lineal y la deriva de la señal, que obligan a realizar calibraciones cíclicas [32].

La estabilidad a largo plazo se ve comprometida por la susceptibilidad de los sensores a la intoxicación permanente, también llamado “envenenamiento” [25]. Este fenómeno ocurre cuando los centros activos del sensor son ocupados por contaminantes de alta concentración, causando cambios irreversibles en sus características. Para contrarrestar esta degradación y aumentar la vida útil, se recomienda reducir el tiempo de exposición



del sensor a los contaminantes, empleando un muestreo secuencial y un lavado con aire limpio.

En el ámbito de la aplicación, existe una dificultad inherente al análisis de olores compuestos por más de una fuente, ya que la identificación precisa depende no solo del tipo de sensor, sino también de la calidad de la base de datos y de los algoritmos de reconocimiento utilizados [33]. Por último, la falta de estándares específicos en la industria es una de las principales limitaciones que dificultan la expansión y homogeneización del uso de las *e-noses* como una herramienta de monitoreo confiable.

3.2. Agentes caninos

Los perros detectores representan el método tradicional y ampliamente utilizado para la detección de sustancias como drogas o explosivos [5]. Su trabajo se basa en la asombrosa capacidad de su sistema olfativo [34].

En los cuerpos de seguridad en España, el servicio canino está altamente especializado. Aunque los perros son los que huelen, el proceso es un trabajo en equipo con el guía.

El servicio cinológico cuenta con varias especialidades principales [34]:

- **Detectores de Droga:** Se encargan de la búsqueda de sustancias estupefacientes ilegales en diversos escenarios, como el control de vuelos en aeropuertos, equipajes en puertos, y dispositivos operativos en carreteras o domicilios [6].
- **Detectores de Explosivos:** Su función es vital en lugares públicos, edificios oficiales, y zonas de visita de personalidades, inspeccionando paquetería y equipajes ante amenazas reales de explosivos [35, 36].
- **Seguridad:** Incluye la protección de personas y edificios públicos, así como el control de masas.



- **Rescate:** Dedicados al salvamento de personas desaparecidas en áreas extensas, catástrofes, avalanchas, o la detección de personas ocultas en vehículos y edificios.

Además, en la actualidad, el servicio amplía su especialización a la localización de cebos envenenados, la protección de especies y la detección de tráfico de marfil.

El entrenamiento de estos perros se basa en el juego. Inicialmente, la sustancia objetivo se pone a la vista y el perro es premiado al acercarse a olerla. El entrenamiento consiste en incrementar gradualmente la complejidad del “juego”, enseñándole al perro a marcar la ubicación de la sustancia para recibir una recompensa, y usar otras sustancias similares para camuflar los olores, con las que no se le da una recompensa, para indicarle que esa sustancia no es la que tiene que buscar.

La eficacia del perro se debe a su capacidad para detectar sustancias incluso cuando se ha intentado eliminarlas [37]. Por ejemplo, pueden detectar una mancha de sangre en el suelo que ha sido limpiada con lejía u otros productos de limpieza fuertes.

A pesar de su increíble sensibilidad, el uso de perros detectores no está exento de limitaciones [5]. Por ejemplo, la actitud del perro puede variar dependiendo del estado de ánimo del guía o de la cantidad de estímulos que pueda haber en la zona, lo que puede resultar en distracción o falta de motivación para el “juego” [38]. Además, los perros requieren descansos regulares para mantener su eficacia y evitar la fatiga. Es fundamental el trabajo constante del guía para asegurar que el perro se mantenga concentrado en la tarea. Por otro lado la detección canina es un método que consume mucho tiempo, es costoso y requiere un amplio entrenamiento.

En este contexto, las *e-noses* se presentan como una tecnología con ventajas logísticas (rapidez, portabilidad y coste-efectividad) que buscan complementar los métodos tradicionales de detección canina.

Capítulo 4

Metodología y herramientas

4.1. Definición de la metodología aplicada

Desde el principio se definieron cinco fases principales:

- Reuniones iniciales, donde se decide el tema del proyecto.
- Búsqueda de información para aprender acerca de las narices electrónicas y la labor de los agentes caninos.
- Formación y entrenamiento de la nariz electrónica (*e-nose*), donde se realiza el desarrollo de todo el sistema.
- Pruebas comparativas con los agentes caninos, para evaluar la efectividad de la nariz electrónica en condiciones reales.
- Redacción final de la documentación.

Sin embargo, el contenido específico, la secuencia detallada y la carga de trabajo dentro de cada fase no podían fijarse de antemano, ya que dependen de la disponibilidad de recursos, de las personas implicadas y de la información que se vaya obteniendo durante



el desarrollo del proyecto. Por este motivo se adoptó un enfoque ligero, apoyado en metodologías ágiles (Scrum y Kanban), que facilita mantener un ritmo constante y una alta capacidad de adaptación ante cambios y necesidades emergentes.

Si bien se puede considerar a la Guardia Civil como el cliente final del proyecto, existen otros roles en este tipo de metodologías ágiles que también son relevantes, como el Product Owner (responsable de definir las prioridades y objetivos del proyecto), el Scrum Master (responsable de facilitar la implementación de la metodología ágil y eliminar obstáculos) y los desarrolladores. En este caso, dada la naturaleza académica del proyecto, el propio autor de este trabajo asume todos estos roles, gestionando tanto la planificación y ejecución del proyecto como la comunicación con los guías caninos de la Guardia Civil.

Control de versiones con Git

Para garantizar la trazabilidad y poder revertir cambios si fuera necesario, se utiliza Git como sistema de control de versiones. Esto permite mantener un historial detallado de todas las modificaciones realizadas en el código y la documentación, facilitando la gestión del proyecto.

Esto se logra a través de un [repositorio público](#) en GitHub gestionado mediante ramas, en el que se almacena todo el código fuente, documentación, información recopilada a modo de apuntes y recursos relacionados con el proyecto. También se hace uso de GitHub Desktop, una aplicación que proporciona una interfaz gráfica para gestionar repositorios Git alojados en GitHub. Esta herramienta facilita la realización de operaciones comunes de Git, como commits, push, pull y la gestión de ramas, sin necesidad de utilizar la línea de comandos.

Sprints adaptativos

Se define como sprint a cada una de las fases del proyecto anteriormente mencionadas. La duración de cada sprint se fija en función de una estimación del tiempo que se cree



necesario para lograr los objetivos establecidos para esa fase.

Antes de que inicie cada sprint (en los últimos días del sprint anterior), se hace una organización aproximada de las tareas a realizar. Al comenzar el sprint, se hace una planificación más detallada de las tareas a realizar, asignando prioridades y fijando fechas de las posibles reuniones con los implicados. Durante el sprint, se revisa periódicamente el progreso y se ajustan las tareas y prioridades según sea necesario.

Github Projects y Tablero Kanban

Para la gestión de tareas, sprints y el seguimiento del progreso del proyecto, se utiliza GitHub Projects sobre el repositorio anteriormente mencionado, que cuenta con un tablero Kanban integrado, como el que se muestra en la Figura 4.1. Este tablero permite visualizar las tareas en diferentes estados (*Backlog*, *Ready*, *In progress*, *Done*) y facilita la organización y priorización de las actividades. Cada tarea se representa como una tarjeta que puede moverse entre las columnas según su estado actual, proporcionando una visión clara del avance del proyecto.

Cada estado del tablero Kanban tiene un significado específico:

- **Backlog:** Aquí se encuentran todas las tareas que se han identificado pero que aún no están listas para ser trabajadas. Estas tareas pueden ser ideas, requisitos o cualquier actividad pendiente que necesite ser planificada.
- **Ready:** En esta columna se colocan las tareas que están listas para ser abordadas. Esto significa que se han definido claramente, tienen los recursos necesarios y están priorizadas para su ejecución en el sprint actual.
- **In progress:** Esta columna contiene las tareas que están actualmente en desarrollo. Los desarrolladores trabajan activamente en estas tareas, y su progreso se monitorea para asegurar que se completen a tiempo.
- **Done:** Aquí se mueven las tareas que han sido completadas con éxito. Esto permite



tener una visión clara de lo que se ha logrado hasta el momento y facilita la evaluación del progreso del proyecto.

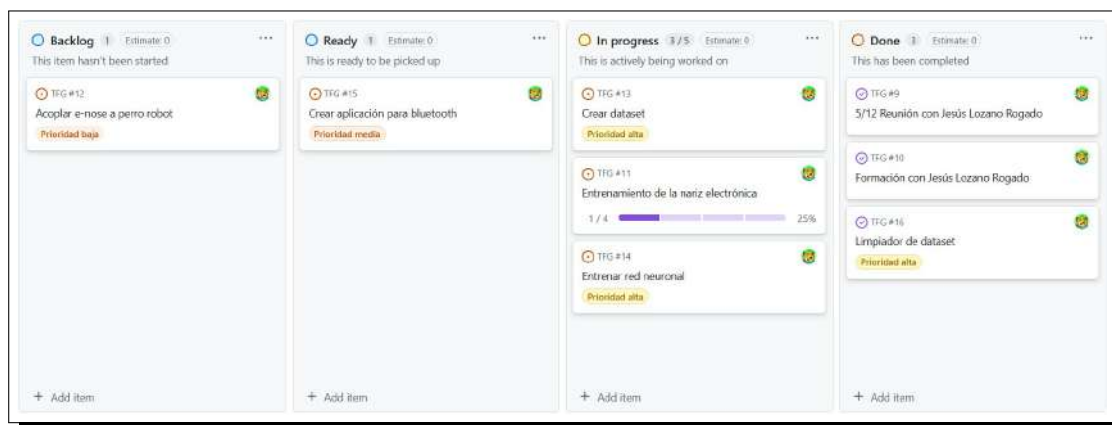


Figura 4.1: Tablero Kanban del segundo sprint.

4.2. Planificación

Pese a no saber exactamente qué tareas se van a realizar en cada sprint, se establece unas fechas aproximadas para cada sprint en base a la duración estimada de cada fase repartiendo el tiempo disponible hasta la fecha límite de entrega del proyecto. Sin embargo, estas fechas son flexibles y pueden ajustarse según las necesidades del proyecto y el progreso real alcanzado en cada fase, el cual puede verse afectado por diversos factores externos. De esta forma es posible distinguir una planificación inicial (fechas aproximadas) y una planificación final (fechas definitivas).

4.2.1. Planificación inicial

Como se ha comentado anteriormente, esta planificación inicial se basa en una estimación del tiempo necesario para completar cada fase del proyecto dentro del tiempo límite, además de tener una semana de margen para posibles retrasos o imprevistos. Esta planificación inicial se muestra en la Tabla 4.1.



Duración	Sprint	Descripción
06/11/2025–14/11/2025	Sprint 1: Reuniones iniciales y elección del tema	Primera reunión con los guías caninos y decisión del tema del proyecto.
20/11/2025–04/12/2025	Sprint 2: Revisión bibliográfica y estado del arte	Recopilación de información necesaria para el proyecto y redacción del marco teórico.
05/12/2025–10/01/2026	Sprint 3: Formación y entrenamiento de la nariz electrónica	Cuerpo principal del proyecto. Entrenamiento de una ANN para que detecte hachís y desarrollo de una aplicación para controlar la nariz electrónica y que use esta red neuronal.
11/01/2026–18/01/2026	Sprint 4: Diseño del enfoque experimental, comparación y análisis de datos	Realización de pruebas para comparar la nariz electrónica con el olfato canino.
19/01/2026–31/01/2026	Sprint 5: Redacción final del TFG	Finalización de la memoria y corregir los posibles errores.

Tabla 4.1: Planificación inicial de sprints.

La fecha límite es el 6 de febrero de 2026, por lo que se deja una semana de margen al final del proyecto para posibles retrasos o imprevistos. Además, hay un sexto sprint dedicado a la preparación de la presentación del TFG, el cual abarca desde el día en el que se entrega el proyecto hasta la fecha de la defensa. Este sprint no se incluye en la tabla ya que no afecta a la planificación del proyecto en sí.

El diagrama de Gantt de la Figura 4.2 muestra una representación visual de esta planificación inicial, con las fechas aproximadas para cada fase del proyecto.

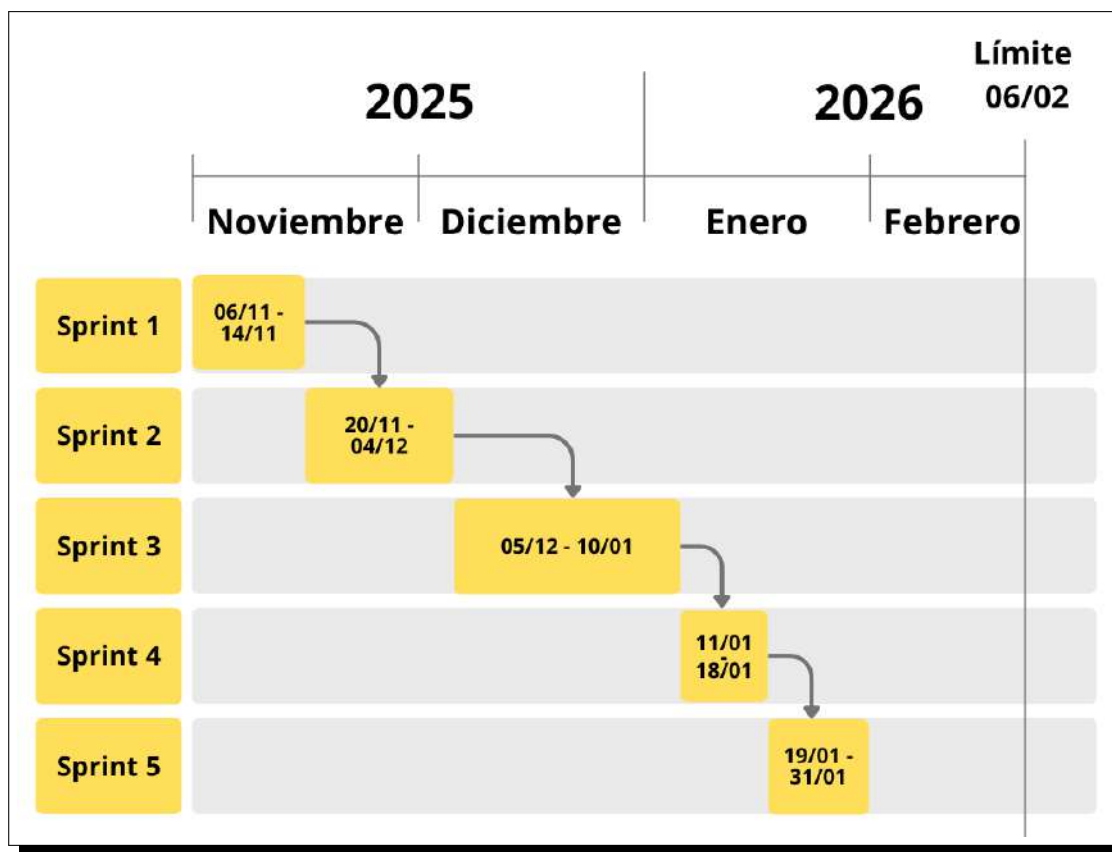


Figura 4.2: Diagrama de Gantt de la planificación inicial.

4.2.2. Planificación final

La planificación inicial es una planificación ideal en la que se cumplen todas las fechas establecidas. Sin embargo, en la práctica, es posible que surjan imprevistos o retrasos que hagan que no se pueda terminar lo establecido en un sprint dentro de su tiempo estimado.

Estos retrasos pueden verse reflejados en la Figura 4.3, donde se muestra una hoja de cálculo en la que se ha llevado un control de las fechas teóricas y reales de cada tarea, así como el estado de cada una de ellas y su respectivo desfase (positivo si se ha acabado antes de lo previsto, negativo si se ha acabado después de lo previsto).

Tarea	Inicio teórico	Fin teórico	Inicio real	Fin real	Desfase	Estado
Reunión con guías caninos	06/11/2025	06/11/2025	06/11/2025	06/11/2025	0	Completado
Elección definitiva de tema del TFG	06/11/2025	14/11/2025	06/11/2025	14/11/2025	0	Completado
Revisión de literatura sobre narices electrónicas	20/11/2025	21/11/2025	20/11/2025	21/11/2025	0	Completado
Revisión de trabajos de Jesús Lozano Rogado	24/11/2025	26/11/2025	24/11/2025	26/11/2025	0	Completado
Revisión sobre detección canina	27/11/2025	28/11/2025	27/11/2025	28/11/2025	0	Completado
Redacción del marco teórico	01/12/2025	04/12/2025	01/12/2025	02/12/2025	2	Completado
Reunión con Jesús Lozano Rogado	05/12/2025	05/12/2025	05/12/2025	05/12/2025	0	Completado
Formación en laboratorio de sensores	09/12/2025	09/12/2025	09/12/2025	09/12/2025	0	Completado
Crear dataset	10/12/2025	19/12/2025	11/12/2025	25/01/2026	-37	Completado
Limpiador dataset	22/12/2025	22/12/2025	18/12/2025	18/12/2025	4	Completado
Ampliar dataset	23/12/2025	23/12/2025	19/12/2025	19/12/2025	4	Completado
Entrenar red neuronal	24/12/2025	26/12/2025	02/01/2026	25/01/2026	-30	Completado
Crear aplicación bluetooth	29/12/2025	06/01/2026	03/01/2026	25/01/2026	-19	Completado
Acoplar e-nose a perro robot	07/01/2026	09/01/2026				Descartado
Acordar reunión	10/01/2026	10/01/2026	25/01/2026	25/01/2026	-15	Completado
Definir pruebas comparativas	12/01/2026	12/01/2026	26/01/2026	02/02/2026	-21	Completado
Pruebas comparativas	13/01/2026	15/01/2026	02/02/2026	02/02/2026	-18	Completado
Discusión de resultados	16/01/2026	16/01/2026	02/02/2026	02/02/2026	-17	Completado
Finalizar redacción	19/01/2026	23/01/2026	02/02/2026	02/02/2026	-10	Completado
Correcciones finales	26/01/2026	31/01/2026	03/02/2026	06/02/2026	-6	Completado

Figura 4.3: Seguimiento de las tareas del proyecto.

Sprint 1: Reuniones iniciales y elección del tema

Duración: 6 de noviembre–14 de noviembre de 2025 (Completado según lo planificado).

En primer lugar se llevó a cabo una reunión con los guías caninos de la Guardia Civil para discutir posibles temas para el TFG. Tras evaluar varias opciones, se decidió centrar el proyecto en la comparación entre una nariz electrónica y agentes caninos para la detección de hachís. Esta decisión se tomó teniendo en cuenta los recursos disponibles, la relevancia del tema y el interés personal del autor. Tras consultarlo con el cotutor del proyecto, Jesús Lozano Rogado (experto en narices electrónicas) se consideró un trabajo viable y factible dentro del tiempo y recursos disponibles.

Sprint 2: Revisión bibliográfica y estado del arte

Duración: 20 de noviembre–4 de diciembre de 2025 (Completado según lo planificado).



Este sprint se dedicó a la recopilación de información relevante sobre narices electrónicas, detección canina y trabajos previos relacionados con el tema del TFG. Se revisaron artículos científicos (algunos de Jesus Lozano), libros y otros recursos para obtener una comprensión sólida del estado del arte en este campo. Esta información se utilizó para redactar el marco teórico del proyecto, que proporciona el contexto necesario para entender la importancia y los desafíos de la comparación entre narices electrónicas y agentes caninos en la detección de sustancias.

Sprint 3: Formación y entrenamiento de la nariz electrónica

Duración: 5 de diciembre–25 de enero de 2026 (Retraso).

Motivo del retraso: Problemas de funcionamiento de la nariz electrónica para la toma de muestras y cambio de enfoque en el entrenamiento de la ANN.

Este es el sprint más largo y complejo del proyecto, ya que implica el desarrollo del producto principal.

Pese a haber recopilado información acerca de las narices electrónicas y su funcionamiento, era necesaria una explicación más detallada sobre el uso. Por ello, este sprint comienza con una reunión con el Dr. Jesús Lozano Rogado para acordar un día en el que poder visitar el laboratorio de sensores de la Escuela de Ingenierías Industriales de Badajoz y recibir una formación práctica por parte de Cristina Brugera Salas sobre el uso de la nariz electrónica diseñada y construida por ella. Esta formación fue fundamental para entender cómo utilizar la nariz electrónica de manera efectiva y terminar de enfocar el rumbo que debía tomar el proyecto.

Tras la formación se decidió que el “entrenamiento de la nariz electrónica” consistiría en el entrenamiento de una red neuronal artificial (ANN) para que, a partir de los datos obtenidos por la nariz electrónica al analizar diferentes muestras, pudiera detectar la presencia de hachís. Este entrenamiento se dividió en cinco subtarefas principales:

1. Toma de muestras con la nariz electrónica para crear el *dataset*.



2. Creación de un script para la limpieza y preprocesamiento de los datos obtenidos.
3. Creación de un script para ampliar el *dataset* mediante técnicas de aumento de datos.
4. Desarrollo y entrenamiento de la ANN.
5. Desarrollo de una aplicación móvil para controlar la nariz electrónica y utilizar la ANN entrenada.

Por otra parte se vio la posibilidad de acoplar la nariz electrónica a un perro robot en caso de que sobrase tiempo (prioridad baja). Sin embargo, debido a retrasos acumulados, esta tarea se ha descartado para centrarse en completar las tareas principales del sprint.

Sprint 4: Diseño del enfoque experimental, comparación y análisis de datos

Duración: 25 de enero–2 de febrero (Retraso)

Motivo del retraso: Retraso acumulado y no disponibilidad del guía hasta el 2 de febrero.

El mismo día que se finalizó el sprint 3, se contactó con el guía canino para planificar las pruebas comparativas entre la nariz electrónica y los agentes caninos. Sin embargo, debido a la agenda del guía, no fue posible realizar estas pruebas hasta el 2 de febrero de 2026. Por lo tanto, este sprint se ha dedicado principalmente a preparar el diseño experimental y analizar los datos obtenidos durante las pruebas, que se llevarán a cabo en la fecha mencionada.

Sprint 5: Redacción final del TFG

Duración: 2 de febrero–6 de febrero (Retraso)

Motivo del retraso: Retraso acumulado.



La memoria del proyecto se ha ido redactando de forma progresiva a lo largo de los sprints, aprovechando los tiempos de espera y los momentos en los que no se podía avanzar en otras tareas. Sin embargo, este sprint está dedicado a finalizar la redacción y corregir los posibles errores o inconsistencias en el documento final.

El diagrama de Gantt de la Figura 4.4 muestra la planificación final del proyecto, con las fechas definitivas para cada fase, teniendo en cuenta los retrasos del proyecto.

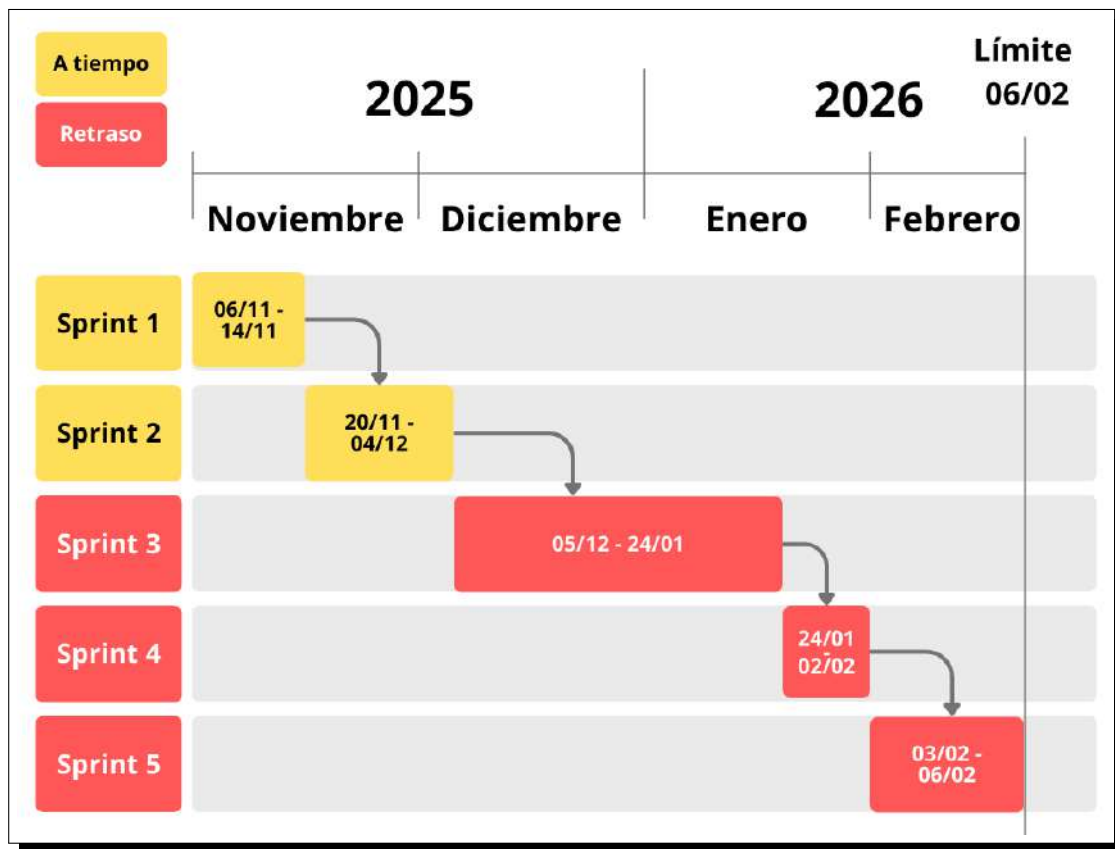


Figura 4.4: Diagrama de Gantt de la planificación final.

4.3. Herramientas utilizadas

Dada la naturaleza del proyecto, es posible distinguir entre dos tipos de herramientas: herramientas hardware y herramientas software.

4.3.1. Herramientas hardware

Aquí se encuentra la principal herramienta para este estudio: la nariz electrónica utilizada para la toma de datos de las distintas muestras y, posteriormente, para la realización de las pruebas con la ANN y la nueva aplicación móvil.



Figura 4.5: Carcasa de la nariz electrónica.

En la Figura 4.5 se puede observar dicha nariz electrónica desde una vista exterior (la carcasa), donde están los orificios de entrada y salida de aire. La “verdadera” nariz electrónica es una placa PCB, diseñada y programada en el laboratorio de sensores de la Escuela de Ingenierías Industriales de Badajoz, que se encuentra en el interior de la carcasa. En esta placa es donde se encuentran los sensores de gases y el microcontrolador PIC32MM0256GPM048, además de contar con un módulo Bluetooth Rn4871 para la comunicación con una aplicación móvil que la controla, y una entrada microUSB para poder conectarla a la corriente y cargar la batería con la que cuenta. Toda esta placa se muestra en la Figura 4.6.

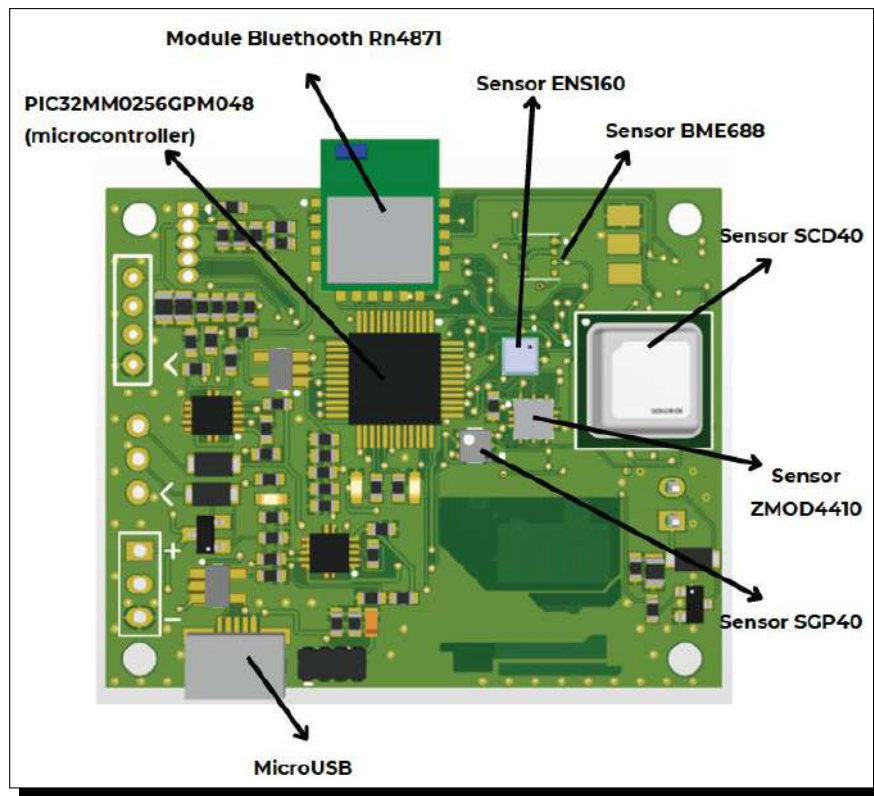


Figura 4.6: Placa PCB de la nariz electrónica.

El conjunto de sensores que componen esta nariz electrónica es el siguiente:

- **SCD40** (PA): Sensor de CO₂, temperatura y humedad, fabricado por Sensirion.
- **SGP40** (MOS): Sensor de COV, fabricado por Sensirion.
- **BME688** (MOS): Sensor de presión, temperatura, humedad, COV, VSC y gas, fabricado por Bosch.
- **ENS160** (MOS): Sensor de COV, calidad del aire, VOX y NO₂, fabricado por ScioSense.
- **ZMOD4410** (MOS): Sensor de COV y CO₂, fabricado por Renesas.

4.3.2. Herramientas software

Toma de muestras

Para la fase de toma de muestras, en conjunto con la nariz electrónica mencionada en el apartado anterior, se ha utilizado una aplicación móvil que permite controlar la nariz electrónica a través de Bluetooth, enviando comandos a través de la aplicación, y recogiendo los datos obtenidos por los sensores. Esta aplicación facilita la configuración de los parámetros de medición, el inicio y la detención de las pruebas, así como el almacenamiento y exportación de los datos recopilados para su posterior análisis. En la Figura 4.7 se muestra una captura de pantalla de la interfaz de la aplicación móvil mencionada.

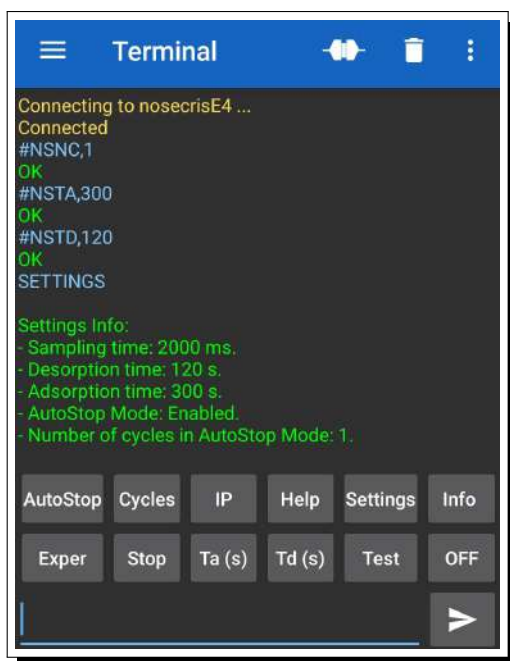


Figura 4.7: Interfaz de la aplicación móvil para el control de la toma de muestras.

Limpieza y ampliación del *dataset*

Para la limpieza y preprocesamiento de los datos obtenidos durante la fase de toma de muestras, se busca crear un script que automatice este proceso y facilite la preparación



del *dataset* para su posterior análisis y entrenamiento de la ANN. Además, también se busca crear otro script para la ampliación del *dataset*. Existen varias posibilidades para llevar a cabo estas tareas, entre las que se incluyen:

- **Python:** Python es un lenguaje de propósito general altamente versátil que se ha convertido en el estándar *de facto* para la ciencia de datos y el análisis de datos. Su sintaxis legible y el abundante ecosistema de librerías especializadas (como NumPy, Pandas, Scikit-learn) lo hacen ideal para tareas de preprocesamiento y *data augmentation*. NumPy permite realizar operaciones vectorizadas eficientes sobre arrays de datos, mientras que Pandas proporciona estructuras de datos poderosas como DataFrames para manipular y transformar datos tabulares. La curva de aprendizaje es suave, permitiendo escribir código legible y mantenible rápidamente.
- **R:** R es un lenguaje estadístico especializado en análisis de datos e investigación científica. Dispone de paquetes muy maduros como dplyr (para manipulación de datos) y caret (para preprocesamiento y aumento de datos). Es particularmente potente para análisis estadísticos complejos y visualización de datos mediante ggplot2. Sin embargo, R puede resultar más lento que Python en operaciones intensivas, especialmente con grandes volúmenes de datos. Su curva de aprendizaje es más pronunciada para usuarios sin experiencia estadística, aunque es muy poderoso una vez se domina.
- **Bash + herramientas UNIX:** Las herramientas de línea de comandos UNIX (grep, awk, sed, cut, join) permiten realizar transformaciones de datos directamente en archivos de texto mediante tuberías (pipes). Esto es extremadamente eficiente en términos de consumo de memoria, ya que procesa el archivo línea a línea sin cargar todo en memoria. Es especialmente útil para el filtrado, extracción de columnas y transformación de datos. Sin embargo, para operaciones más complejas como normalización estadística o aumento de datos basado en algoritmos sofisticados, resulta tedioso y propenso a errores. Requiere conocimiento profundo de las herramientas UNIX y puede afectar la reproducibilidad si no se documentan bien



los comandos.

- **SQL + scripts:** Si los datos se encuentran en una base de datos relacional, SQL es una opción poderosa para filtrado, agregación y transformación de datos. Lenguajes como SQL con Python (ejecutando consultas SQL desde scripts de Python) combinan la potencia declarativa de SQL con la flexibilidad de un lenguaje de programación. Sin embargo, esto requiere que los datos estén en una base de datos, añadiendo complejidad infraestructural. Además, el aumento de datos generativo, como la síntesis de nuevas muestras es más difícil de implementar usando solo SQL.
- **Apache Spark:** Apache Spark es un framework de procesamiento distribuido diseñado para trabajar con Big Data. Proporciona APIs en Python (PySpark), R (SparkR) y SQL. Es tremendamente escalable y puede procesar datos que no caben en una sola máquina distribuyéndolos en un clúster. Sin embargo, Spark tiene un *overhead* significativo de inicialización y sincronización entre nodos, lo que lo hace ineficiente para *datasets* pequeños (típicamente menor a 1 GB). Es más apropiado para pipelines de datos en producción con millones de registros distribuidos.

A continuación, en la Tabla 4.3 se presenta una comparativa de estas herramientas considerando criterios relevantes para este proyecto.



Criterio	Ventajas	Desventajas	Idoneidad
Python	■ Ecosistema completo (NumPy, Pandas, Scikit-learn) ■ Sintaxis clara y fácil de aprender ■ Múltiples librerías de <i>data augmentation</i> ■ Reproducibilidad y documentación excelente ■ Comunidad muy activa ■ Versatilidad para otros usos del proyecto	■ Rendimiento más bajo con Big Data ■ Requiere bibliotecas externas para casi todo	Muy alta. Ideal para <i>datasets</i> pequeños-medianos con requerimientos complejos.
R	■ Diseño especializado en estadística ■ Paquetes estadísticos muy maduros (dplyr, caret) ■ Gráficas y análisis exploratorio avanzados ■ Comunidad científica sólida	■ Más lento en operaciones intensivas ■ Curva de aprendizaje pronunciada ■ Comunidad menos grande que Python	Media. Bueno si los requerimientos son principalmente estadísticos.

Continúa en la siguiente página



Criterio	Ventajas	Desventajas	Idoneidad
Bash + UNIX	■ Máxima eficiencia de memoria ■ No requiere nuevos lenguajes si ya se domina ■ Extremadamente rápido para transformaciones simples	■ Difícil de mantener para operaciones complejas ■ Baja reproducibilidad ■ Operaciones avanzadas (ML, normalización) muy tediosas	Baja. Solo útil para transformaciones triviales.
SQL + scripts	■ Excelente para filtrado y agregación en BDs ■ Declarativo e intuitivo ■ Escalabilidad mediante RDBMS optimizadas	■ Requiere infraestructura de base de datos ■ <i>data augmentation</i> generativa compleja ■ Mezcla de lenguajes dificulta mantenimiento	Baja-Media. Aplicable si los datos están en una BD, pero requiere lenguajes auxiliares.
Apache Spark	■ Escalabilidad a Big Data ■ Procesamiento distribuido eficiente ■ Múltiples lenguajes (PySpark, SparkR, SQL) ■ Optimizaciones automáticas	■ <i>Overhead</i> significativo para datos pequeños ■ Curva de aprendizaje pronunciada ■ Requiere clúster/cloud (costo infraestructural) ■ Complejidad operacional	Baja para este proyecto. Excesivo para <i>datasets</i> pequeños-medianos.

Tabla 4.3: Comparativa de herramientas para limpieza y ampliación del *dataset*.

Dado que se tratan de unos scripts sencillos, se ha optado por utilizar Python para la limpieza y ampliación de dichos datos. Esta elección se basa en la familiaridad del



autor con el lenguaje, ya que posee experiencia en Python. Además, Python dispone de librerías especializadas como Pandas para manipulación de DataFrames o NumPy para operaciones vectorizadas. Asimismo, existe una integración natural con otras fases del proyecto, ya que Python se usa también en el entrenamiento de la ANN, facilitando un flujo de trabajo coherente. Finalmente, el preprocesamiento requiere complejidad moderada, siendo necesario realizar normalización, manejo de valores faltantes, detección de outliers y síntesis de datos sintéticos, tareas donde Python brilla por su versatilidad y potencia.

Entrenamiento de la ANN

Lenguajes de programación Para el desarrollo y entrenamiento de redes neuronales existen múltiples lenguajes de programación, cada uno con características distintivas:

- **Python:** Es un lenguaje de propósito general que se ha convertido en el estándar *de facto* para Machine Learning y Deep Learning. Ofrece un excelente ecosistema de librerías especializadas (TensorFlow, PyTorch, Keras), es sencillo de aprender y utilizar, cuenta con una amplia comunidad y soporte, proporciona gran flexibilidad para prototipado rápido, y el autor posee familiaridad significativa con el lenguaje. Sin embargo, presenta un rendimiento más lento comparado con lenguajes compilados.
- **C++:** Es un lenguaje compilado que destaca por su velocidad inigualable de ejecución y gestión manual de memoria, permitiendo mayor control sobre los recursos del sistema. Sin embargo, presenta una curva de aprendizaje muy pronunciada, es más complejo de desarrollar, y cuenta con menos librerías de alto nivel especializadas en Machine Learning.
- **Julia:** Es un lenguaje científico de reciente desarrollo que proporciona gran velocidad de ejecución comparable a C++, resolviendo el “problema de los dos lenguajes”, en el que se usa lenguaje de alto nivel para escribir código y lenguaje de bajo nivel para la ejecución, permitiendo investigar y producir en el mismo entorno sin pérdida de



velocidad. Está específicamente diseñado para computación científica. Sin embargo, su comunidad es más pequeña que la de Python, cuenta con menos librerías de ML maduras, y su ecosistema aún está en desarrollo.

- **Mojo:** Es un lenguaje experimental que combina la sintaxis de Python con el rendimiento de C++, estando específicamente diseñado para hardware de IA (TPUs y GPUs). A pesar de estas ventajas, es un lenguaje muy nuevo con pocos precedentes, comunidad incipiente, y librerías limitadas.

A continuación, en la Tabla 4.5, se presenta una comparativa de estos lenguajes considerando criterios relevantes para este proyecto.

Lenguaje	Ventajas	Desventajas	Idoneidad
Python	■ Buen ecosistema y librerías ■ Sencillo de aprender y utilizar ■ Amplia comunidad y soporte ■ Flexibilidad y prototipado ■ Familiaridad con el lenguaje	■ Rendimiento más lento comparado con lenguajes compilados	Muy alta. Estándar <i>de facto</i> para Machine Learning/Deep Learning.
C++	■ Velocidad inigualable de ejecución ■ Gestión manual de memoria ■ Mayor control de recursos	■ Curva de aprendizaje muy pronunciada ■ Complejidad en desarrollo ■ Menos librerías de alto nivel para ML	Baja. Solo si se necesita máximo rendimiento.

Continúa en la siguiente página



Lenguaje	Ventajas	Desventajas	Idoneidad
Julia	■ Gran velocidad de ejecución ■ Resuelve el problema de dos lenguajes ■ Diseñado para computación científica	■ Comunidad más pequeña que Python ■ Menos librerías de ML maduras ■ Ecosistema más joven	Media. Bueno para investigación, pero menos maduro.
Mojo	■ Sintaxis Python con rendimiento C++ ■ Diseñado para IA/ML ■ Optimizado para TPUs y GPUs	■ Muy nuevo, pocos precedentes ■ Comunidad incipiente ■ Librerías limitadas	Baja para este proyecto. Demasiado experimental.

Tabla 4.5: Comparativa de lenguajes de programación para entrenamiento de ANN.

Se ha elegido Python (versión 3.12) como lenguaje principal debido a su amplio dominio en la comunidad de Machine Learning, el excelente ecosistema de librerías especializadas, la familiaridad del autor con el lenguaje, y la abundancia de recursos y documentación disponibles.

Entornos de ejecución Existen varias plataformas para ejecutar el entrenamiento de modelos, proporcionando diferentes características y capacidades:

- **Orange/KNIME:** Son plataformas que permiten crear tuberías de datos completas (limpieza, entrenamiento y evaluación) de forma visual sin necesidad de código. Ofrecen una interfaz intuitiva y son ideales para exploración inicial de datos. Sin embargo, son menos flexibles que el código directo, tienen limitaciones para modelos muy complejos, y ofrecen control limitado sobre parámetros específicos.
- **Kaggle Kernels:** Proporciona un entorno en la nube listo para ejecutar *notebooks*



con acceso gratuito a GPUs y TPUs. Cuenta con una comunidad muy activa con competiciones regulares, está enfocado en ciencia de datos, y ofrece *datasets* listos para usar. Sin embargo, tiene límites de tiempo de ejecución, ofrece menos control sobre hardware, y está más orientado a competiciones que a investigación general.

- **Google Colab:** Ofrece un entorno en la nube basado en *notebooks* Jupyter con acceso gratuito a GPUs y TPUs de Google. Permite integración directa con Google Drive para almacenar *notebooks*, modelos y datos, es fácil de compartir y colaborar en proyectos, y es suficiente para *datasets* pequeños-medianos. Las desventajas incluyen que las sesiones se desconectan si no se usan, hay cuotas limitadas de acceso a GPU, y existe latencia en la comunicación de red.

A continuación, en la Tabla 4.7, se presenta una comparativa de estos entornos.

Entorno	Ventajas	Desventajas	Idoneidad
Orange/KNIME	■ Interfaz visual intuitiva ■ No requiere código ■ Tuberías completas de datos ■ Bueno para exploración	■ Menos flexible que código directo ■ Limitaciones para modelos complejos ■ Control limitado sobre parámetros	Baja. Mejor para prototipado rápido.
Kaggle Kernels	■ Acceso gratuito a GPUs/TPUs ■ Comunidad activa con competiciones ■ <i>datasets</i> listos para usar ■ Reproducibilidad facilitada	■ Tiempo de ejecución limitado ■ Menos control sobre hardware ■ Más orientado a competiciones	Media. Bueno para competiciones.

Continúa en la siguiente página



Entorno	Ventajas	Desventajas	Idoneidad
Google Colab	■ Acceso gratuito a GPUs/TPUs de Google ■ Integración con Google Drive ■ <i>Notebooks</i> Jupyter nativos ■ Fácil de compartir y colaborar ■ Suficiente para <i>datasets</i> pequeños-medianos	■ Sesiones se desconectan si no se usan ■ Cuotas limitadas de acceso a GPU ■ Latencia en red	Muy alta. Ideal para este proyecto.

Tabla 4.7: Comparativa de entornos de ejecución para entrenamiento de ANN.

Se ha optado por Google Colab como entorno de ejecución, ya que proporciona acceso gratuito a GPUs y TPUs, facilitando el entrenamiento de modelos complejos sin necesidad de hardware potente en local. Además, permite guardar *notebooks*, modelos y *datasets* directamente en Google Drive, garantizando persistencia y facilitando el acceso desde diferentes dispositivos.

Frameworks Existen varios frameworks especializados para el desarrollo y entrenamiento de redes neuronales con características distintivas:

- **PyTorch:** Es un framework basado en gráficos dinámicos que permite cambiar el modelo en pleno entrenamiento, facilitando la experimentación. Proporciona facilidad de depuración mediante herramientas estándar de Python, cuenta con un ecosistema enorme de librerías complementarias, posee una comunidad muy activa y especializada, y el autor tiene familiaridad con este framework. Sin embargo, está menos optimizado para producción comparado con TensorFlow, y presenta una curva de aprendizaje pronunciada para principiantes.



- **TensorFlow/Keras:** Está diseñado principalmente para producción y despliegue en aplicaciones reales. Ofrece una interfaz de alto nivel muy intuitiva que facilita la creación de modelos, cuenta con un ecosistema móvil robusto (TensorFlow Lite), permite crear redes complejas en pocas líneas de código, y tiene amplia documentación y tutoriales. Las desventajas incluyen que es menos flexible que PyTorch, la depuración es más complicada, y la comunidad está menos orientada hacia investigación experimental.
- **JAX:** Es un framework de bajo nivel para computación numérica con diferenciación automática, construido sobre NumPy. Proporciona flexibilidad extrema y es muy usado en investigación científica e IA. El enfoque funcional permite optimizaciones avanzadas y JIT *compilation* para mayor velocidad. Sin embargo, requiere más código para tareas simples, la comunidad es más pequeña que PyTorch/TensorFlow, y tiene menos librerías de alto nivel preimplementadas.
- **PyTorch Lightning:** Es una abstracción sobre PyTorch que simplifica el código de entrenamiento, mejorando la legibilidad y reproducibilidad. Separa la lógica del modelo de la lógica de entrenamiento, facilita el uso de GPUs/TPUs sin cambios de código, y proporciona un framework estándar para el desarrollo. Las desventajas son que añade una capa de abstracción que puede ser excesiva para prototipos rápidos, y reduce la flexibilidad en ciertos casos de uso muy específicos.

A continuación, en la Tabla 4.9, se presenta una comparativa de estos frameworks.



Framework	Ventajas	Desventajas	Idoneidad
PyTorch	■ Gráficos dinámicos permiten cambios durante entrenamiento ■ Facilidad de depuración con herramientas Python ■ Ecosistema enorme ■ Comunidad muy activa ■ Familiaridad del autor	■ Menos optimizado para producción que TensorFlow ■ Curva de aprendizaje pronunciada	Muy alta. Excelente para investigación.
TensorFlow/Keras	■ Interfaz de alto nivel muy intuitiva ■ Excelente para producción ■ Ecosistema móvil robusto ■ Redes complejas en pocas líneas ■ Amplia documentación	■ Menos flexible que PyTorch ■ Depuración más complicada ■ Comunidad menos orientada a investigación	Muy alta. Bueno para producción y despliegue.

Continúa en la siguiente página



Framework	Ventajas	Desventajas	Idoneidad
JAX	■ Flexibilidad extrema ■ JIT <i>compilation</i> para velocidad ■ Diferenciación automática avanzada ■ Ideal para investigación ■ Enfoque funcional permite optimizaciones	■ Requiere más código para tareas simples ■ Comunidad más pequeña ■ Menos librerías de alto nivel ■ Curva de aprendizaje empinada	Media. Excelente para investigación avanzada.
PyTorch Lightning	■ Simplifica código de entrenamiento ■ Mejora reproducibilidad ■ Separación clara de responsabilidades ■ GPU/TPU sin cambios de código ■ Estándares de desarrollo	■ Abstracción excesiva para prototipos rápidos ■ Reduce flexibilidad en casos específicos ■ <i>Overhead</i> para proyectos sencillos	Alta. Bueno para proyectos estructurados.

Tabla 4.9: Comparativa de frameworks para entrenamiento de ANN.

Se han elegido PyTorch y Keras como frameworks principales para este proyecto. Se entrenarán modelos con ambos frameworks utilizando los mismos datos y parámetros para realizar una comparación rigurosa y determinar cuál se adapta mejor a las necesidades específicas del proyecto. Se utilizarán librerías complementarias como Scikit-Learn para preprocesamiento, Pandas para manipulación de datos y NumPy para operaciones numéricas.



Aplicación Bluetooth

Para el desarrollo de la aplicación móvil que se comunicará con la nariz electrónica a través de Bluetooth, se han evaluado diferentes opciones de frameworks y plataformas. A continuación se presentan las herramientas consideradas:

- **Flutter:** Framework multiplataforma que compila a código nativo, permitiendo crear aplicaciones para iOS y Android con una única base de código. Ofrece estabilidad en la comunicación Bluetooth mediante bibliotecas como *flutter_blue_plus*, soporte para cargar modelos de TensorFlow Lite directamente desde assets, reactividad en la interfaz mediante Streams, y capacidad de ejecutar procesos intensivos en Isolates sin congelar la UI. Ideal para aplicaciones en tiempo real con restricciones de latencia.
- **React Native:** Plataforma *web-first* que permite código compartido entre iOS y Android. Facilita la gestión de lógica compleja mediante *hooks*, selector dinámico de modelos, y temporizadores. Sin embargo, la configuración de permisos iniciales (Bluetooth, *storage*) es tediosa y la latencia introducida por el puente JavaScript-nativo puede ser problemática para comunicaciones de tiempo real.
- **Kotlin:** Lenguaje nativo de Android que proporciona control total sobre APIs de Bluetooth (BluetoothGatt), latencia predecible, y máxima eficiencia energética mediante integración con TensorFlow Lite nativo. Requiere desarrollo separado para iOS y está limitado a ecosistema Android, aumentando complejidad de mantenimiento.
- **Capacitor:** Framework que actúa como puente entre tecnologías web (Angular, React, Vue) y APIs nativas mediante plugins. Permite ejecutar modelos con TensorFlow.js en capa web y acceder a APIs nativas de Bluetooth, pero introduce latencia adicional por su arquitectura basada en capas web, limitando viabilidad para procesamiento en tiempo real de flujos sensoriales continuos.



A continuación, en la Tabla 4.11, se presenta una comparativa de estas herramientas.

Herramienta	Ventajas	Desventajas	Idoneidad
Flutter	■ Compilación a código nativo en iOS/Android ■ Latencia baja mediante Streams nativos ■ Carga eficiente de modelos TensorFlow Lite ■ Reactividad de UI sin congelación ■ Única base de código multiplataforma	■ Ecosistema más joven que React Native ■ Comunidad más pequeña para Bluetooth específicamente ■ Requiere aprendizaje de Dart	Muy alta. Óptimo para tiempo real.
React Native	■ Lógica compleja fácil con <i>hooks</i> ■ Código compartido web y móvil ■ Comunidad muy grande ■ Buena documentación ■ Gestión de estado madura	■ Latencia introducida por puente JS-nativo ■ Configuración de permisos tediosa ■ <i>Overhead</i> de performance para datos continuos ■ Compilación lenta en proyectos grandes	Media. Viable solo con optimizaciones.

Continúa en la siguiente página



Herramienta	Ventajas	Desventajas	Idoneidad
Kotlin	■ Control total sobre APIs Bluetooth ■ Latencia predecible y mínima ■ Máxima eficiencia energética ■ Integración nativa TensorFlow Lite ■ Sintaxis moderna y seguridad de tipos	■ Solo para Android (iOS requiere Swift separado) ■ Duplicación de código para multiplataforma ■ Mayor complejidad de desarrollo ■ Menor agilidad de prototipado	Media-Alta. Solo para Android.
Capacitor	■ Puente oficial entre web y nativo ■ Acceso a APIs nativas mediante plugins ■ Compatible con frameworks web populares ■ Prototipado rápido ■ Reutilización de código web	■ Latencia excesiva para Bluetooth continuo ■ Arquitectura en capas (<i>overhead</i>) ■ Menos control sobre hardware ■ <i>Overhead</i> de máquina virtual web	Baja. No recomendado para tiempo real.

Tabla 4.11: Comparativa de herramientas para desarrollo de aplicación Bluetooth.

Se ha elegido Flutter como framework de desarrollo para la aplicación Bluetooth debido a su capacidad de compilación nativa, latencia mínima en comunicación Bluetooth, y equilibrio óptimo entre rendimiento hardware y agilidad de desarrollo multiplataforma. Flutter es especialmente adecuado para este proyecto porque permite ejecutar modelos de TensorFlow Lite directamente en dispositivos móviles con aceleración por hardware, gestiona el flujo continuo de datos sensoriales mediante Streams sin congelación de interfaz, y facilita iteración rápida en funcionalidades de prototipado. La biblioteca



flutter_blue_plus proporciona abstracción estable sobre las APIs de Bluetooth nativo, mientras que los Isolates de Dart permiten procesamiento intensivo de datos en hilos independientes, garantizando fluidez de la aplicación incluso durante análisis complejos de la red neuronal.

Capítulo 5

Implementación, desarrollo y pruebas comparativas

5.1. Implementación y desarrollo

Con el objetivo de contar con una nariz electrónica operativa que complemente el trabajo de los agentes caninos, se decidió entrenar una red neuronal artificial. Esta se integrará en una aplicación Bluetooth para dispositivos móviles que controlará la nariz electrónica. Dicha red neuronal será la encargada de determinar si la muestra analizada corresponde o no a la sustancia objetivo.

La implementación y el desarrollo de la nariz electrónica se estructuran en tres fases:

- Construcción del conjunto de datos (*dataset*).
- Entrenamiento de la red neuronal.
- Desarrollo de la aplicación móvil para la comunicación Bluetooth con la nariz electrónica.

A continuación, se detallan cada una de estas fases.



5.1.1. Construcción del *dataset*

Para la construcción del conjunto de datos, se llevaron a cabo múltiples experimentos (tomas de muestras) utilizando la nariz electrónica para recopilar datos de diversas muestras. En total se realizaron 144 experimentos, de los cuales los datos son almacenados en ficheros de texto (uno por cada experimento) que contienen las lecturas de los sensores. Estos ficheros se encuentran disponibles en el repositorio del proyecto mencionado en el capítulo de metodología.

Se busca hacer una clasificación binaria, es decir, distinguir entre dos clases, además de que el entrenamiento de la ANN se realice con un conjunto de datos equilibrado. Para ello, se ha dividido el *dataset* en dos carpetas (“BUENO” y “MALO”): una para las muestras positivas (hachís) y otra para las muestras negativas (otras sustancias) respectivamente. La primera carpeta tiene 62 archivos, mientras que la segunda tiene 82 archivos.

En principio se realizaron 76 experimentos (38 de muestras negativas y 38 de muestras positivas). Sin embargo, tras probar la red neuronal en un entorno real, se observó que la ANN captaba bien la sustancia objetivo pero solo si se acercaba mucho el tubo de la nariz a dicha sustancia. Esto es debido a que casi todos los datos positivos eran de la sustancia pura en un bote de muestreo. Por ello, se añadieron más experimentos, tanto positivos como negativos, hasta llegar 58 muestras de cada tipo. No obstante, pese a que en este punto la red neuronal detectaba con precisión la sustancia objetivo, el número de falsos positivos era demasiado alto. Para reducir estos falsos positivos, se añadieron más muestras negativas (además de 4 muestras positivas más como pequeña ampliación), llegando a un total de 82 muestras negativas (y 62 positivas), dando como resultado final las 144 muestras mencionadas anteriormente.

Para asignarle un nombre identificativo a cada experimento, se ha usado la nomenclatura EXP_X_Y-Z, donde:

- **EXP:** Es una abreviatura de “EXPERIMENTO”.
- **X:** Indica si la muestra pertenece a la clase “BUENO” (B) o “MALO” (M).



- **Y:** Es un índice que, en las muestras positivas, sirve para llevar un conteo de los experimentos realizados, mientras que, en las muestras negativas, se aumenta dicho índice cada vez que se cambia de sustancia.
- **Z:** Es un número secuencial que identifica el experimento dentro de cada sustancia. Es usado en las muestras negativas para diferenciar entre experimentos realizados con la misma sustancia bajo distintas condiciones. En las muestras positivas se omite ya que siempre se trabaja con hachís.

En cada archivo hay hasta 210 filas de datos, y en un entrenamiento a tiempo real cada una de las filas será una entrada independiente para la ANN. Al procesar los archivos se eliminan varias de estas filas de datos. Tras el procesamiento, se eliminan las 59 primeras filas que corresponden a la fase de desorción, la última fila, que también corresponde a esta fase y las 15 primeras filas de la fase de adsorción.

El cálculo del número total de entradas después del procesamiento se puede expresar matemáticamente como:

$$\text{Filas útiles por experimento} = 210 - 59 - 1 - 15 = 135$$

$$\text{Total de entradas} = 135 \times 144 = 19,440$$

donde 210 representa las filas iniciales por archivo, se restan las 59 filas de desorción inicial, 1 fila de desorción final, y 15 filas iniciales de adsorción, resultando en 135 filas de datos válidos. Al multiplicar por los 144 experimentos realizados, se obtiene un total de 19.440 entradas para el entrenamiento de la red neuronal.

Proceso de toma de muestras

Se trata de un proceso cíclico que consta de dos fases: desorción (toma de aire limpio para limpiar los sensores y establecer la línea base) y adsorción (toma de la muestra).

Para este proyecto se ha buscado recrear situaciones lo más parecidas posibles a una situación real, como controles de carretera. Para ello, se ha establecido un solo ciclo, con una fase de desorción de 120 segundos (dos minutos), para eliminar cualquier posible resto de impureza que haya podido quedar en los sensores, y una fase de adsorción de 300 segundos (cinco minutos).

Para que el aire entre en la carcasa y esté en contacto con los sensores, se conectan tubos flexibles a las entradas de la carcasa. En la entrada de desorción (IM_D) se conecta a un tubo a su vez conectado a un pequeño purificador de aire para que el aire esté limpio. En la entrada de adsorción (IM_A) se conecta a un tubo que conecta con el bote de muestreo donde estará la sustancia. A su vez, este bote de muestreo también está conectado a otro purificador de aire para que entre aire limpio en el bote y se cree una corriente de aire, permitiendo que el aire de dentro del bote pueda ser absorbido mas facilmente por el tubo conectado a la nariz. Esta disposición puede observarse en la Figura 5.1.



Figura 5.1: Disposición del equipo para la muestra EXP_M_5-1.

Por otra parte, también es posible tomar muestras sin necesidad de usar el bote de muestreo, conectando directamente un tubo de adsorción que entre en contacto con el aire, por ejemplo para tomar muestra de la calidad de aire de una habitación entera, o colocándolo cerca de una sustancia, como ocurre en la Figura 5.2.



Figura 5.2: Disposición del equipo para la muestra EXP_B_34.

Antes de poder empezar a tomar muestras, es imprescindible encender la nariz electrónica con una hora o 45 minutos de antelación para que los sensores se estabilicen térmicamente y las lecturas sean fiables.

Una vez los sensores se han calentado y la sustancia está preparada, solo queda conectar la nariz electrónica a un teléfono móvil Android mediante Bluetooth y usar la aplicación móvil desarrollada para controlar la toma de muestras. Habría que configurar en número de ciclos y los tiempos de adsorción y desorción en las condiciones que se han mencionado anteriormente (1 ciclo, 120 segundos de desorción y 300 segundos de adsorción) y ya podría iniciarse la toma de muestras enviando el comando a través de la aplicación. Para saber como configurar la aplicación y conectarla a la nariz electrónica, se puede consultar el Anexo C.

Una vez haya finalizado la toma de la muestra, los datos se habrán almacenado en un



fichero de texto llamado “DATOS.txt” en la tarjeta SD de la nariz electrónica. El siguiente paso es copiar este fichero al ordenador y cambiarle el nombre por el correspondiente a la nomenclatura mencionada anteriormente. Es importante borrar el contenido de la tarjeta SD antes de iniciar una nueva toma de muestras. De no hacerse, los datos de la siguiente muestra se añadirán al final del fichero “DATOS.txt”.

Muestras negativas

Para las muestras negativas, se han utilizado diversas sustancias que pueden encontrarse en un entorno cotidiano y que no deben ser confundidas con la sustancia objetivo. Estas sustancias incluyen alimentos como cacao en polvo, limón exprimido, salsas, cajas vacías, aire limpio, etc. Un seguimiento detallado de la obtención de las características de estas sustancias se encuentra en la Tabla A.2 del Anexo A.

Para cada sustancia (excepto la primera, que se hizo como una prueba), se han realizado múltiples experimentos bajo diferentes condiciones, como el estado del bote de muestreo (abierto o cerrado) y la presencia o ausencia de corriente de aire. Esto permite evaluar cómo estas variables afectan las lecturas de los sensores y la capacidad de la ANN para distinguir entre la sustancia objetivo y otras sustancias.

Por otra parte, no todas las muestras negativas se han tomado de sustancias usando el bote de muestreo. Algunas muestras se han tomado directamente del aire de una habitación, una caja o de un coche, sitios donde puede encontrarse el hachís.

Muestras positivas

Las muestras positivas se han tomado usando hasta cinco tipos diferentes de hachís, es decir, que provienen de cinco fuentes distintas, y cinta de envolver con restos de dicha sustancia. Esto se ha hecho para que la ANN pueda generalizar mejor y no se sobreentrene con las características específicas de un solo tipo de hachís. Pueden verse algunas de estas muestras en la Figura 5.3, la Figura 5.4 y la Figura 5.5.



Figura 5.3: Muestra de hachís usada de la toma 1 a la 4.



Figura 5.4: Muestra de hachís usada de la toma 17 a la 20.



Figura 5.5: Muestra de hachís usada de la toma 33 a la 38.

Para los cuatro primeros tipos se han realizado múltiples experimentos bajo distintas condiciones, como el estado del bote de muestreo (abierto o cerrado) o si la tapa del recipiente donde se encontraba el hachís estaba puesta o no.

En el caso del quinto tipo de hachís (la muestra más grande), los experimentos se han realizado sin usar el bote de muestreo, tomando los datos de la muestra directamente del aire de una caja cerrada y de un coche cerrado. Esto se ha hecho para simular situaciones reales donde el hachís puede estar oculto en espacios cerrados.

Por otro lado, la cinta de envolver se ha usado para tomar muestras, tanto usando el bote de muestreo, como en distintas situaciones como puede ser dentro de una caja o una maleta, o incluso simplemente con el bote donde esta almacenada abierto en una habitación.

Además, dado que para las muestras positivas se dispone de un menor número de muestras en comparación con las negativas, se ha optado por repetir algunos experimentos para aumentar la cantidad de datos disponibles para el entrenamiento de la ANN, ya que, pese a tratarse de la misma muestra, los resultados pueden ser ligeramente diferentes entre una toma y otra.



Un seguimiento detallado de estas tomas de muestras se encuentra en la Tabla B.2 del Anexo B.

Limpieza del *dataset*

Todos los archivos de texto de las muestras se han ido guardando en una carpeta llamada `Por_procesar`, en la que también se encuentran divididas en las subcarpetas “BUENO” y “MALO”. Estos archivos cuentan con datos, como la presión o la humedad, que no son útiles para el entrenamiento de la ANN, por lo que se ha llevado a cabo un proceso de limpieza y preprocesamiento de los datos para eliminar información irrelevante y preparar los datos para su uso en el entrenamiento de la red neuronal.

Este proceso de limpieza se realiza con un script en Python llamado `Limpiador.py`, el cual se describe a continuación detallando cada una de sus partes.

Importación de dependencias: El script comienza importando las librerías necesarias para el procesamiento de datos y la gestión de archivos:

```
1 import pandas as pd
2 import argparse
3 from pathlib import Path
4 import sys
5 import subprocess
```

Código 5.1: Importación de dependencias del script `Limpiador.py`.

Como se muestra en el Código 5.1, se utiliza `pandas` para la manipulación de datos tabulares, `argparse` para gestionar los argumentos de línea de comandos, `pathlib` para el manejo multiplataforma de rutas de archivos, y `subprocess` para la ejecución opcional del script de aumento de datos.

Lectura robusta de archivos: Dado que los archivos de datos pueden estar codificados en diferentes formatos (UTF-8, Windows-1252, Latin-1, etc.), se ha implementado una función que intenta múltiples codificaciones hasta encontrar una que



funcione correctamente:

```
1 def _leer_csv_robusto(file_path: str) -> pd.DataFrame:
2     """Lee el TXT/CSV con varios encodings comunes en Windows."""
3     # Encodings ordenados por frecuencia en sistemas Windows
4     encodings = [
5         'utf-8', # Estandar moderno
6         'utf-8-sig', # UTF-8 con marca de orden de bytes
7         'cp1252', # Windows-1252 (comun en español)
8         'latin-1', # ISO-8859-1 como alternativa
9     ]
10
11     last_error = None
12     # Intenta leer con cada encoding hasta encontrar uno que funcione
13     for enc in encodings:
14         try:
15             return pd.read_csv(
16                 file_path,
17                 skiprows=1,
18                 skipinitialspace=True,
19                 encoding=enc,
20             )
21         except UnicodeDecodeError as e:
22             last_error = e
23             continue
24
25     # Ultimo intento con motor Python (mas tolerante)
26     try:
27         return pd.read_csv(
28             file_path,
29             skiprows=1,
30             skipinitialspace=True,
```




```
31         encoding='latin-1',
32         engine='python',
33     )
34 except Exception:
35     if last_error:
36         raise last_error
37     raise
```

Código 5.2: Función para lectura robusta de archivos CSV con múltiples codificaciones.

En el Código 5.2, la función `_leer_csv_robusto` (líneas 1–36) itera sobre una lista de codificaciones comunes en sistemas Windows (líneas 4–8). Para cada codificación, intenta leer el archivo CSV saltando la primera fila (`skiprows=1`) que contiene metadatos. Si falla con un `UnicodeDecodeError`, continúa con la siguiente codificación. Como último recurso, utiliza el motor Python de pandas que es más tolerante con caracteres problemáticos.

Procesamiento y limpieza de datos La función principal de procesamiento elimina las columnas innecesarias y filtra las filas correspondientes a la fase de desorción:

```
1 def procesar_archivo(file_path: str, save_path: str) -> None:
2     """Lee, limpia y guarda un archivo de datos."""
3     df = _leer_csv_robusto(file_path)
4
5     # Columnas que se descartan del \textit{dataset}
6     columnas_a_borrar = [
7         'n', 'temp(scd40)', 'hum(scd40)', 'temp(ens160)',
8         'etoh(zmod4410)', 'temperatura(bme688)',
9         'humedad(bme688)', 'presion(bme688)'
10    ]
11
12    # Elimina columnas innecesarias y filtra donde Aire/muestra != 0
13    df = df.drop(columns=[c for c in columnas_a_borrar if c in df.columns])
14    df = df[df['Aire/muestra'] != 0]
```



```
15
16     # Elimina las 15 primeras lineas donde Aire/muestra == 1
17     indices_ones = df[df['Aire/muestra'] == 1].index[:15]
18     df = df.drop(indices_ones)
19
20     df.to_csv(save_path, sep=' ', decimal=',', index=False)
```

Código 5.3: Función principal para procesar y limpiar un archivo de datos.

En el Código 5.3, la función `procesar_archivo` (líneas 1–20) lee primero el archivo usando la función robusta descrita anteriormente (línea 3). A continuación, se define una lista con las columnas a eliminar (líneas 6–10), que incluyen datos de temperatura, humedad y presión que no aportan información relevante para la detección de sustancias. La línea 13 elimina estas columnas del DataFrame, mientras que la línea 14 filtra las filas donde la columna `Aire/muestra` es diferente de 0, descartando así los datos de la fase de desorción. Las líneas 17–18 eliminan las primeras 15 filas de la fase de adsorción, ya que durante este período inicial los sensores aún no han alcanzado una respuesta estable. Finalmente, el archivo procesado se guarda con separador de espacios y coma como separador decimal (línea 20).

Procesamiento por lotes: Para procesar todos los archivos de forma automática, se ha implementado una función que recorre las carpetas de entrada y aplica el procesamiento a cada archivo:

```
1 def procesar_lote() -> None:
2     """Procesa todos los archivos en modo lote."""
3     base = Path(__file__).parent
4     in_root = base / "Por_procesar"
5     # Pares (carpeta_entrada, carpeta_salida)
6     parejas = [("BUENO", "BUENO"), ("MALO", "MALO")]
7
8     total, ok, err = 0, 0, 0 # Contadores para estadísticas
9
```



```
10     # Procesa cada categoria (BUENO, MALO)
11     for sub_in, sub_out in parejas:
12         src_dir = in_root / sub_in
13         dst_dir = base / sub_out
14         dst_dir.mkdir(parents=True, exist_ok=True)
15
16     if not src_dir.exists():
17         print(f"[AVISO] Carpeta no encontrada: {src_dir}")
18         continue
19
20     # Procesa cada archivo de texto en orden alfabético
21     for txt in sorted(src_dir.glob("*.txt")):
22         total += 1
23         out_path = dst_dir / txt.name
24         try:
25             procesar_archivo(str(txt), str(out_path))
26             ok += 1
27             print(f"[OK] {txt.name} -> {out_path}")
28         except Exception as error:
29             err += 1
30             print(f"[ERROR] {txt.name}: {error}")
31
32     print(f"Resumen lote: {ok}/{total} OK, {err} errores")
```

Código 5.4: Función para el procesamiento por lotes de archivos.

El Código 5.4 muestra la función `procesar_lote` (líneas 1–32), que determina automáticamente las rutas de trabajo a partir de la ubicación del script (líneas 3–4). Define los pares de carpetas de entrada/salida (línea 6) y crea las carpetas de destino si no existen (línea 14). El bucle principal (líneas 11–30) recorre cada categoría, busca todos los archivos `.txt` en orden alfabético y aplica el procesamiento, mostrando el progreso y manejando posibles errores de forma individual.



Interfaz de línea de comandos: El script ofrece flexibilidad mediante una interfaz de línea de comandos que permite tanto el procesamiento por lotes como el de archivos individuales:

```
1 def seleccionar_y_limpiar_cli():
2     """Interfaz de línea de comandos para procesar archivos."""
3     parser = argparse.ArgumentParser(
4         description="Limpia un \textit{dataset} de sensores.")
5     parser.add_argument("--input", "-i",
6         help="Ruta del archivo de entrada (.txt)")
7     parser.add_argument("--output", "-o",
8         help="Ruta del archivo de salida (.txt)")
9     parser.add_argument("--batch", action="store_true",
10        help="Procesa por lotes (por defecto si no hay argumentos)")
11     args = parser.parse_args()
12
13     # Modo lote: si no hay argumentos de entrada/salida
14     if args.batch or (not args.input and not args.output):
15         procesar_lote()
16         return
17
18     # Valida argumentos para modo de archivo individual
19     if not args.input or not args.output:
20         parser.error("Debe indicar --batch o bien --input y --output.")
21
22     procesar_archivo(args.input, args.output)
23     print(f"Archivo procesado correctamente: {args.output}")
```

Código 5.5: Interfaz de línea de comandos del script.

En el Código 5.5, la función `seleccionar_y_limpiar_cli` configura tres argumentos opcionales: `--input` y `--output` para procesar un archivo específico, y `--batch` para el modo por lotes. Si no se proporcionan argumentos, el script ejecuta automáticamente el



modo por lotes (líneas 14–16).

Punto de entrada y encadenamiento con aumento de datos: Finalmente, el punto de entrada del script ejecuta el procesamiento y ofrece la opción de continuar con el aumento de datos:

```
1  if __name__ == "__main__":
2      # Ejecuta el limpiador de datos
3      seleccionar_y_limpiar_cli()
4
5      # Opcion para ejecutar el script de aumento de datos
6      print("\n" + "-"*40)
7      ans = input("Desea llevar a cabo el aumento de datos? (s/n): ")
8      if ans.strip().lower() == 's':
9          script_path = Path(__file__).parent / "Augmentation.py"
10         if script_path.exists():
11             print(f"Se lanza {script_path.name}...")
12             subprocess.run([sys.executable, str(script_path)])
13         else:
14             print(f"[ERROR] No se encuentra: {script_path}")
```

Código 5.6: Punto de entrada principal del script.

El Código 5.6 (líneas 1–14) permite encadenar el proceso de limpieza con el de aumento de datos de forma interactiva. Tras completar la limpieza, pregunta al usuario si desea continuar con el script `Augmentation.py`, facilitando así un flujo de trabajo completo en una sola ejecución.

Los archivos limpios se guardan en las carpetas `Entrenamiento/BUENO` y `Entrenamiento/MALO` respectivamente, manteniendo la misma estructura de nombres de archivo para facilitar su identificación.



Aumento de datos

Dado que la obtención de muestras físicas es un proceso costoso en tiempo y recursos, el conjunto de datos original puede resultar limitado para entrenar modelos de aprendizaje profundo robustos. Para mitigar el riesgo de sobreajuste (*overfitting*) y mejorar la capacidad de generalización de la red, se ha implementado una técnica de aumento de datos (*Data Augmentation*) mediante un script desarrollado en Python (`Augmentation.py`).

Este script procesa los archivos originales de las carpetas `Entrenamiento/BUENO` y `Entrenamiento/MALO`, donde se encuentran los datos “limpios”, para generar un nuevo conjunto de datos aumentado (“BUENO-PLUS” y “MALO-PLUS”). El algoritmo se ha configurado para generar 10 nuevas muestras sintéticas por cada muestra original, multiplicando significativamente el volumen del *dataset* disponible. A continuación se describe el funcionamiento del script detallando cada una de sus partes.

Importación de dependencias y configuración inicial: El script comienza importando las librerías necesarias y estableciendo la ruta de trabajo:

```
1 import pandas as pd
2 import numpy as np
3 import os
4
5 # Ruta automatica al directorio del script
6 script_dir = os.path.dirname(os.path.abspath(__file__))
```

Código 5.7: Importación de dependencias del script `Augmentation.py`.

Como se muestra en el Código 5.7, se utiliza `pandas` para la manipulación de datos tabulares, `numpy` para las operaciones numéricas y generación de ruido aleatorio, y `os` para la gestión de archivos y directorios. La variable `script_dir` (línea 6) almacena la ruta absoluta del directorio donde se encuentra el script, permitiendo que funcione correctamente independientemente de desde dónde se ejecute.



Función de aumento de datos: La función principal implementa la metodología de generación de datos sintéticos combinando dos técnicas: ventaneo adaptativo (*Sliding Window*) e inyección de ruido proporcional (*Jittering*):

```
1 def augment_pipeline_fixed(df, window_size=100, target_files=10,
2                             iterations=2, noise_level=0.005):
3     """Aumenta datos extrayendo ventanas y aplicando ruido."""
4     augmented_samples = []
5
6     # Calcula cuantas ventanas se necesitan extraer
7     needed_windows = target_files // iterations
8
9     # Ajusta ventanas si el archivo es mas corto que window_size
10    if len(df) <= window_size:
11        actual_window_size = len(df)
12        actual_windows = 1
13        actual_iterations = target_files
14        step_size = 1
15    else:
16        actual_window_size = window_size
17        actual_windows = needed_windows
18        actual_iterations = iterations
19        # Calcula el salto para distribuir ventanas uniformemente
20        if actual_windows > 1:
21            step_size = (len(df) - actual_window_size) // (actual_windows - 1)
22        else:
23            step_size = 0
```

Código 5.8: Función principal de aumento de datos con ventaneo y ruido.

En el Código 5.8, la función `augment_pipeline_fixed` (líneas 1–23) recibe como parámetros el DataFrame a aumentar, el tamaño de ventana (por defecto 100 filas), el número de archivos objetivo a generar (10), las iteraciones por ventana (2) y el nivel de



ruido (0.5 %). Las líneas 7–23 implementan una lógica adaptativa: si el archivo original es más corto que el tamaño de ventana, se ajustan los parámetros para generar igualmente el número objetivo de muestras. En caso contrario, se calcula el `step_size` (línea 21) para distribuir las ventanas uniformemente a lo largo del archivo.

```
1      # Extrae ventanas y genera variaciones con ruido
2      for w in range(actual_windows):
3          start = w * step_size
4          # Evita superar el final del dataframe
5          if start + actual_window_size > len(df):
6              start = len(df) - actual_window_size
7
8          window = df.iloc[start : start + actual_window_size].copy()
9
10         # Genera multiples versiones con ruido por cada ventana
11         for _ in range(actual_iterations):
12             # Separa características de la etiqueta
13             features = window.iloc[:, :-1].copy()
14             label = window.iloc[:, -1:].copy()
15
16             # Aplica ruido proporcional (Jittering) a las características
17             noise = np.random.normal(1, noise_level, features.shape)
18             features_aug = features * noise
19
20             # Concatena características aumentadas con la etiqueta original
21             df_aug = pd.concat([features_aug, label], axis=1)
22             augmented_samples.append(df_aug)
23
24     return augmented_samples[:target_files]
```

Código 5.9: Extracción de ventanas y aplicación de ruido gaussiano.



En el Código 5.9, el bucle principal (líneas 2–22) extrae ventanas del DataFrame original. Para cada ventana, se generan múltiples variaciones (líneas 11–22). El proceso de *Jittering* (líneas 17–18) aplica ruido gaussiano multiplicativo: se genera una matriz de valores aleatorios con distribución normal $\mathcal{N}(1, 0,005)$ y se multiplica elemento a elemento con las características. Esto altera ligeramente la amplitud de las señales preservando su morfología. La etiqueta original (columna Aire/muestra) se mantiene intacta (línea 14) para no corromper la información de clase.

Procesamiento de carpetas: La siguiente función se encarga de procesar todos los archivos de una carpeta y guardar los resultados:

```
1 def process_balanced_data(input_folder_name, output_folder_name,
2                             target_per_file=10):
3     """Procesa todos los archivos de una carpeta y
4     genera datos aumentados."""
5     input_path = os.path.join(script_dir, input_folder_name)
6     output_path = os.path.join(script_dir, output_folder_name)
7
8     # Valida que la carpeta de entrada existe
9     if not os.path.exists(input_path):
10         print(f"Error: No existe {input_folder_name}")
11         return
12
13     # Crea la carpeta de salida si no existe
14     if not os.path.exists(output_path):
15         os.makedirs(output_path)
16
17     # Procesa cada archivo de texto de la carpeta de entrada
18     for filename in os.listdir(input_path):
19         if filename.endswith(".txt"):
20             try:
21                 # Lee el archivo CSV con separador espacio y coma decimal
```



```
22         df = pd.read_csv(os.path.join(input_path, filename),
23                           sep='\s+', decimal=',')
24
25         # Aplica aumento de datos
26         new_samples = augment_pipeline_fixed(
27             df,
28             window_size=100,
29             target_files=target_per_file,
30             iterations=2,
31             noise_level=0.005
32         )
33
34         # Guarda cada muestra aumentada con sufijo -P{numero}
35         name_base = os.path.splitext(filename)[0]
36         for idx, aug_df in enumerate(new_samples, 1):
37             new_name = f"{name_base}-P{idx}.txt"
38             aug_df.to_csv(os.path.join(output_path, new_name),
39                           sep=' ', decimal=',', index=False)
40
41             print(f"Se generaron {len(new_samples)} archivos
42                   de {filename}.")
43         except Exception as error:
44             print(f"Error en {filename}: {error}")
```

Código 5.10: Función para procesar carpetas completas.

El Código 5.10 muestra la función `process_balanced_data` (líneas 1–42), que construye las rutas de entrada y salida (líneas 4–5) y valida su existencia (líneas 8–14). El bucle principal (líneas 17–42) itera sobre cada archivo `.txt`, lo lee con el formato correcto (separador de espacios y coma decimal, líneas 21–22), aplica el aumento de datos (líneas 25–31) y guarda cada muestra generada con un sufijo `-P{número}` (líneas 34–38). El manejo de excepciones (líneas 19, 41–42) asegura que un error en un archivo no detenga



el procesamiento completo.

Punto de entrada y ejecución: El script finaliza con la configuración de parámetros y la ejecución del procesamiento:

```
1  # Numero de archivos aumentados a generar por cada archivo original
2  OBJETIVO_POR_ARCHIVO = 10
3
4  # Mapeo de carpetas de entrada a carpetas de salida
5  folders = {"BUENO": "BUENO-PLUS", "MALO": "MALO-PLUS"}
6
7  # Procesa cada categoria de datos
8  for ori, dest in folders.items():
9      process_balanced_data(ori, dest, target_per_file=OBJETIVO_POR_ARCHIVO)
10
11 print(f"\nSe finaliza el proceso. Cada archivo
12       genera {OBJETIVO_POR_ARCHIVO} versiones.")
```

Código 5.11: Configuración y ejecución del aumento de datos.

En el Código 5.11, la constante `OBJETIVO_POR_ARCHIVO` (línea 2) define cuántas muestras sintéticas se generarán por cada archivo original. El diccionario `folders` (línea 5) establece el mapeo entre carpetas de entrada y salida. El bucle final (líneas 8–9) procesa ambas categorías (BUENO y MALO) de forma secuencial.

Resultados del aumento de datos: El *dataset* original constaba de 144 muestras (62 positivas y 82 negativas). Tras aplicar el proceso de aumento de datos, se han generado un total de 1440 muestras sintéticas adicionales (620 positivas y 820 negativas). El nombre de los archivos aumentados sigue la nomenclatura original, añadiendo un sufijo “-PX” donde “X” es un número secuencial que identifica cada muestra sintética generada a partir de la misma muestra original.

En estos nuevos archivos el número total de filas por archivo es de 100, correspondientes a las 100 lecturas temporales extraídas en cada ventana. Multiplicando estas 100 filas por



las 1440 muestras aumentadas, se obtiene un total de 144.000 nuevas entradas para el entrenamiento de la red neuronal.

De esta forma, se consigue un *dataset* más rico y variado, permitiendo que la red neuronal aprenda a reconocer patrones incluso en presencia de ruido o variaciones temporales.

5.1.2. Entrenamiento de la red neuronal

Como se ha mencionado en apartados anteriores de este documento, no se entrenará una sola ANN, sino que se entrenarán varios modelos con distintos frameworks (Keras y PyTorch) y conjuntos de datos, concretamente con un conjunto de datos normal y otro conjunto de datos aumentado, para comparar resultados y determinar cuál se adapta mejor a las necesidades del proyecto.

Entre esos modelos se entrenaron modelos estándar y modelos optimizados para su uso en tiempo real en la aplicación móvil que controla la nariz electrónica. Para ver como se han entrenado los modelos estándar y los resultados obtenidos, se puede consultar el Anexo D.

Keras a tiempo real

Esta versión del modelo entrenado en Keras está diseñada para su uso en la aplicación móvil que controla la nariz electrónica en tiempo real. A diferencia del modelo estándar, que procesa datos completos después de la toma de muestras, este modelo está optimizado para realizar inferencias de manera continua a medida que llegan los datos de los sensores.

A diferencia de las versiones estándar, este desarrollo pone especial énfasis en los parámetros de normalización necesarios para el entorno de producción. El entrenamiento se ha realizado en un notebook de Jupyter ejecutado en Google Colab, cuyo código se describe a continuación.



Importación de librerías: El notebook comienza importando todas las dependencias necesarias para el procesamiento de datos, entrenamiento y evaluación del modelo:

```
1 import os
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.metrics import confusion_matrix, classification_report
9 import tensorflow as tf
10 from tensorflow.keras.models import Sequential
11 from tensorflow.keras.layers import Dense, Dropout
12 from tensorflow.keras.callbacks import EarlyStopping
13 from google.colab import drive
14
15 print("Librerías cargadas correctamente.")
```

Código 5.12: Importación de librerías para el entrenamiento del modelo.

Como se muestra en el Código 5.12, se utilizan `numpy` y `pandas` para la manipulación de datos, `matplotlib` y `seaborn` para visualización, `sklearn` para el preprocesamiento y división del *dataset*, y `tensorflow.keras` para la construcción y entrenamiento de la red neuronal. La línea 13 importa el módulo de Google Drive para acceder a los archivos almacenados en la nube.

Carga y etiquetado de datos: La función de carga de datos está diseñada específicamente para procesar los archivos generados por el script de limpieza, extrayendo únicamente las lecturas de la fase de adsorción:

```
1 drive.mount('/content/drive')
2
3 # Configura las rutas.
```



```

4 bueno_dir = '/content/drive/MyDrive/Colab Notebooks/BUENO'
5 malo_dir = '/content/drive/MyDrive/Colab Notebooks/MALO'
6
7 X_features = []
8 y_labels = []
9
10 def cargar_datos_tiempo_real(ruta_carpeta, etiqueta):
11     conteo_archivos = 0
12     conteo_filas = 0
13     for archivo in os.listdir(ruta_carpeta):
14         if archivo.endswith(".txt"):
15             ruta_completa = os.path.join(ruta_carpeta, archivo)
16             try:
17                 # Carga archivos de 14 columnas (limpiados previamente).
18                 df = pd.read_csv(ruta_completa, sep=' ',
19                                 decimal=',', header=0)
20                 df = df.apply(pd.to_numeric, errors='coerce').dropna()
21
22                 # Filtra por fase: Solo filas donde Aire/Muestra es 1.
23                 df_muestra = df[df.iloc[:, 13] == 1]
24
25                 # Extrae solo las 13 columnas de sensores de gas.
26                 for row in df_muestra.values:
27                     X_features.append(row[:13])
28                     y_labels.append(etiqueta)
29                     conteo_filas += 1
30                     conteo_archivos += 1
31             except Exception as e:
32                 print(f"Error en {archivo}: {e}")
33
34     print(f"Carpeta {os.path.basename(ruta_carpeta)}: ")

```



```
35         f"{conteo_archivos} archivos, {conteo_filas} filas.")
```

Código 5.13: Función para cargar y etiquetar los datos de entrenamiento.

En el Código 5.13, la función `cargar_datos_tiempo_real` (líneas 10–34) procesa cada archivo `.txt` de la carpeta especificada. La línea 18 lee el archivo con separador de espacios y coma decimal. El filtro de la línea 22 selecciona únicamente las filas donde la columna 14 (índice 13) tiene valor 1, correspondiente a la fase de muestra. Las líneas 25–28 extraen las 13 columnas de sensores de gas y asignan la etiqueta correspondiente (1 para BUENO, 0 para MALO).

```
1  # Carga los datos.
2  n_buenos = cargar_datos_tiempo_real(bueno_dir, 1) # Clase Positiva
3  n_malos = cargar_datos_tiempo_real(malo_dir, 0) # Clase Negativa
4
5  X = np.array(X_features)
6  y = np.array(y_labels)
7
8  print(f"Forma de la matriz X: {X.shape}")
```

Código 5.14: Ejecución de la carga de datos.

División del *dataset* y escalado: Para una evaluación rigurosa del modelo, se realiza una división estratificada del *dataset* en tres conjuntos: entrenamiento (70 %), validación (15 %) y test (15 %):

```
1  # Primera division: Separa el 15% para el TEST FINAL.
2  X_temp, X_test, y_temp, y_test = train_test_split(
3      X, y, test_size=0.15, random_state=42, stratify=y
4  )
5
6  # Segunda division: Del resto (85%), separa ~18% para VALIDACION.
7  # (0.85 * 0.176 = 0.15 del total)
8  X_train, X_val, y_train, y_val = train_test_split(
```



```
9     X_temp, y_temp, test_size=0.176, random_state=42, stratify=y_temp
10 )
11
12 # Escalado estandar (Z-score).
13 scaler = StandardScaler()
14 X_train_scaled = scaler.fit_transform(X_train)
15 X_val_scaled = scaler.transform(X_val)
16 X_test_scaled = scaler.transform(X_test)
17
18 print(f"Muestras totales: {len(X)}")
19 print(f"Entrenamiento: {len(X_train_scaled)} (70%)")
20 print(f"Validacion: {len(X_val_scaled)} (15%)")
21 print(f"Test Final: {len(X_test_scaled)} (15%)")
```

Código 5.15: División estratificada y escalado de los datos.

En el Código 5.15, la primera división (líneas 2–4) reserva el 15 % de los datos para el test final. La segunda división (líneas 8–10) extrae aproximadamente el 18 % del 85 % restante, resultando en un 15 % del total para validación. El parámetro `stratify` asegura que la proporción de clases se mantenga en cada subconjunto. El `StandardScaler` (líneas 13–16) normaliza los datos aplicando la transformación $z = (x - \mu)/\sigma$, donde μ y σ se calculan únicamente con los datos de entrenamiento para evitar *data leakage*.

Definición y entrenamiento de la red neuronal: La arquitectura de la red neuronal se define utilizando la API Sequential de Keras, optimizada para un equilibrio entre eficiencia computacional y precisión:

```
1 # Define el modelo.
2 model = Sequential([
3     Dense(16, activation='relu', input_shape=(13,)),
4     Dropout(0.2), # Regularizacion para evitar sobreajuste.
5     Dense(8, activation='relu'),
6     Dense(1, activation='sigmoid')
```




```
7  ])
8
9  model.compile(optimizer='adam', loss='binary_crossentropy',
10               metrics=['accuracy'])
11
12  # Configura la parada temprana monitorizando la perdida de VALIDACION.
13  early_stop = EarlyStopping(
14      monitor='val_loss',
15      patience=10,
16      restore_best_weights=True
17  )
```

Código 5.16: Definición de la arquitectura de la red neuronal.

En el Código 5.16, la arquitectura consta de una capa de entrada de 16 neuronas con activación ReLU (línea 3), una capa de Dropout del 20 % para regularización (línea 4), una capa oculta de 8 neuronas (línea 5), y una capa de salida con activación sigmoide para clasificación binaria (línea 6). El modelo se compila con el optimizador Adam y la función de pérdida *binary_crossentropy* (línea 9). El *EarlyStopping* (líneas 12–16) detiene el entrenamiento si la pérdida de validación no mejora durante 10 épocas consecutivas, restaurando los mejores pesos encontrados.

```
1  # Entrena el modelo.
2  history = model.fit(
3      X_train_scaled, y_train,
4      epochs=100,
5      batch_size=32,
6      validation_data=(X_val_scaled, y_val),
7      callbacks=[early_stop],
8      verbose=1
9  )
```

Código 5.17: Entrenamiento del modelo con validación.



En el Código 5.17, el entrenamiento se ejecuta durante un máximo de 100 épocas con un tamaño de lote de 32 muestras. El conjunto de validación se utiliza para monitorizar el rendimiento durante el entrenamiento y activar la parada temprana cuando sea necesario.

Evaluación y visualización de resultados: Una vez completado el entrenamiento, se evalúa el modelo con el conjunto de test y se visualizan las curvas de aprendizaje:

```
1  # Evalua con el conjunto de Test.
2  loss, acc = model.evaluate(X_test_scaled, y_test)
3  print(f"\n>> PRECISION FINAL EN EL TEST: {acc*100:.2f}%")
4
5  # Grafica de Precision (Accuracy).
6  plt.subplot(1, 2, 1)
7  plt.plot(history.history['accuracy'], label='Entrenamiento')
8  plt.plot(history.history['val_accuracy'], label='Validacion')
9  plt.title('Precision (Accuracy)', fontsize=14)
10 plt.xlabel('Epocas')
11 plt.ylabel('Precision')
12 plt.legend()
13
14 # Grafica de Perdida (Loss).
15 plt.subplot(1, 2, 2)
16 plt.plot(history.history['loss'], label='Entrenamiento')
17 plt.plot(history.history['val_loss'], label='Validacion')
18 plt.title('Perdida (Loss)', fontsize=14)
19 plt.xlabel('Epocas')
20 plt.ylabel('Perdida')
21 plt.legend()
22
23 plt.tight_layout()
24 plt.show()
```

Código 5.18: Evaluación del modelo y visualización de curvas de aprendizaje.



En el Código 5.18, la evaluación en el conjunto de test (línea 2) proporciona una estimación imparcial del rendimiento del modelo en datos no vistos. Las gráficas de precisión y pérdida permiten visualizar si el modelo sufre de sobreajuste (cuando las curvas de entrenamiento y validación divergen significativamente).

```
1  # Realiza predicciones en el conjunto de prueba.
2  y_pred = (model.predict(X_test_scaled) > 0.5).astype(int)
3  cm = confusion_matrix(y_test, y_pred)
4
5  plt.figure(figsize=(6, 5))
6  sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
7              xticklabels=['Malo', 'Bueno'], yticklabels=['Malo', 'Bueno'])
8  plt.title('Matriz de Confusion - Modelo Real-Time')
9  plt.ylabel('Real')
10 plt.xlabel('Prediccion')
11 plt.show()
```

Código 5.19: Generación de la matriz de confusión.

En el Código 5.19, la matriz de confusión (líneas 2–3) muestra el número de verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos, permitiendo evaluar el comportamiento del modelo en cada clase.

Exportación del modelo y parámetros de normalización: El paso final consiste en convertir el modelo a formato TensorFlow Lite y extraer los parámetros necesarios para la normalización en la aplicación móvil:

```
1  converter = tf.lite.TFLiteConverter.from_keras_model(model)
2  tflite_model = converter.convert()
3
4  tflite_path = '/content/modelo_hachis.tflite'
5  with open(tflite_path, 'wb') as f:
6      f.write(tflite_model)
7
```



```
8 print(f"\nProceso finalizado. Descarga el archivo: {tflite_path}")
```

Código 5.20: Conversión del modelo a TensorFlow Lite.

En el Código 5.20, el `TFLiteConverter` (líneas 1–2) transforma el modelo Keras en un formato binario optimizado para dispositivos móviles con procesadores ARM. El archivo resultante tiene un tamaño significativamente menor que el modelo original y está optimizado para inferencia eficiente.

```
1 print("Medias para Flutter:", list(scaler.mean_))
2 print("Desviaciones para Flutter:", list(np.sqrt(scaler.var_)))
```

Código 5.21: Extracción de parámetros de normalización para Flutter.

El Código 5.21 muestra el último paso, que es crítico para el despliegue: se extraen las medias (`scaler.mean_`) y desviaciones estándar ($\sqrt{\text{scaler.var}_}$) calculadas durante el entrenamiento. Estos 13 valores (uno por cada sensor) deben configurarse en la aplicación móvil para normalizar las lecturas en tiempo real de la misma forma que se normalizaron los datos de entrenamiento, garantizando así la fidelidad de las predicciones.

5.1.3. Desarrollo de la aplicación móvil

Anteriormente se ha mencionado que la aplicación se ha desarrollado usando Flutter y es importante destacar que, para que los modelos puedan ser ejecutados en dispositivos móviles, es necesario convertirlos a un formato compatible con TensorFlow Lite (.tflite). Los modelos entrenados en Keras se han convertido a este formato directamente, mientras que los modelos desarrollados en PyTorch requieren una conversión adicional. Esta conversión se realiza primero exportando el modelo a ONNX (*Open Neural Network Exchange*) y luego utilizando el convertidor de ONNX a TensorFlow Lite. Sin embargo, la conversión de PyTorch a ONNX presenta ciertos problemas de compatibilidad, por lo que no se ha podido completar este proceso para los modelos desarrollados en PyTorch. Por otro lado, los modelos deben de ser entrenados y optimizados específicamente para



su uso a tiempo real, ya que la aplicación móvil debe ser capaz de procesar los datos de los sensores y realizar inferencias en tiempo real sin que haya retrasos perceptibles para el usuario.

Por este motivo, la aplicación móvil solo integra los modelos desarrollados en Keras a tiempo real. La aplicación permite al usuario seleccionar entre el modelo normal y el modelo plus, dependiendo de sus necesidades y del entorno en el que se utilice la nariz electrónica.

A continuación se describe como se ha desarrollado la aplicación:

Configuración del entorno de desarrollo

El desarrollo de software para sistemas de sensado químico requiere un entorno que permita la iteración rápida y la visualización de datos en tiempo real. Para ello, Flutter, cuyo lenguaje de programación es Dart, fue elegido por su capacidad de gestionar interfaces reactivas. Esto es crucial cuando los datos de los sensores llegan con una frecuencia alta (por ejemplo cada 2 segundos), ya que la UI debe actualizarse sin bloquear el hilo principal de ejecución.

Para poder comenzar con la creación de la aplicación, es necesario instalar Flutter SDK y configurar un editor de código compatible, como Visual Studio Code o Android Studio. Además, se deben instalar los plugins necesarios para Flutter y Dart en el editor elegido. Además, hay que instalar Java Development Kit (JDK) para la compilación de la aplicación. Por otra parte, es necesario tener instalado Android Studio para poder emular dispositivos Android y probar la aplicación en un entorno controlado.

Una vez instalado todo el software necesario y creado el proyecto Flutter usando el comando `flutter create nombre_del_proyecto`, se deben añadir las dependencias necesarias en el archivo `pubspec.yaml`. Las dependencias principales son:

- **flutter_blue_plus:** Para el escaneo y comunicación de baja energía (BLE) con el microcontrolador PIC32 de la nariz electrónica.



- **tflite_flutter**: Motor de ejecución para el modelo de Inteligencia Artificial en el dispositivo móvil.
- **vibration**: Para el feedback háptico en alertas de seguridad (vibración cuando se detecta la sustancia objetivo).

Arquitectura de la aplicación

La aplicación sigue una arquitectura modular para separar la lógica de comunicación de la interfaz de usuario.

La estructura de ficheros de la aplicación móvil se muestra en la Figura 5.6. En el proyecto hay dos carpetas importantes: la carpeta **assets**, donde están el icono de la aplicación y los modelos y su configuración, y la carpeta **lib**, donde hay otras subcarpetas. La carpeta **screens** contiene las distintas pantallas de la aplicación, como la pantalla principal (listado de dispositivos disponibles), la pantalla de análisis (control del análisis de datos) y la pantalla de configuración (modificación de los ajustes del análisis). La carpeta **utils** alberga funciones auxiliares, en este caso, la navegación entre pantallas. La carpeta **widgets** contiene componentes reutilizables, actualmente solo la barra de navegación superior. El archivo **main.dart** es el punto de entrada de la aplicación, donde se inicializan las dependencias y se configura la navegación entre pantallas.

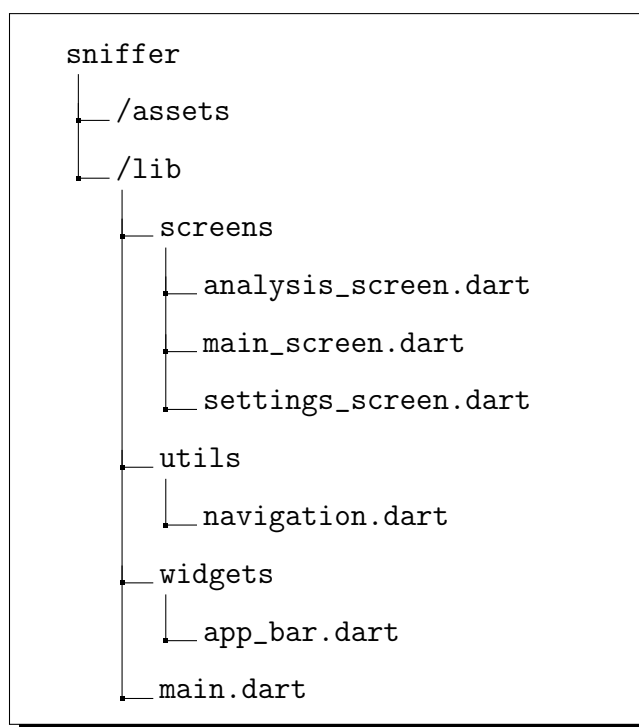


Figura 5.6: Estructura de ficheros de la aplicación móvil

Desarrollo del protocolo de comunicación

La comunicación entre la aplicación móvil y la nariz electrónica se realiza mediante Bluetooth Low Energy (BLE). El microcontrolador PIC32 en la nariz electrónica actúa como un servidor BLE, mientras que la aplicación móvil funciona como un cliente BLE. La comunicación se establece utilizando servicios y características BLE predefinidos.

- **Sincronización de comandos:** Se utiliza el prefijo » *Enviando:* para registrar las órdenes enviadas (Por ejemplo *#NSNC,1*) y se espera el *OK* de confirmación del PIC32.
- **Gestión de datos (Buffering):** Se ha implementado un buffer de línea (*_rawBuffer*) que acumula fragmentos de datos hasta detectar un salto de línea (*\n*), evitando duplicados visuales y errores de lectura en las tramas de 13 sensores.
- **Control de ciclos (AutoStop):** La aplicación monitoriza la cadena “STOP” o



“AutoStop Mode enabled” enviada por el firmware *v1.5_Cristina* para finalizar el experimento automáticamente y mostrar un cuadro de diálogo al usuario.

Implementación de la inteligencia artificial

Se ha implementado la capacidad de ejecutar modelos de TensorFlow Lite directamente en el dispositivo móvil utilizando la librería `tflite_flutter`. Esto permite realizar inferencias de manera eficiente sin necesidad de conexión a internet. Además, es posible cambiar fácilmente de modelo desde la interfaz de usuario, permitiendo al usuario seleccionar entre el modelo normal y el modelo plus según sus necesidades.

Es necesario que los datos de los sensores se preprocesen de la misma manera que durante el entrenamiento del modelo. Para ello la función *procesarTramaParaIA* actúa como un portero lógico de la siguiente manera:

1. Quita el espacio que hay al inicio de la primera fila de datos.
2. Elimina la primera fila que contiene la cabecera.
3. Ignora la fila con los nombres de las columnas. Es necesario para que el programa no intente convertir texto en números.
4. Dart usa el punto como decimal por defecto, así que no es necesario cambiar puntos por comas. Además, en vez de cambiar las comas por espacios, las usa como separadores directamente.
5. Si el valor de la ultima columna es 0 (aire limpio de la desorción) lo omite.
6. Elimina las columnas que no son necesarias para la inferencia (presión, humedad, temperatura, etc.).

Acto seguido se aplica la normalización estadística aplicando un “StandardScaler” $z = (x - \mu)/\sigma$. Esto nivela sensores con rangos dispares, como el CO₂ (400–2000 ppm) frente a



la IAQ del sensor ZMOD4410 (0–500), permitiendo que la red neuronal aprenda patrones químicos en lugar de magnitudes absolutas.

Para finalizar se utiliza “Zero-Padding” para adaptar la entrada individual de sensores al tamaño esperado por el modelo (242.676 datos), devolviendo una probabilidad de impureza.

Diseño de interfaz y experiencia de usuario

La interfaz de usuario (UI) de la aplicación móvil ha sido diseñada para ser intuitiva y fácil de usar, permitiendo a los usuarios interactuar con la nariz electrónica de manera eficiente. Se han implementado varias pantallas clave tales como una pantalla principal en la que se muestra una lista de dispositivos Bluetooth disponibles para conectarse; una pantalla de ajustes en la que se pueden configurar parámetros del análisis, tales como el número de ciclos, los tiempos de adsorción y desorción y la activación del modo AutoStop y de la vibración; y una pantalla de análisis donde se visualizan los datos en tiempo real. La pantalla de analisis cuenta con los siguientes componenetes:

- **Desplegable de modelos:** Un desplegable en el que se puede seleccionar el modelo de IA a utilizar.
- **Terminal:** Donde se muestran los mensajes y comandos que se envían y reciben al comunicarse con la nariz electrónica.
- **Indicador de impureza:** Muestra la impureza del aire en tiempo real, actualizándose cada vez que llega una nueva lectura de los sensores.
- **Botón de iniciar/cancelar:** Permite al usuario iniciar o detener el análisis en cualquier momento.

Se utiliza un sistema de semáforo en el que si la impureza calculada es $\geq 0,7$, el contenedor de impureza cambia a rojo con el mensaje “¡SUSTANCIA DETECTADA!”. Si



es menor, se mantiene verde como “AIRE LIMPIO”. Además, se ha implementado un sistema de alertas hápticas que hace vibrar el dispositivo cuando se detecta la sustancia objetivo, proporcionando una notificación inmediata al usuario sin necesidad de mirar a la pantalla.

5.1.4. Funcionamiento de la aplicación

Para terminar esta sección, se describe cual es el funcionamiento de la aplicación internamente cuando el usuario interactúa con ella. El flujo de ejecución de la aplicación es el siguiente.

Conexión con la nariz electrónica

Cuando se abre la aplicación por primera vez, hay que aceptar los permisos necesarios para el funcionamiento de la aplicación, como el acceso a Bluetooth y la ubicación. Una vez aceptados los permisos, se muestra la pantalla principal con la lista de dispositivos Bluetooth disponibles, que se ve en la Figura 5.7, omitiendo los dispositivos que aparecen como “Desconocido” y resaltando los que contengan “NOSE” en su nombre. El usuario selecciona la nariz electrónica de la lista y se establece la conexión BLE.

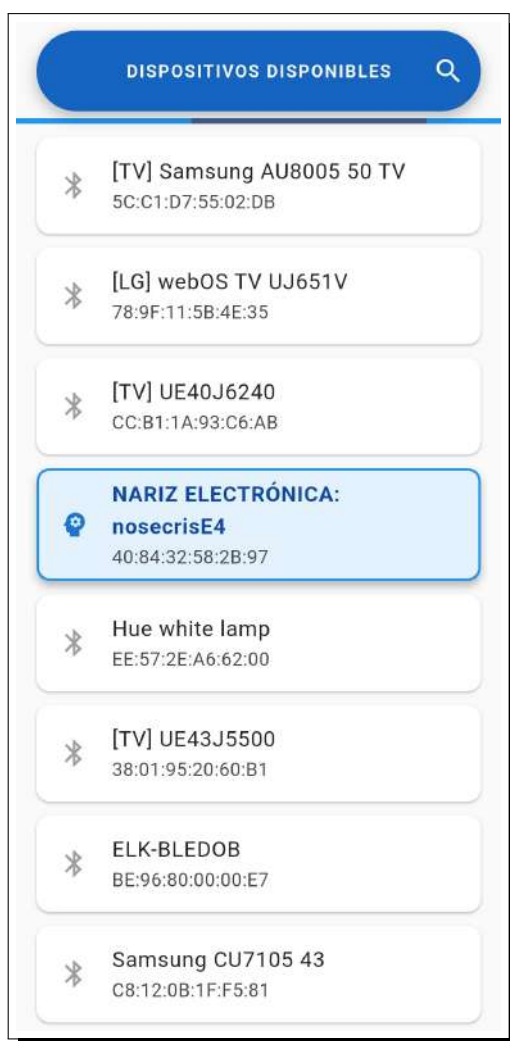


Figura 5.7: Pantalla principal de la aplicación móvil con la lista de dispositivos Bluetooth disponibles

En caso de querer volver a buscar dispositivos, se puede pulsar el botón de búsqueda en la esquina superior derecha de la pantalla. Dicho botón puede verse en la Figura 5.8.



Figura 5.8: Botón de búsqueda de dispositivos Bluetooth en la aplicación Sniffer.

Una vez se ha realizado la conexión con la nariz electrónica, se accederá a la pantalla de análisis de sustancias.

Detección de sustancias

La pantalla de análisis cuenta con la opción de elegir que modelo de red neuronal se desea usar para analizar las muestras recibidas por la nariz electrónica. Esta opción puede verse en la Figura 5.9. Para ello, la aplicación lee la información de los modelos guardada en `assets/models_config.json` donde están los ficheros de los modelos, sus medias y desviaciones estándar para la normalización, y los nombres que se mostrarán en el desplegable.



Figura 5.9: Selección del modelo de red neuronal en la aplicación Sniffer.

Además cuenta con una terminal donde se muestran los mensajes enviados y recibidos.



El usuario puede iniciar el análisis pulsando el botón de iniciar. Esto envía el comando de inicio “Exper” a la nariz electrónica y comienza a recibir datos en tiempo real. La aplicación procesa cada trama de datos recibida, aplicando el preprocesamiento necesario (normalización y padding) antes de pasar los datos al modelo de IA seleccionado para obtener una predicción de impureza. La impureza se muestra en la pantalla y se actualiza cada vez que llega una nueva lectura. Si la impureza es mayor o igual al 70 %, la aplicación cambia el indicador a rojo y muestra una alerta de sustancia detectada, además de activar la vibración del dispositivo si está habilitada. La ejecución continúa hasta que acaban los ciclos y se ejecuta el AutoStop, o hasta que el usuario decide cancelar el análisis pulsando el botón de cancelar, lo que envía el comando “Stop” a la nariz electrónica y detiene la recepción de datos. En la Figura 5.10 se puede ver la pantalla de análisis en funcionamiento.

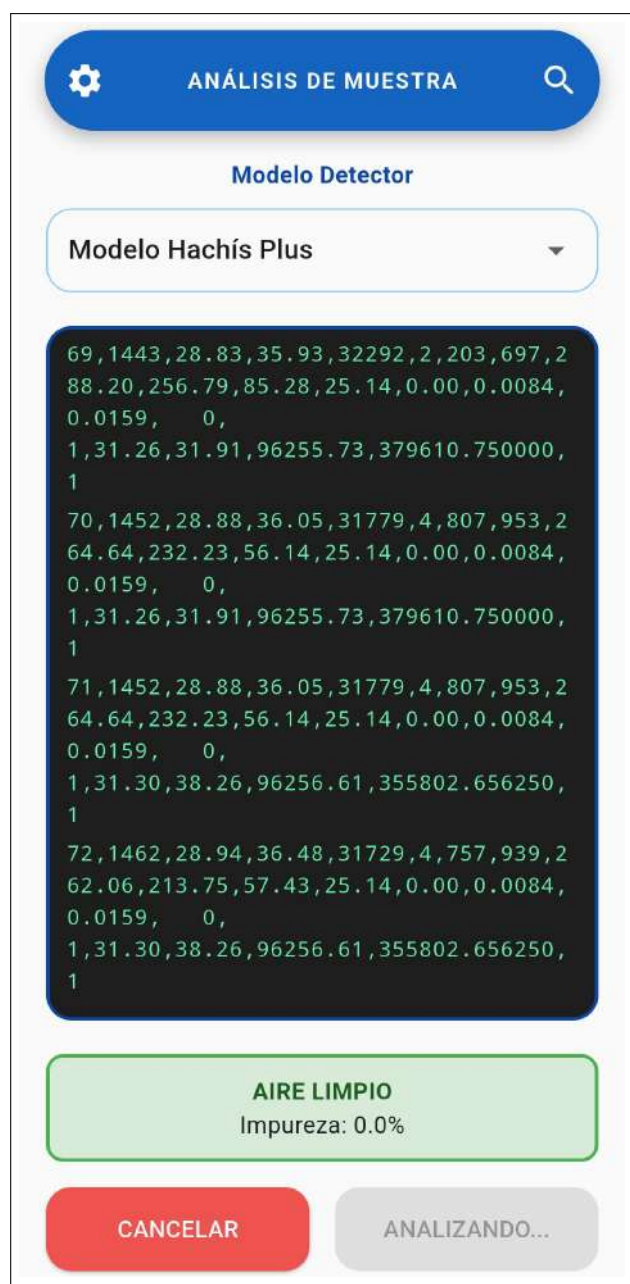


Figura 5.10: Pantalla de análisis de sustancias en la aplicación Sniffer.

Desde esta pantalla también es posible acceder a la pantalla de ajustes de la aplicación o volver a la pantalla principal para desconectar la nariz electrónica y conectar otro dispositivo si se desea. En la Figura 5.11 se pueden ver los botones correspondientes en la barra superior de la pantalla de análisis. La lupa para volver a la pantalla principal y el icono de engranaje para acceder a la pantalla de ajustes.



Figura 5.11: Botones de la barra superior en la pantalla de análisis de sustancias.

Configuración del análisis

Al ir a la pantalla de ajustes (vease en la Figura 5.12), la aplicación enviará el comando “SETTINGS” a la nariz electrónica para que esta le devuelva un mensaje con la configuración que incluye el número de ciclos, los tiempos de adsorción y desorción, y el estado del modo AutoStop. Estos datos recibidos se reflejan en los campos correspondientes de la pantalla de ajustes (además del estado de la vibración). El usuario puede modificar estos parámetros y guardar los cambios pulsando el botón de guardar. Esto envía un comando “#NSNC,X”, donde X es el número de ciclos, seguido de los tiempos de adsorción (“#NSTA,X”) y desorción (“#NSTD,X”), donde X es el tiempo en segundos, y el estado del modo AutoStop (1 para activado, 0 para desactivado). La nariz electrónica responde con un “OK” confirmando que ha recibido y aplicado la nueva configuración.

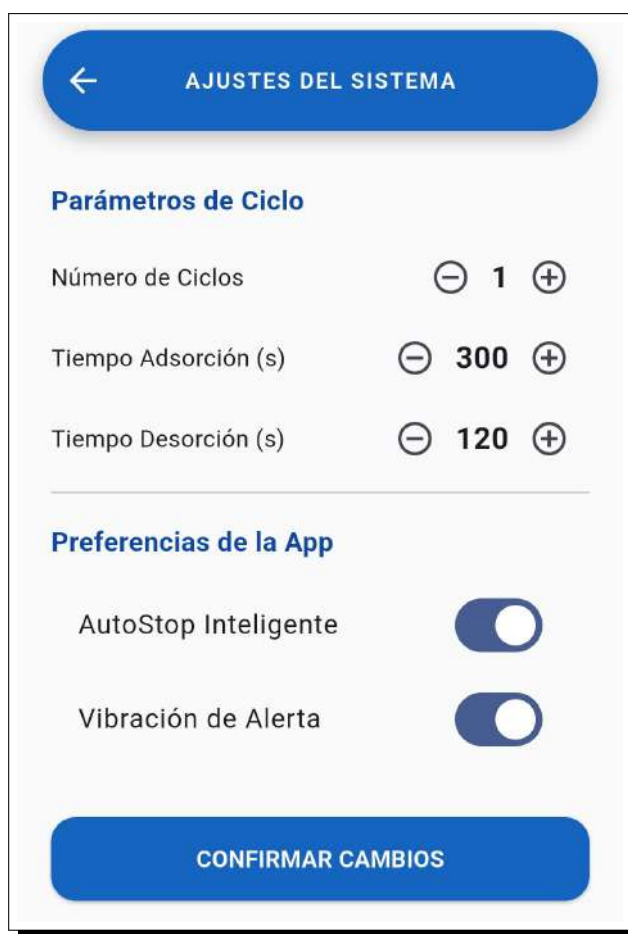


Figura 5.12: Pantalla de ajustes de la aplicación móvil

También está la opción de volver a la pantalla de análisis sin guardar los cambios. Para ello simplemente hay que pulsar el icono de la flecha hacia atrás en la barra superior, el cual puede observarse en la Figura 5.13.



Figura 5.13: Botón de volver en la barra superior de la pantalla de ajustes

A continuación se muestra, en la Figura 5.14, un diagrama de flujo que resume este funcionamiento de manera gráfica.

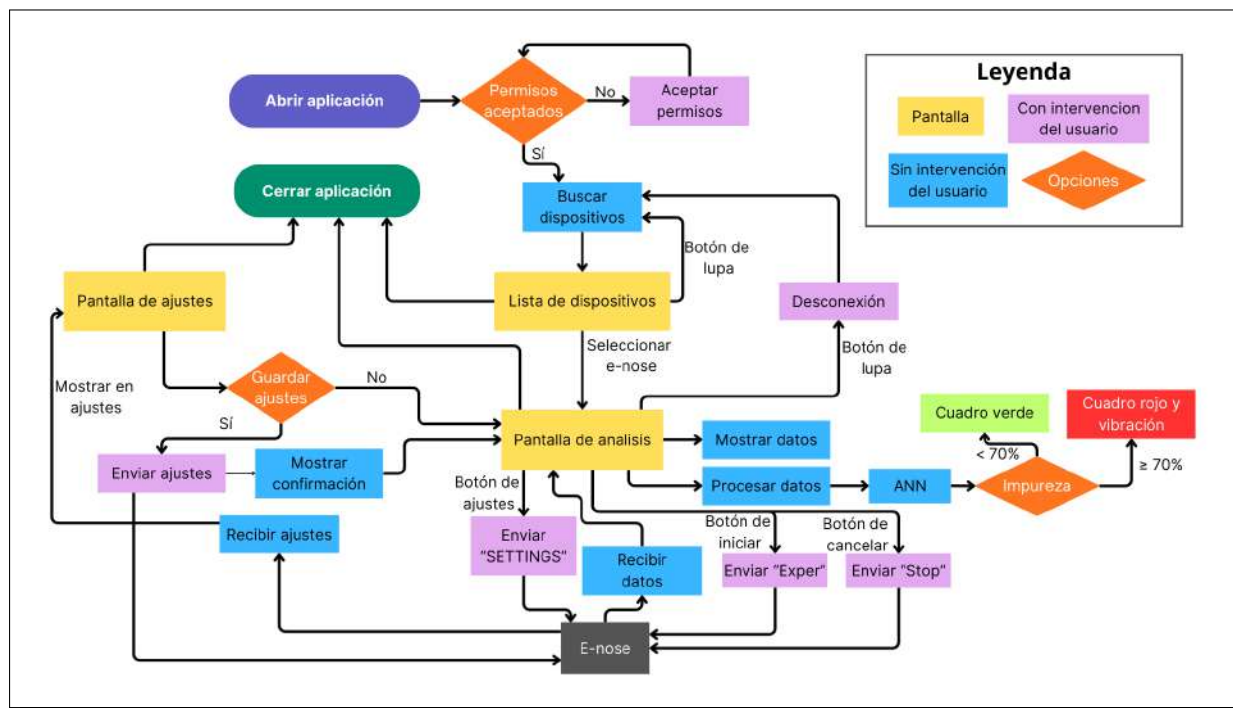


Figura 5.14: Diagrama de flujo del funcionamiento de la aplicación móvil

Además, este documento cuenta con los pseudocódigos de los archivos de la aplicación móvil en el Anexo E.

5.2. Pruebas comparativas

En esta sección se describen las distintas pruebas que se han llevado a cabo para evaluar el rendimiento de la nariz electrónica en comparación con el olfato canino bajo las mismas circunstancias. El modelo utilizado en las pruebas fue “Modelo Hachís Plus”, ya que ambos modelos presentan una gran precisión y este es el que aparece por defecto en la aplicación móvil.

Para estas pruebas se ha contado con la colaboración de un guía canino del cuerpo cinológico de la Guardia Civil, quien ha aportado su experiencia y la de su perro adiestrado

para la detección de drogas. En la Figura 5.15 es posible ver Zape, el agente canino que ha participado en las pruebas.



Figura 5.15: Imagen de Zape, agente canino de la Guardia Civil

Tras una conversación con el guía canino, se han decidido que pruebas llevar a cabo basándose en las prácticas habituales de entrenamiento y detección que suelen llevar a cabo en la comandancia. En total se han realizado 4 pruebas, además de otra prueba realizada sin comparar con Zape para coprobar el funcionamiento del sistema desarrollado, a esa prueba se la denominará prueba 0.

Para cada una de las pruebas realizadas, la persona que ha usado la nariz electrónica ha sido el propio autor de este trabajo, mientras que el guía canino ha sido el encargado de trabajar con Zape. En todas las pruebas se ha utilizado la misma cantidad de hachís proveniente de la misma fuente. Para realizar las pruebas de la nariz electrónica, el guía canino ocultaba la muestra para que el autor no supiera su ubicación, y viceversa, para evitar sesgos en los resultados.



A continuación se explican cada una de las pruebas realizadas:

5.2.1. Prueba 0: Cajas vacías

Una vez finalizado el entrenamiento de la red neuronal y el desarrollo de la aplicación móvil, se realizó una prueba inicial para verificar el correcto funcionamiento del sistema. En esta prueba se metió un recipiente con hachís dentro de una caja de cartón y se pasó la nariz por encima de la caja para comprobar si el sistema era capaz de detectar la sustancia oculta, seguido de otra pasada por una caja vacía para comprobar que no se detectaba nada cuando no había sustancia oculta. Esta prueba sirvió para validar que tanto la nariz electrónica como la aplicación móvil funcionaban correctamente antes de proceder con las pruebas comparativas con el perro adiestrado.

5.2.2. Prueba 1: Paquetería

Para esta prueba se simula la inspección de una furgoneta de reparto que contiene varios paquetes. Esos paquetes se simulan con cajas de cartón, unas vacías y otras con papel de relleno en su interior, colcadas al azar dentro del compartimento de una furgoneta de transporte de mercancías. En una de las cajas se oculta hachís que debe ser detectado por la nariz electrónica y Zape, pero no se usa la misma caja para cada uno, sino que se realiza la prueba con Zape, se quita la caja usada, y se añade una nueva, poniendo nuevamente el hachís en cualquiera de las cajas.

5.2.3. Prueba 2: Interior de vehículos

En esta prueba se simula la inspección del interior de un vehículo, en este caso un coche. Se ha hecho uso de coches fuera de uso y parcialmente desmantelados para evitar daños en vehículos en buen estado. Para la prueba de Zape se ha ocultado el recipiente con hachís en una cavidad situada detrás del plástico que bordea la radio. Luego, para



la prueba de la nariz electrónica, se ha pegado el recipiente a la parte superior del coche, concretamente en la parte interior del parasol.

5.2.4. Prueba 3: Personas

Otra práctica que hacen los guías caninos es la inspección de personas, por lo que se ha decidido incluir esta prueba en el conjunto de pruebas comparativas. Normalmente se trabaja en grandes grupos pero para esta prueba solo ha sido con una persona voluntaria. Dicha voluntaria es otra integrante de la Guardia Civil, la cual se oculta el recipiente con hachís en un bolsillo del pantalón para la prueba con Zape, y en la prueba con la nariz electrónica oculta el hachís en el interior de su camiseta.

5.2.5. Prueba 4: Equipajes

La última prueba consiste en la inspección de equipajes, simulando la revisión de maletas en un control de seguridad, los cuales pueden darse, por ejemplo, en aeropuertos o estaciones de tren. En esta prueba se han utilizado una maleta, una nevera y dos mochilas (una normal y otra de deporte). En este caso se ha ocultado el recipiente con hachís en el interior del bolsillo inferior de una de la mochila de deporte, donde había protecciones de entrenamiento, para ambas pruebas, tanto la de Zape como la de la nariz electrónica.

Capítulo 6

Resultados

En este capítulo se presentan los resultados obtenidos tras el entrenamiento y evaluación de los modelos de redes neuronales implementados en Keras en tiempo real, así como una comparación de los resultados de las evaluaciones. Además, se incluyen los resultados de las pruebas comparativas realizadas entre la nariz electrónica entrenada y el agente canino de la Guardia Civil encargado en la detección de drogas.

6.1. Resultados de redes neuronales

el rendimiento se visualiza mediante curvas de aprendizaje Accuracy (Precisión) y Loss (Pérdida) y una matriz de confusión.

Empezando con el modelo que no cuenta con datos aumentados (con una precisión del 96 %), se observan sus gráficas en la Figura 6.1 y su matriz de confusión en la Figura 6.2.

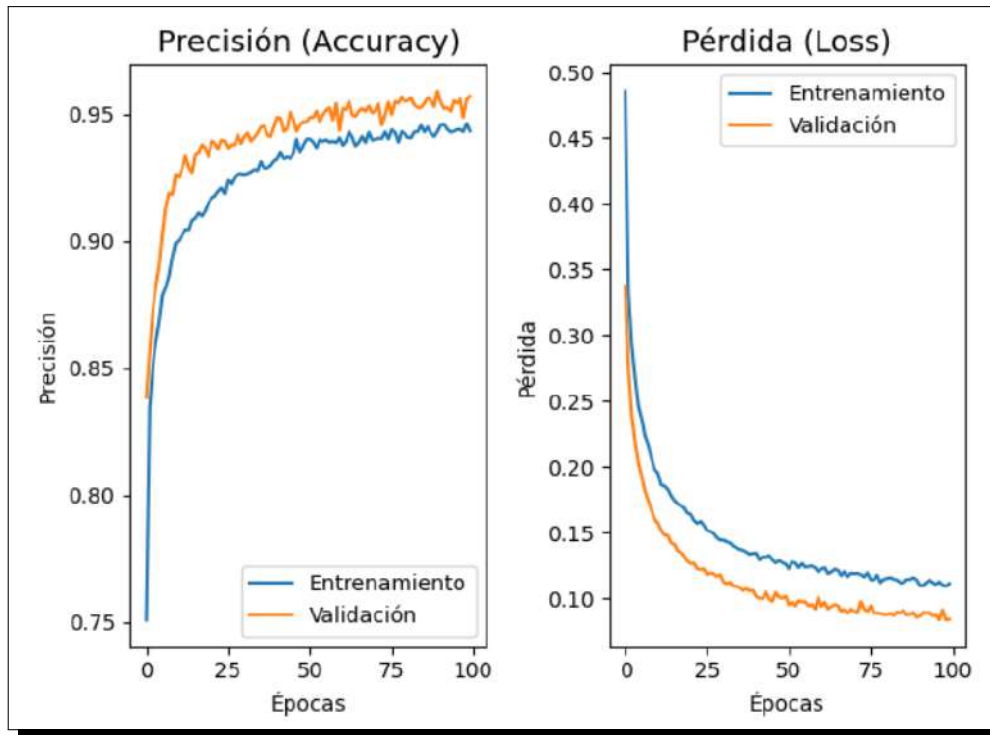


Figura 6.1: Curvas de aprendizaje del modelo ANN sin aumento de datos

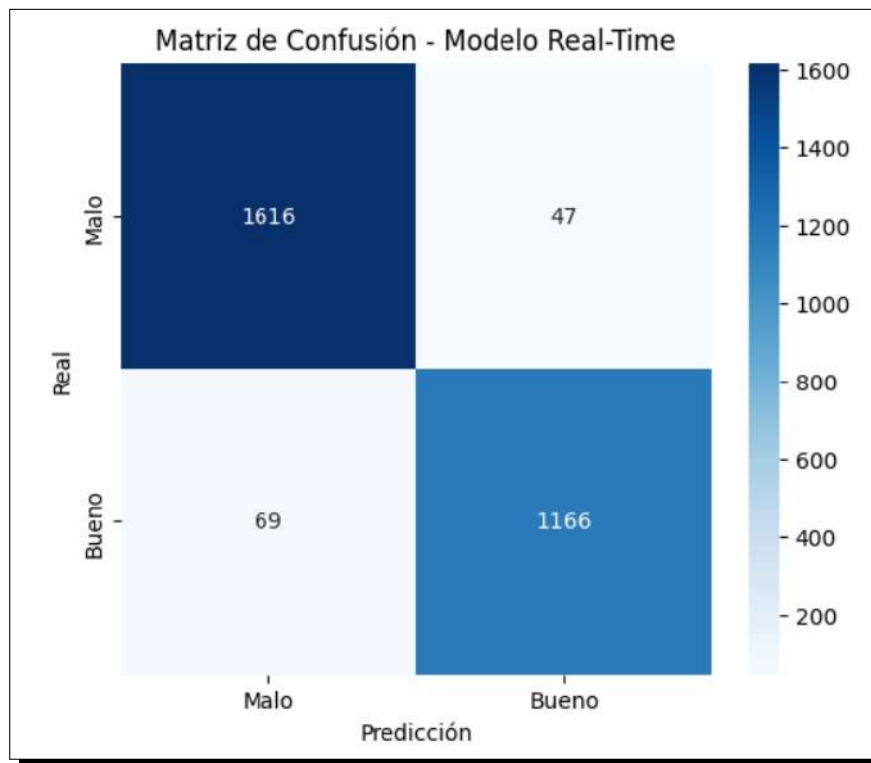


Figura 6.2: Matriz de confusión del modelo ANN sin aumento de datos

En la matriz de confusión se puede apreciar que, pese a haber falsos positivos y negativos, son pocos en comparación con los aciertos. También es posible observar que el número de falsos negativos es ligeramente mayor que el de falsos positivos, lo cual no es un problema dado que se toman datos cada 2 segundos y, como se toman tantos datos a la hora de buscar la sustancia, no es relevante que alguno de ellos sea un falso negativo.

En la gráfica de precisión se observa que tanto el conjunto de entrenamiento como el de validación alcanzan una precisión alta y similar, lo que indica que el modelo generaliza bien y no sufre de sobreajuste. Esto se respalda con la gráfica de pérdida, que muestra una disminución constante en ambas curvas, lo que también es un buen indicativo del rendimiento del modelo y una muestra de falta de sobreajuste. Además, se han completado todas las épocas disponibles sin que se haya activado el *Early Stopping*, lo que indica que el modelo ha podido aprender de manera efectiva durante todo el proceso de entrenamiento.

Continuando con el modelo que cuenta con datos aumentados (con una precisión del 94.67 %), se observan sus gráficas en la Figura 6.3 y su matriz de confusión en la Figura 6.4.

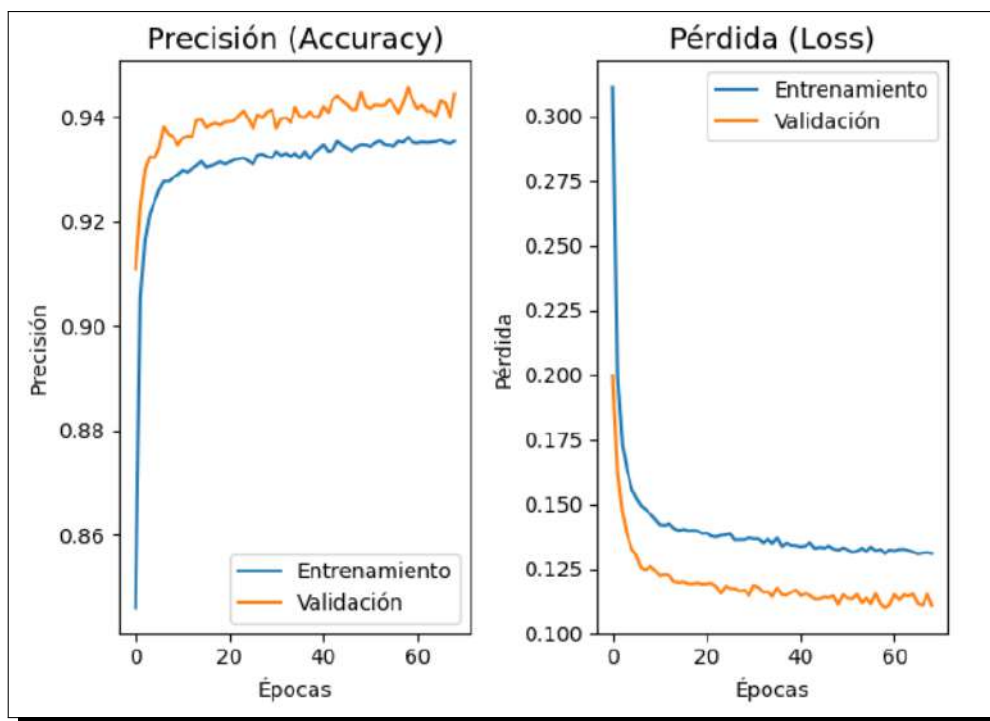


Figura 6.3: Curvas de aprendizaje del modelo ANN con aumento de datos

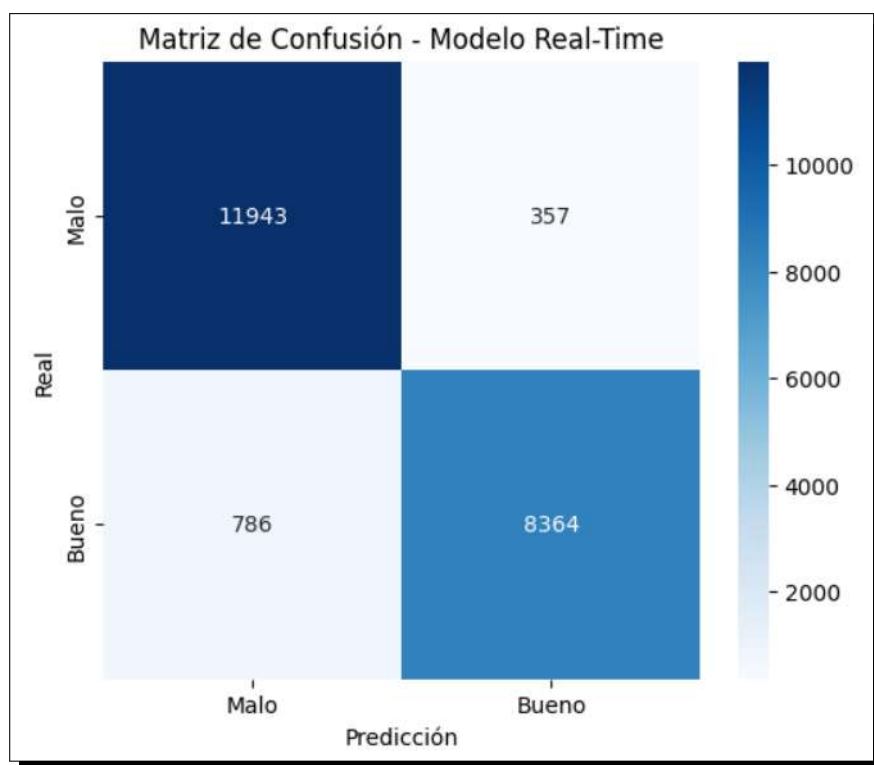


Figura 6.4: Matriz de confusión del modelo ANN con aumento de datos

En la matriz de confusión se observa un comportamiento similar al del modelo sin aumento de datos, con un número reducido de falsos positivos y negativos en comparación con los aciertos. De nuevo, el número de falsos negativos es ligeramente mayor que el de falsos positivos, lo cual no es un problema por el mismo motivo explicado anteriormente.

En la gráfica de precisión, tanto el conjunto de entrenamiento como el de validación alcanzan una precisión alta, aunque en este caso hay una mayor diferencia entre el entrenamiento y la validación, alcanzando valores más altos durante la validación. La gráfica de pérdida muestra una disminución constante en ambas curvas, siendo una disminución mayor en la validación, lo que también es un buen indicativo de que, pese a haber más diferencia entre entrenamiento y validación en la precisión, no hay sobreajuste.



Comparando ambos modelos, se observa que el modelo sin aumento de datos tiene una precisión ligeramente superior (96 % frente a 94.67 %). Sin embargo, ambos modelos muestran un buen rendimiento y generalización, con matrices de confusión que indican un bajo número de falsos positivos y falsos negativos.

6.2. Resultados de pruebas comparativas

Al llevar a cabo la prueba número 0, se pudo comprobar que la nariz electrónica funcionaba correctamente, ya que fue capaz de detectar el hachís oculto en una de las cajas vacías sin ningún problema.

Para poder cuantificar y comparar los resultados obtenidos por la nariz electrónica y Zape en el resto de pruebas, se han establecido una serie de métricas de evaluación para cada una de las pruebas. Estas métricas incluyen:

- **Acierto:** Si ha habido un falso positivo (-1), se ha podido detectar la sustancia objetivo correctamente (1) o no (0).
- **Tiempo:** Segundos que se ha tardado en dar una respuesta final cuando se detecta la sustancia objetivo.
- **Precisión:** Como de concreta es la zona o área marcada donde se detecta la sustancia objetivo (del 1 al 10).

A medida que se han ido realizando las pruebas, se han ido anotando los resultados obtenidos en una hoja de cálculo, la cual se muestra en la Figura 6.5.

PRUEBAS	A-Z	T(s)-Z	P-Z	A-N	T(s)-N	P-N
Paquetería	1	6	10	1	155	3
Interior de vehículos	1	25	10	0	440	0
Personas	1	15	10	0	390	3
Equipaje	1	22	10	1	170	10

Figura 6.5: Tabla comparativa de resultados entre Zape y la nariz electrónica



En esta tabla es posible ver las distintas pruebas realizadas, junto a la puntuación obtenida en cada una de las métricas establecidas para Zape y la nariz electrónica siendo A-Z y A-N los aciertos de Zape y la nariz electrónica respectivamente, T(s)-Z y T(s)-N los tiempos de Zape y la nariz electrónica respectivamente, y P-Z y P-N las precisiones de Zape y la nariz electrónica respectivamente.

A continuación se muestran métrica por métrica las puntuaciones asignadas a cada una de las pruebas realizadas, empezando por los aciertos, donde Zape ha conseguido una cantidad de aciertos del 100 %, habiendo detectado correctamente el hachís en todas las pruebas realizadas. Por otro lado, la nariz electrónica ha conseguido un 50 % de aciertos, habiendo fallado en las pruebas 2 y 3.

El fallo en la prueba 2 se debió a la fuerte corriente de aire que había en ese momento, lo cual dificulta en gran medida la obtención de partículas por parte de la nariz electrónica. En la prueba 3, el fallo se debió a que no se han incluido datos de entrenamiento relacionados con ropa, perfumes o colonias, el aroma del gel o champú que use dicha persona, o incluso el propio olor corporal humano.

Se decidió dar por fallida la prueba por parte de la nariz electrónica en el momento en el que se desveló la localización del hachís y, pese a saber donde estaba, no se consiguió detectar la sustancia.

Continuando con la velocidad, que está representada de forma gráfica en la Figura 6.6 se observa una clara diferencia entre ambos sistemas siendo Zape más rápido en la detección. Lo máximo que ha tardado Zape en detectar el hachís han sido 25 segundos en la segunda prueba, mientras que la nariz electrónica ha tenido un tiempo mínimo de 155 segundos (2 minutos y 35 segundos) en la primera prueba. Además en las pruebas 2 y 3, donde la nariz electrónica no ha conseguido detectar el hachís, se ha superado el tiempo máximo establecido de 5 minutos (300 segundos) por lo que después de un tiempo tras superarlo se decidió dar por finalizada la prueba. Esta gran cantidad de tiempo que ha tardado la nariz electrónica en detectar el hachís se debe a la velocidad con la que se toman los datos (cada 2 segundos) y a la necesidad de mover el tubo por toda la zona

para intentar localizar la sustancia, lo cual requiere mucho tiempo en comparación con Zape, que solo necesita unos segundos para analizar el aire y poder indicar en que zona está la sustancia.

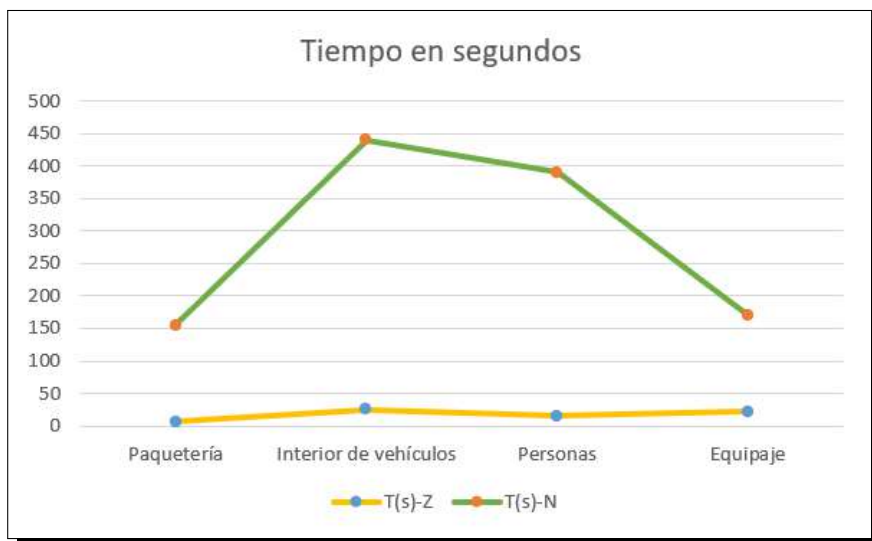


Figura 6.6: Comparativa de tiempos entre Zape y la nariz electrónica

Finalmente, la precisión está representada de forma gráfica en la Figura 6.7 donde se observa que Zape ha conseguido una precisión perfecta, obteniendo un 10 en todas las pruebas. Por otro lado, la nariz electrónica solo ha conseguido esta puntuación en la prueba 4, donde se pudo determinar bien en que mochila estaba la sustancia gracias a la subida de porcentaje que marcaba la aplicación, permitiendo concretar en los bolsillos de qué mochila había que indagar más. En la prueba 1 ha obtenido un 3 debido a las dificultades que tuvo para localizar la caja concreta donde estaba oculto el hachís, debido a que esta contenía papel de relleno que dificultaba la dispersión del aroma, teniendo que meter el tubo debajo del papel para poder detectar la sustancia. En la prueba 3 no consiguió detectar el hachís, pero se le ha asignado una puntuación de 3 debido a que, pese a no pasar el umbral de detección cuando debía hacerlo, marcaba un porcentaje del 35 %, indicando que había posibilidades de que la sustancia estuviera cerca (como ocurrió en la prueba 4), pero no terminó de diferenciarla. Por otro lado, en la prueba 2 se le ha asignado una puntuación de 0 al no haber detectado el hachís en ningún momento debido a que la corriente fuerte de aire impedía que la nariz electrónica pudiera captar bien las

partículas provenientes del recipiente con el hachís.

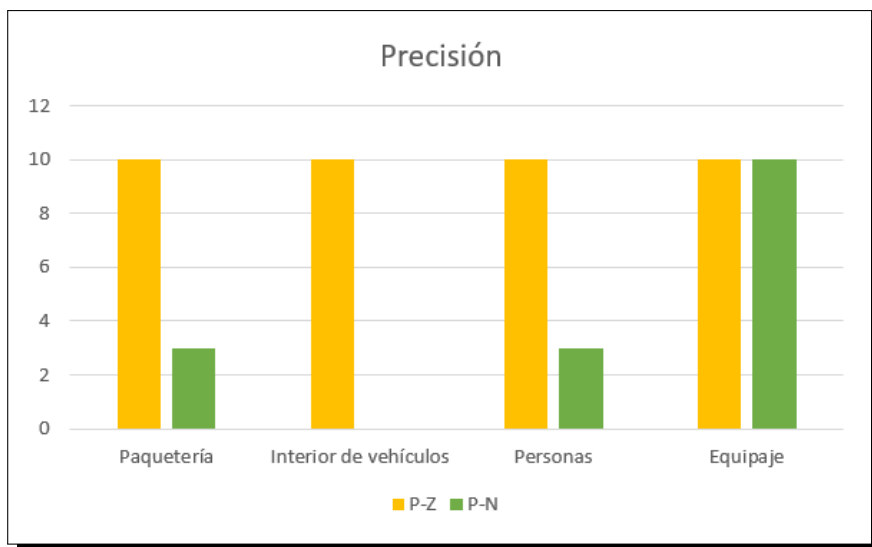


Figura 6.7: Comparativa de precisión entre Zape y la nariz electrónica

En resumen, Zape ha demostrado ser un sistema de detección de drogas altamente efectivo, superando a la nariz electrónica en todas las métricas evaluadas. La rapidez y precisión de Zape en la detección del hachís lo convierten en una herramienta valiosa para las fuerzas de seguridad en la lucha contra el tráfico de drogas. Por otro lado, pese a que en algunos casos ha dado resultados favorables, la nariz electrónica ha mostrado limitaciones en su capacidad para detectar el hachís en ciertas situaciones, sobre todo en condiciones ambientales adversas, lo que resalta la importancia de seguir mejorando y desarrollando tecnologías de detección más avanzadas.

Capítulo 7

Conclusiones y futuras mejoras

7.1. Conclusiones

Cumpliendo con los objetivos específicos planteados en el capítulo 2, se ha desarrollado una red neuronal capaz de detectar hachís a partir de los datos obtenidos por una nariz electrónica con los que se ha creado un *dataset* adecuado para el entrenamiento del modelo. Además, de desarrollar una aplicación que permita la comunicación entre la nariz electrónica y el modelo de detección, así como una interfaz gráfica para facilitar su uso por parte del usuario final.

Gracias a estos objetivos específicos se ha logrado cumplir el objetivo general de estudiar la viabilidad de una nariz electrónica como material de apoyo para las labores que desempeñan los agentes caninos y sus guías. Esto ha sido posible gracias a que se ha podido llevar a cabo una comparación entre la nariz electrónica y un perro de detección de drogas a través de una serie de pruebas bajo las mismas circunstancias, obteniendo resultados que demuestran la gran superioridad del perro en cuanto a rapidez, precisión y capacidad de detección. Estos resultados resaltan las limitaciones actuales de las narices electrónicas y la necesidad de seguir investigando y mejorando estas tecnologías para acercarse al rendimiento de los perros entrenados. Estas limitaciones indican que, en su



estado actual, las narices electrónicas no pueden usarse para trabajos como el control de vehículos en carreteras o la inspección de grandes áreas o grupos de personas, donde la rapidez es crucial y hay que abarcar una gran superficie de detección en poco tiempo.

Aun así también se ha podido demostrar que este sistema basado en una nariz electrónica y una red neuronal es capaz de detectar hachís en diferentes escenarios, por lo que puede ser viable su uso como apoyo para prestar este tipo de servicios en algunas situaciones concretas en las que los agentes caninos no estén disponibles o como complemento a estos, siempre que se disponga del tiempo necesario y no haya una gran corriente de aire que dificulte la detección, por ejemplo para registrar el equipaje de alguien en concreto por cualquier motivo, para los controles rutinarios de los aeropuertos o para el control de paquetes sospechosos. El resto de inconvenientes actuales, como en la detección de personas, pueden llegar a solucionarse con futuras mejoras en el sistema.

Después de poder ver el desempeño de la nariz electrónica en las pruebas realizadas, se puede aprender lo importante que es la variedad del *dataset* a la hora de entrenar la red neuronal para que esta pueda generalizar mejor y adaptarse a diferentes situaciones. También se ha podido observar que no hace falta que el *dataset* sea muy grande, sino que con un *dataset* de tamaño moderado pero con gran variedad de situaciones se pueden obtener buenos resultados. De hecho, en los entrenamientos realizados puede verse claramente, ya que en el entrenamiento con el *dataset* ampliado tuvo que intervenir la técnica de early stopping para evitar el sobreajuste, mientras que en el entrenamiento con el *dataset* reducido no fue necesario, lo que indica que la red neuronal pudo generalizar mejor con menos datos.

Por lo general, la mayoría de los trabajos con narices electrónicas se realizan en entornos controlados, como los laboratorios, pero, si bien el uso de narices electrónicas para la detección de sustancias es un campo explorado, este trabajo aporta novedad en la integración de un ecosistema portátil completo. La arquitectura desarrollada no solo clasifica la sustancia mediante una ANN optimizada para hachís (o cualquier otra sustancia con la que se desee entrenar la ANN), sino que permite la monitorización remota



en tiempo real mediante una interfaz móvil, reduciendo la latencia de respuesta.

Como conclusión final, se puede afirmar que, aunque las narices electrónicas aún no pueden realizar a la perfección la labor que realizan los perros de detección de drogas en la mayoría de las situaciones, el desarrollo de sistemas portátiles y autónomos basados en narices electrónicas y redes neuronales representa un avance significativo en la tecnología de detección de sustancias ilegales. Con futuras mejoras y avances tecnológicos, estas narices electrónicas podrían convertirse en herramientas valiosas para complementar el trabajo de los agentes caninos y sus guías.

7.2. Futuras mejoras

Existen varios aspectos en los que se podrían implementar mejoras para optimizar el rendimiento y la funcionalidad del sistema desarrollado. Algunas de estas mejoras incluyen:

7.2.1. Mejora del *dataset*

Cambiar el punto de vista del *dataset* en el que predomine la variedad frente la cantidad de datos de manera que la red neuronal aprenda a discriminar mejor en todos los casos posibles.

7.2.2. Desarrollo de nuevos modelos

Investigar y desarrollar modelos de detección para otras sustancias ilegales, ampliando así la utilidad del sistema. Gracias a la estructura modular del software, se podrían integrar fácilmente nuevos modelos para detectar diferentes drogas simplemente con entrenar el modelo con un *dataset* adecuado.



7.2.3. Especialización

Enfocar los modelos en un tipo de trabajo en concreto, como coches, paquetería, etc. De esta manera se podría optimizar el rendimiento del sistema para situaciones específicas además de no necesitar un dataset tan grande para cada modelo.

7.2.4. Cambiar la terminal en la interfaz de usuario

Para simplificar el sistema y hacerlo más intuitivo, se podría añadir una opción para ocultar la terminal donde llegan todos los datos y cambiarlo por aspectos visuales, tales como un perro que realice diferentes acciones dependiendo de si detecta o no la sustancia, o incluso que haya indicativos visuales dependiendo de si se ha comenzado el análisis o no.

Anexos

Anexo A

Aromas de varias sustancias

En este anexo se lleva un seguimiento de la obtención de las características de distintas sustancias que se usarán para el entrenamiento de la nariz electrónica y serán clasificadas como “MALO”.

Nombre	Sustancia	Condiciones	Control
EXP_M_1	Cacao en polvo (2 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 11-12-2025 ■ Hora inicio: 17:16 ■ Hora fin: 17:22 ■ Comentarios:
EXP_M_2-1	Limón exprimido (3 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 12-12-2025 ■ Hora inicio: 11:37 ■ Hora fin: 11:44 ■ Comentarios: Se ha cerrado la puerta a las 11:39.
EXP_M_2-2	Limón exprimido (3 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 12-12-2025 ■ Hora inicio: 11:56 ■ Hora fin: 12:03 ■ Comentarios:

Continúa en la siguiente página

Anexo A. Aromas de varias sustancias

Nombre	Sustancia	Condiciones	Control
EXP_M_3-1	Salsa chipotle y adobo (3 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 12-12-2025 ■ Hora inicio: 12:16 ■ Hora fin: 12:23 ■ Comentarios: Ha salido alguien de la habitación a las 12:17 y ha vuelto a entrar a las 12:18.
EXP_M_2-3	Limón exprimido (3 g).	Bote de muestreo abierto a temperatura ambiente con corriente de aire.	■ Fecha: 12-12-2025 ■ Hora inicio: 12:38 ■ Hora fin: 12:45 ■ Comentarios: El bote se ha volcado a las 12:44. No se ha derramado nada y ha sido un corto periodo de tiempo así que continua el experimento.
EXP_M_3-2	Salsa chipotle y adobo (3 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 12-12-2025 ■ Hora inicio: 13:13 ■ Hora fin: 13:20 ■ Comentarios:
EXP_M_3-2	Salsa chipotle y adobo (3 g).	Bote de muestreo abierto a temperatura ambiente con corriente de aire.	■ Fecha: 12-12-2025 ■ Hora inicio: 12:52 ■ Hora fin: 12:59 ■ Comentarios: Está lloviendo.
EXP_M_4-1	Sal (2 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 16-12-2025 ■ Hora inicio: 13:02 ■ Hora fin: 13:09 ■ Comentarios:

Continúa en la siguiente página

Anexo A. Aromas de varias sustancias

Nombre	Sustancia	Condiciones	Control
EXP_M_4-2	Sal (2 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 16-12-2025 ■ Hora inicio: 13:24 ■ Hora fin: 13:31 ■ Comentarios:
EXP_M_4-3	Sal (2 g).	Bote de muestreo abierto a temperatura ambiente con corriente de aire.	■ Fecha: 16-12-2025 ■ Hora inicio: 13:44 ■ Hora fin: 13:51 ■ Comentarios: Está lloviendo y entra bastante viento.
EXP_M_5-1	Salsa picante (6 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 23-12-2025 ■ Hora inicio: 10:59 ■ Hora fin: 11:06 ■ Comentarios: Ha salido una persona de la habitación a las 11:01.
EXP_M_5-2	Salsa picante (7 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 26-12-2025 ■ Hora inicio: 12:52 ■ Hora fin: 12:59 ■ Comentarios: La salsa está un poco fría.
EXP_M_5-3	Salsa picante (7 g).	Bote de muestreo abierto a temperatura ambiente con corriente de aire.	■ Fecha: 26-12-2025 ■ Hora inicio: 13:02 ■ Hora fin: 13:09 ■ Comentarios: La salsa está un poco fría.

Continúa en la siguiente página

Anexo A. Aromas de varias sustancias

Nombre	Sustancia	Condiciones	Control
EXP_M_6-1	Jengibre en polvo (3 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 11-01-2026 ■ Hora inicio: 19:22 ■ Hora fin: 19:29 ■ Comentarios:
EXP_M_6-2	Jengibre en polvo (3 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 11-01-2026 ■ Hora inicio: 16:31 ■ Hora fin: 16:38 ■ Comentarios:
EXP_M_6-3	Jengibre en polvo (3 g).	Bote de muestreo abierto a temperatura ambiente con corriente de aire.	■ Fecha: 11-01-2026 ■ Hora inicio: 16:43 ■ Hora fin: 16:50 ■ Comentarios:
EXP_M_7-1	Granos de pimienta (3 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 11-01-2026 ■ Hora inicio: 19:59 ■ Hora fin: 20:06 ■ Comentarios:
EXP_M_7-2	Granos de pimienta (3 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 11-01-2026 ■ Hora inicio: 20:15 ■ Hora fin: 20:22 ■ Comentarios:
EXP_M_7-3	Granos de pimienta (3 g).	Bote de muestreo abierto a temperatura ambiente con corriente de aire.	■ Fecha: 11-01-2026 ■ Hora inicio: 20:26 ■ Hora fin: 20:33 ■ Comentarios:
EXP_M_8-1	Café molido (2 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 12-01-2026 ■ Hora inicio: 10:26 ■ Hora fin: 10:33 ■ Comentarios:

Continúa en la siguiente página

Anexo A. Aromas de varias sustancias

Nombre	Sustancia	Condiciones	Control
EXP_M_8-2	Café molido (2 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 12-01-2026 ■ Hora inicio: 10:37 ■ Hora fin: 10:44 ■ Comentarios: Ha entrado alguien a la habitación a las 10:41.
EXP_M_8-3	Café molido (2 g).	Bote de muestreo abierto a temperatura ambiente con corriente de aire.	■ Fecha: 12-01-2026 ■ Hora inicio: 10:46 ■ Hora fin: 10:53 ■ Comentarios:
EXP_M_9-1	Canela en polvo (3 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 12-01-2026 ■ Hora inicio: 11:00 ■ Hora fin: 11:07 ■ Comentarios:
EXP_M_9-2	Canela en polvo (3 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 12-01-2026 ■ Hora inicio: 11:09 ■ Hora fin: 11:16 ■ Comentarios:
EXP_M_9-3	Canela en polvo (3 g).	Bote de muestreo abierto a temperatura ambiente con corriente de aire.	■ Fecha: 12-01-2026 ■ Hora inicio: 11:18 ■ Hora fin: 11:25 ■ Comentarios:
EXP_M_10-1	Cinta de embalar (0.5 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 12-01-2026 ■ Hora inicio: 11:34 ■ Hora fin: 11:41 ■ Comentarios:
EXP_M_10-2	Cinta de embalar (0.5 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 12-01-2026 ■ Hora inicio: 11:52 ■ Hora fin: 11:59 ■ Comentarios:

Continúa en la siguiente página

Anexo A. Aromas de varias sustancias

Nombre	Sustancia	Condiciones	Control
EXP_M_10-3	Cinta de embalar (0.5 g).	Bote de muestreo abierto a temperatura ambiente con corriente de aire.	■ Fecha: 12-01-2026 ■ Hora inicio: 12:01 ■ Hora fin: 12:08 ■ Comentarios:
EXP_M_11-1	Colonia de baño (4 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 12-01-2026 ■ Hora inicio: 12:19 ■ Hora fin: 12:26 ■ Comentarios: Ha tenido que abrirse la puerta a las 12:23 para que saliera el rumban de la habitación.
EXP_M_11-2	Colonia de baño (4 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 12-01-2026 ■ Hora inicio: 12:27 ■ Hora fin: 12:34 ■ Comentarios:
EXP_M_11-3	Colonia de baño (4 g).	Bote de muestreo abierto a temperatura ambiente con corriente de aire.	■ Fecha: 12-01-2026 ■ Hora inicio: 12:36 ■ Hora fin: 12:43 ■ Comentarios:
EXP_M_12-1	Amoniaco (4 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 12-01-2026 ■ Hora inicio: 12:51 ■ Hora fin: 12:58 ■ Comentarios: Ha tenido que abrirse la puerta a las 12:23 para que saliera el rumban de la habitación.

Continúa en la siguiente página

Anexo A. Aromas de varias sustancias

Nombre	Sustancia	Condiciones	Control
EXP_M_12-2	Amoniac (4 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 12-01-2026 ■ Hora inicio: 12:59 ■ Hora fin: 13:06 ■ Comentarios: Alguien ha entrado a la habitación a las 13:05.
EXP_M_12-3	Amoniac (4 g).	Bote de muestreo abierto a temperatura ambiente con corriente de aire.	■ Fecha: 12-01-2026 ■ Hora inicio: 13:08 ■ Hora fin: 13:15 ■ Comentarios:
EXP_M_13	Habitación.	Puertas y ventanas cerradas a temperatura ambiente.	■ Fecha: 13-01-2026 ■ Hora inicio: 10:21 ■ Hora fin: 10:28 ■ Comentarios:
EXP_M_14	Caja de carton.	<i>e-nose</i> dentro de la caja cerrada a temperatura ambiente.	■ Fecha: 13-01-2026 ■ Hora inicio: 10:42 ■ Hora fin: 10:49 ■ Comentarios:
EXP_M_15	Debajo del asiento del coche.	Frío en el exterior pero temperatura normal el interior del coche.	■ Fecha: 13-01-2026 ■ Hora inicio: 17:43 ■ Hora fin: 17:50 ■ Comentarios:
EXP_M_16	En el maletero del coche.	Frío en el exterior pero temperatura normal el interior del coche.	■ Fecha: 13-01-2026 ■ Hora inicio: 17:52 ■ Hora fin: 17:59 ■ Comentarios:
EXP_M_17-1	Caja de carton.	<i>e-nose</i> fuera de la caja abierta, con tubo dentro, a temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 10:07 ■ Hora fin: 10:14 ■ Comentarios:

Continúa en la siguiente página

Anexo A. Aromas de varias sustancias

Nombre	Sustancia	Condiciones	Control
EXP_M_17-2	Caja de carton.	<i>e-nose</i> fuera de la caja abierta, con tubo dentro, a temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 10:19 ■ Hora fin: 10:26 ■ Comentarios:
EXP_M_17-3	Caja de carton.	<i>e-nose</i> fuera de la caja abierta, con tubo dentro, a temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 10:48 ■ Hora fin: 10:55 ■ Comentarios:
EXP_M_17-4	Caja de carton.	<i>e-nose</i> fuera de la caja abierta, con tubo dentro, a temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 10:56 ■ Hora fin: 11:03 ■ Comentarios:
EXP_M_17-5	Caja de carton.	<i>e-nose</i> fuera de la caja abierta, con tubo dentro, a temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 11:25 ■ Hora fin: 11:32 ■ Comentarios:
EXP_M_17-6	Caja de carton.	<i>e-nose</i> fuera de la caja abierta, con tubo dentro, a temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 11:34 ■ Hora fin: 11:41 ■ Comentarios: Ha salido alguien de la habitación a las 11:35 y ha vuelto a entrar a las 11:36.
EXP_M_17-7	Caja de carton.	<i>e-nose</i> fuera de la caja abierta, con tubo dentro, a temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 12:01 ■ Hora fin: 12:08 ■ Comentarios:
EXP_M_17-8	Caja de carton.	<i>e-nose</i> fuera de la caja abierta, con tubo dentro, a temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 12:09 ■ Hora fin: 12:16 ■ Comentarios:

Continúa en la siguiente página

Anexo A. Aromas de varias sustancias

Nombre	Sustancia	Condiciones	Control
EXP_M_18-1	Habitación (Cocina).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 13:49 ■ Hora fin: 13:56 ■ Comentarios:
EXP_M_18-2	Habitación.	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 13:57 ■ Hora fin: 14:04 ■ Comentarios:
EXP_M_18-3	Habitación.	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 14:06 ■ Hora fin: 14:13 ■ Comentarios:
EXP_M_18-4	Habitación (Baño).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 14:14 ■ Hora fin: 14:21 ■ Comentarios:
EXP_M_19-1	Cajón medio abierto.	<i>e-nose</i> dentro del cajón a temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 18:35 ■ Hora fin: 18:42 ■ Comentarios:
EXP_M_19-2	Cajón medio abierto.	<i>e-nose</i> dentro del cajón a temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 18:49 ■ Hora fin: 18:56 ■ Comentarios: Ha entrado y saldio seguidamente alguien de la habitación a las 18:54

Continúa en la siguiente página

Anexo A. Aromas de varias sustancias

Nombre	Sustancia	Condiciones	Control
EXP_M_19-3	Cajón medio abierto.	<i>e-nose</i> sobre del cajón a temperatura ambiente.	■ Fecha: 22-01-2026 ■ Hora inicio: 10:32 ■ Hora fin: 10:39 ■ Comentarios:
EXP_M_19-4	Cajón medio abierto.	<i>e-nose</i> sobre del cajón a temperatura ambiente.	■ Fecha: 22-01-2026 ■ Hora inicio: 10:40 ■ Hora fin: 10:47 ■ Comentarios:
EXP_M_20-1	Bolsillo de una maleta.	<i>e-nose</i> fuera del bolsillo con tubo dentro a temperatura ambiente.	■ Fecha: 23-01-2026 ■ Hora inicio: 11:52 ■ Hora fin: 11:59 ■ Comentarios:
EXP_M_20-2	Bolsillo de una maleta.	<i>e-nose</i> fuera del bolsillo con tubo dentro a temperatura ambiente.	■ Fecha: 23-01-2026 ■ Hora inicio: 12:04 ■ Hora fin: 12:11 ■ Comentarios:
EXP_M_20-3	Maleta.	<i>e-nose</i> fuera del de la maleta con tubo dentro a temperatura ambiente.	■ Fecha: 23-01-2026 ■ Hora inicio: 12:45 ■ Hora fin: 12:52 ■ Comentarios:
EXP_M_20-4	Maleta.	<i>e-nose</i> fuera del de la maleta con tubo dentro a temperatura ambiente.	■ Fecha: 23-01-2026 ■ Hora inicio: 12:56 ■ Hora fin: 13:03 ■ Comentarios:
EXP_M_21-1	Habitación.	Temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 16:48 ■ Hora fin: 16:55 ■ Comentarios:
EXP_M_21-2	Habitación.	Temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 16:58 ■ Hora fin: 17:05 ■ Comentarios:

Continúa en la siguiente página

Anexo A. Aromas de varias sustancias

Nombre	Sustancia	Condiciones	Control
EXP_M_21-3	Habitación.	Temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 17:08 ■ Hora fin: 17:15 ■ Comentarios:
EXP_M_21-4	Habitación.	Temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 17:20 ■ Hora fin: 17:27 ■ Comentarios:
EXP_M_22-1	Maleta.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 17:34 ■ Hora fin: 17:41 ■ Comentarios:
EXP_M_22-2	Maleta.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 17:58 ■ Hora fin: 18:05 ■ Comentarios:
EXP_M_22-3	Maleta.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 18:13 ■ Hora fin: 18:20 ■ Comentarios:
EXP_M_22-4	Maleta.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 18:22 ■ Hora fin: 18:29 ■ Comentarios:
EXP_M_23-1	Caja.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 18:50 ■ Hora fin: 18:57 ■ Comentarios:
EXP_M_23-2	Caja.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 19:00 ■ Hora fin: 19:07 ■ Comentarios:

Continúa en la siguiente página

Anexo A. Aromas de varias sustancias

Nombre	Sustancia	Condiciones	Control
EXP_M_23-3	Caja.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 19:18 ■ Hora fin: 19:25 ■ Comentarios:
EXP_M_23-4	Caja.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 19:40 ■ Hora fin: 19:47 ■ Comentarios:
EXP_M_23-5	Caja.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 19:51 ■ Hora fin: 19:58 ■ Comentarios:
EXP_M_23-6	Caja.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 20:02 ■ Hora fin: 20:09 ■ Comentarios:
EXP_M_23-8	Caja.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 24-01-2026 ■ Hora inicio: 20:20 ■ Hora fin: 20:27 ■ Comentarios:
EXP_M_23-9	Caja.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 25-01-2026 ■ Hora inicio: 11:10 ■ Hora fin: 11:17 ■ Comentarios:
EXP_M_23-10	Caja.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 25-01-2026 ■ Hora inicio: 11:21 ■ Hora fin: 11:28 ■ Comentarios:
EXP_M_23-11	Caja.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 25-01-2026 ■ Hora inicio: 11:29 ■ Hora fin: 11:36 ■ Comentarios:

Continúa en la siguiente página

Anexo A. Aromas de varias sustancias

Nombre	Sustancia	Condiciones	Control
EXP_M_23-12	Caja.	<i>e-nose</i> dentro a temperatura ambiente.	■ Fecha: 25-01-2026 ■ Hora inicio: 11:38 ■ Hora fin: 11:45 ■ Comentarios:
EXP_M_24-1	Aire limpio.	<i>e-nose</i> en una habitación desocupada con filtro de aire en la entrada de adsorción a temperatura ambiente.	■ Fecha: 25-01-2026 ■ Hora inicio: 13:26 ■ Hora fin: 13:33 ■ Comentarios:
EXP_M_24-2	Aire limpio.	<i>e-nose</i> en una habitación desocupada con filtro de aire en la entrada de adsorción a temperatura ambiente.	■ Fecha: 25-01-2026 ■ Hora inicio: 13:36 ■ Hora fin: 13:43 ■ Comentarios:
EXP_M_24-3	Aire limpio.	<i>e-nose</i> en una habitación desocupada con filtro de aire en la entrada de adsorción a temperatura ambiente.	■ Fecha: 25-01-2026 ■ Hora inicio: 13:45 ■ Hora fin: 13:52 ■ Comentarios:
EXP_M_24-4	Aire limpio.	<i>e-nose</i> en una habitación desocupada con filtro de aire en la entrada de adsorción a temperatura ambiente.	■ Fecha: 25-01-2026 ■ Hora inicio: 13:57 ■ Hora fin: 14:04 ■ Comentarios:

Tabla A.2: Registro de mediciones de sustancias y condiciones de muestreo.

Anexo B

Aroma de hachís

En este anexo se lleva un seguimiento de la obtención de las características de la sustancia objetivo en distintas condiciones. Estos datos se usarán para el entrenamiento de la nariz electrónica y serán clasificadas como “BUENO”.

Para este proyecto la sustancia a detectar será hachís, ofrecido por el cuerpo cinológico de la Guardia Civil.

Nombre	Forma de la muestra	Condiciones	Control
EXP_B_1	En un recipiente destapado (5 g).	Bote de muestreo cerrado con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 9:26 ■ Hora fin: 9:33 ■ Comentarios: Hay tres personas en la sala.
EXP_B_2	En un recipiente destapado (5 g).	Bote de muestreo cerrado con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 9:38 ■ Hora fin: 9:45 ■ Comentarios: Hay tres personas en la sala.

Continúa en la siguiente página

Nombre	Forma de la muestra	Condiciones	Control
EXP_B_3	En un recipiente destapado (5 g).	Bote de muestreo abierto con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 9:53 ■ Hora fin: 10:00 ■ Comentarios: Hay tres personas en la sala.
EXP_B_4	En un recipiente destapado (5 g).	Bote de muestreo abierto con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 10:02 ■ Hora fin: 10:09 ■ Comentarios: Hay tres personas en la sala.
EXP_B_5	En un recipiente tapado (5 g).	Bote de muestreo cerrado con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 10:12 ■ Hora fin: 10:19 ■ Comentarios: Hay tres personas en la sala.
EXP_B_6	En un recipiente tapado (5 g).	Bote de muestreo cerrado con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 10:22 ■ Hora fin: 10:29 ■ Comentarios: Hay tres personas en la sala.
EXP_B_7	En un recipiente tapado (5 g).	Bote de muestreo abierto con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 10:31 ■ Hora fin: 10:38 ■ Comentarios: Hay tres personas en la sala y se ha abierto un momento la puerta.

Continúa en la siguiente página

Nombre	Forma de la muestra	Condiciones	Control
EXP_B_8	En un recipiente tapado (5 g).	Bote de muestreo abierto con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 10:39 ■ Hora fin: 10:46 ■ Comentarios: Hay tres personas en la sala y se ha abierto un momento la puerta.
EXP_B_9	En un recipiente tapado (7 g).	Bote de muestreo cerrado con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 12:06 ■ Hora fin: 12:13 ■ Comentarios:
EXP_B_10	En un recipiente tapado (7 g).	Bote de muestreo cerrado con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 12:21 ■ Hora fin: 12:28 ■ Comentarios: La puerta está abierta.
EXP_B_11	En un recipiente tapado (7 g).	Bote de muestreo abierto con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 12:30 ■ Hora fin: 12:37 ■ Comentarios: La puerta está abierta. Se ha cerrado a las 12:35.
EXP_B_12	En un recipiente tapado (7 g).	Bote de muestreo abierto con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 12:41 ■ Hora fin: 12:48 ■ Comentarios: Ha salido una persona de la sala a las 12:44.

Continúa en la siguiente página

Anexo B. Aroma de hachís

Nombre	Forma de la muestra	Condiciones	Control
EXP_B_13	En un recipiente abierto (7 g).	Bote de muestreo cerrado con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 13:05 ■ Hora fin: 13:12 ■ Comentarios: Ha salido una persona a las 13:06.
EXP_B_14	En un recipiente abierto (7 g).	Bote de muestreo cerrado con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 13:14 ■ Hora fin: 13:21 ■ Comentarios: Ha salido alguien de la sala a las 13:21.
EXP_B_15	En un recipiente abierto (7 g).	Bote de muestreo abierto con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 13:23 ■ Hora fin: 13:30 ■ Comentarios:
EXP_B_16	En un recipiente abierto (7 g).	Bote de muestreo abierto con radiador encendido para evitar el frío.	■ Fecha: 15-12-2025 ■ Hora inicio: 13:32 ■ Hora fin: 13:39 ■ Comentarios:
EXP_B_17	Cinta de embalar con restos de hachís (3 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 16-12-2025 ■ Hora inicio: 11:02 ■ Hora fin: 11:09 ■ Comentarios:
EXP_B_18	Cinta de embalar con restos de hachís (3 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 16-12-2025 ■ Hora inicio: 11:19 ■ Hora fin: 11:26 ■ Comentarios:
EXP_B_19	Cinta de embalar con restos de hachís (3 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 16-12-2025 ■ Hora inicio: 11:38 ■ Hora fin: 11:45 ■ Comentarios:

Continúa en la siguiente página

Anexo B. Aroma de hachís

Nombre	Forma de la muestra	Condiciones	Control
EXP_B_20	Cinta de embalar con restos de hachís (3 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 16-12-2025 ■ Hora inicio: 11:47 ■ Hora fin: 11:54 ■ Comentarios:
EXP_B_21	Pequeño bloque sin cubrir (2 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 16-12-2025 ■ Hora inicio: 12:03 ■ Hora fin: 12:10 ■ Comentarios:
EXP_B_22	Pequeño bloque sin cubrir (2 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 16-12-2025 ■ Hora inicio: 12:13 ■ Hora fin: 12:20 ■ Comentarios:
EXP_B_23	Pequeño bloque sin cubrir (2 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 16-12-2025 ■ Hora inicio: 12:22 ■ Hora fin: 12:29 ■ Comentarios:
EXP_B_24	Pequeño bloque sin cubrir (2 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 16-12-2025 ■ Hora inicio: 12:33 ■ Hora fin: 12:40 ■ Comentarios:
EXP_B_25	Envuelto en plastico, metido en un recipiente cerrado (5 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 17-12-2025 ■ Hora inicio: 9:35 ■ Hora fin: 9:42 ■ Comentarios:
EXP_B_26	Envuelto en plastico, metido en un recipiente cerrado (5 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 17-12-2025 ■ Hora inicio: 9:44 ■ Hora fin: 9:51 ■ Comentarios:
EXP_B_27	Envuelto en plastico, metido en un recipiente cerrado (5 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 17-12-2025 ■ Hora inicio: 9:53 ■ Hora fin: 10:00 ■ Comentarios:

Continúa en la siguiente página

Anexo B. Aroma de hachís

Nombre	Forma de la muestra	Condiciones	Control
EXP_B_28	Envuelto en plastico, metido en un recipiente cerrado (5 g).	Bote de muestreo abierto a temperatura ambiente.	■ Fecha: 17-12-2025 ■ Hora inicio: 10:02 ■ Hora fin: 10:09 ■ Comentarios:
EXP_B_29	Envuelto en plastico, metido en un recipiente abiaerto (5 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 17-12-2025 ■ Hora inicio: 10:46 ■ Hora fin: 10:53 ■ Comentarios:
EXP_B_30	Envuelto en plastico, metido en un recipiente abiaerto (5 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 17-12-2025 ■ Hora inicio: 10:55 ■ Hora fin: 11:02 ■ Comentarios:
EXP_B_31	Envuelto en plastico, metido en un recipiente abierto (5 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 17-12-2025 ■ Hora inicio: 11:04 ■ Hora fin: 11:11 ■ Comentarios:
EXP_B_32	Envuelto en plastico, metido en un recipiente abierto (5 g).	Bote de muestreo cerrado a temperatura ambiente.	■ Fecha: 17-12-2025 ■ Hora inicio: 11:14 ■ Hora fin: 11:21 ■ Comentarios:
EXP_B_33	Bien envuelto metido en un recipiente de cristal con tapa sobrepuesta (98 g).	Temperatura ambiente.	■ Fecha: 17-12-2025 ■ Hora inicio: 11:45 ■ Hora fin: 11:52 ■ Comentarios:
EXP_B_34	Bien envuelto metido en un recipiente de cristal destapado (98 g).	Temperatura ambiente.	■ Fecha: 17-12-2025 ■ Hora inicio: 11:57 ■ Hora fin: 12:04 ■ Comentarios:
EXP_B_35	Bien envuelto metido en un recipiente de cristal destapado (96 g).	Temperatura ambiente.	■ Fecha: 17-12-2025 ■ Hora inicio: 12:20 ■ Hora fin: 12:27 ■ Comentarios:

Continúa en la siguiente página

Nombre	Forma de la muestra	Condiciones	Control
EXP_B_36	Bien envuelto metido una caja de cartón (96 g).	Temperatura ambiente.	■ Fecha: 17-12-2025 ■ Hora inicio: 12:46 ■ Hora fin: 12:53 ■ Comentarios:
EXP_B_37	Bien envuelto metido en la guantera de la puerta del coche (96 g).	Frío en el exterior pero temperatura normal el interior del coche.	■ Fecha: 17-12-2025 ■ Hora inicio: 13:27 ■ Hora fin: 13:34 ■ Comentarios: La muestra se ha metido en el coche hace 25 minutos. Hay más cosas en la guantera.
EXP_B_38	Bien envuelto metido debajo del asiento del coche (96 g).	Frío en el exterior pero temperatura normal el interior del coche.	■ Fecha: 17-12-2025 ■ Hora inicio: 13:37 ■ Hora fin: 13:44 ■ Comentarios: La muestra se ha metido en el coche hace 35 minutos.
EXP_B_39	En un bote de cristal metido en una caja de cartón (5 g).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 10:29 ■ Hora fin: 10:36 ■ Comentarios:
EXP_B_40	En un bote de cristal metido en una caja de cartón (5 g).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 10:39 ■ Hora fin: 10:46 ■ Comentarios:
EXP_B_41	En un bote de cristal metido en una caja de cartón (5 g).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 11:06 ■ Hora fin: 11:13 ■ Comentarios:

Continúa en la siguiente página

Anexo B. Aroma de hachís

Nombre	Forma de la muestra	Condiciones	Control
EXP_B_42	En un bote de cristal metido en una caja de cartón (5 g).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 11:14 ■ Hora fin: 11:21 ■ Comentarios:
EXP_B_43	En un bote de cristal metido en una caja de cartón (5 g).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 11:43 ■ Hora fin: 11:50 ■ Comentarios:
EXP_B_44	En un bote de cristal metido en una caja de cartón (5 g).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 11:52 ■ Hora fin: 11:59 ■ Comentarios:
EXP_B_45	En un bote de cristal metido en una caja de cartón (5 g).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 12:18 ■ Hora fin: 12:25 ■ Comentarios:
EXP_B_46	En un bote de cristal metido en una caja de cartón (5 g).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 12:44 ■ Hora fin: 12:51 ■ Comentarios:
EXP_B_47	En un bote de cristal cerca de la nariz (5 g).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 16:31 ■ Hora fin: 16:38 ■ Comentarios:
EXP_B_48	En un bote de cristal cerca de la nariz (5 g).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 16:40 ■ Hora fin: 16:47 ■ Comentarios: Se ha abierto un momento la puerta a las 16:43.

Continúa en la siguiente página

Anexo B. Aroma de hachís

Nombre	Forma de la muestra	Condiciones	Control
EXP_B_49	En un bote de cristal alejado de la nariz (5 g).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 16:51 ■ Hora fin: 16:58 ■ Comentarios:
EXP_B_50	En un bote de cristal alejado de la nariz (5 g).	Temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 16:59 ■ Hora fin: 17:06 ■ Comentarios:
EXP_B_51	En un bote de cristal dentro de un cajón abierto (5 g).	<i>e-nose</i> sobre el borde del cajón a temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 17:29 ■ Hora fin: 17:36 ■ Comentarios:
EXP_B_52	En un bote de cristal dentro de un cajón abierto (5 g).	<i>e-nose</i> sobre el borde del cajón a temperatura ambiente.	■ Fecha: 21-01-2026 ■ Hora inicio: 17:38 ■ Hora fin: 17:45 ■ Comentarios:
EXP_B_53	En un bote de cristal dentro de un cajón abierto (casi a en la entrada) (5 g).	<i>e-nose</i> sobre el borde del cajón a temperatura ambiente.	■ Fecha: 23-01-2026 ■ Hora inicio: 11:27 ■ Hora fin: 11:34 ■ Comentarios:
EXP_B_54	En un bote de cristal dentro de un cajón abierto (al fondo) (5 g).	<i>e-nose</i> sobre el borde del cajón a temperatura ambiente.	■ Fecha: 23-01-2026 ■ Hora inicio: 11:36 ■ Hora fin: 11:43 ■ Comentarios:
EXP_B_55	En un bote de cristal en el bolsillo de una maleta (5 g).	<i>e-nose</i> fuera del bolsillo con tubo dentro a temperatura ambiente.	■ Fecha: 23-01-2026 ■ Hora inicio: 12:19 ■ Hora fin: 12:26 ■ Comentarios:
EXP_B_56	En un bote de cristal en el bolsillo de una maleta (5 g).	<i>e-nose</i> fuera del bolsillo con tubo dentro a temperatura ambiente.	■ Fecha: 23-01-2026 ■ Hora inicio: 12:29 ■ Hora fin: 12:36 ■ Comentarios:

Continúa en la siguiente página

Nombre	Forma de la muestra	Condiciones	Control
EXP_B_57	En un bote de cristal en una maleta (5 g).	<i>e-nose</i> fuera del de la maleta con tubo dentro a temperatura ambiente.	■ Fecha: 23-01-2026 ■ Hora inicio: 13:10 ■ Hora fin: 13:17 ■ Comentarios:
EXP_B_58	En un bote de cristal en una maleta (5 g).	<i>e-nose</i> fuera del de la maleta con tubo dentro a temperatura ambiente.	■ Fecha: 23-01-2026 ■ Hora inicio: 13:22 ■ Hora fin: 13:29 ■ Comentarios:
EXP_B_59	En un bote de cristal en una caja (5 g).	Tubo de <i>e-nose</i> dentro de la caja a temperatura ambiente.	■ Fecha: 25-01-2026 ■ Hora inicio: 12:09 ■ Hora fin: 12:16 ■ Comentarios:
EXP_B_60	En un bote de cristal en una caja (5 g).	Tubo de <i>e-nose</i> dentro de la caja a temperatura ambiente.	■ Fecha: 25-01-2026 ■ Hora inicio: 12:10 ■ Hora fin: 12:26 ■ Comentarios:
EXP_B_61	En un bote de cristal en una caja (5 g).	Tubo de <i>e-nose</i> dentro de la caja a temperatura ambiente.	■ Fecha: 25-01-2026 ■ Hora inicio: 12:33 ■ Hora fin: 12:49 ■ Comentarios:
EXP_B_62	En un bote de cristal en una caja (5 g).	Tubo de <i>e-nose</i> dentro de la caja a temperatura ambiente.	■ Fecha: 25-01-2026 ■ Hora inicio: 12:52 ■ Hora fin: 13:08 ■ Comentarios:

Tabla B.2: Registro de mediciones de hachís y condiciones de muestreo.

Anexo C

Configuración de la aplicación para la toma de muestras

El nombre de la aplicación utilizada para la toma de muestras es *Serial Bluetooth Terminal* y puede descargarse directamente desde la Play Store. Véase en la Figura C.1.



Figura C.1: Aplicación Serial Bluetooth Terminal en la Play Store.

C.1. Configuración de ajustes

Una vez instalada, es necesario configurar la aplicación para que los datos se reciban correctamente y la vista en la interfaz sea más clara. Para acceder a los ajustes hay que pulsar el icono de las tres rayas horizontales en la esquina superior izquierda de la pantalla principal de la aplicación. A continuación, se



debe seleccionar la opción *Settings*. Una vez dentro de los ajustes pueden verse cuatro pestañas: Terminal, Receive, Send y Misc. A continuación se describen los ajustes necesarios en cada una de las pestañas.

C.1.1. Terminal

Esta configuración puede verse en la Figura C.2.

- **Font Size:** 14.
- **Font Style:** Normal.
- **Charset:** UTF-8.
- **Display mode:** Terminal.
- **Auto scroll to end of buffer:** Debe estar activado.
- **Show connection messages:** Debe estar activado.
- **Show timestamps:** Debe estar desactivado.
- **Buffer size:** Unlimited.

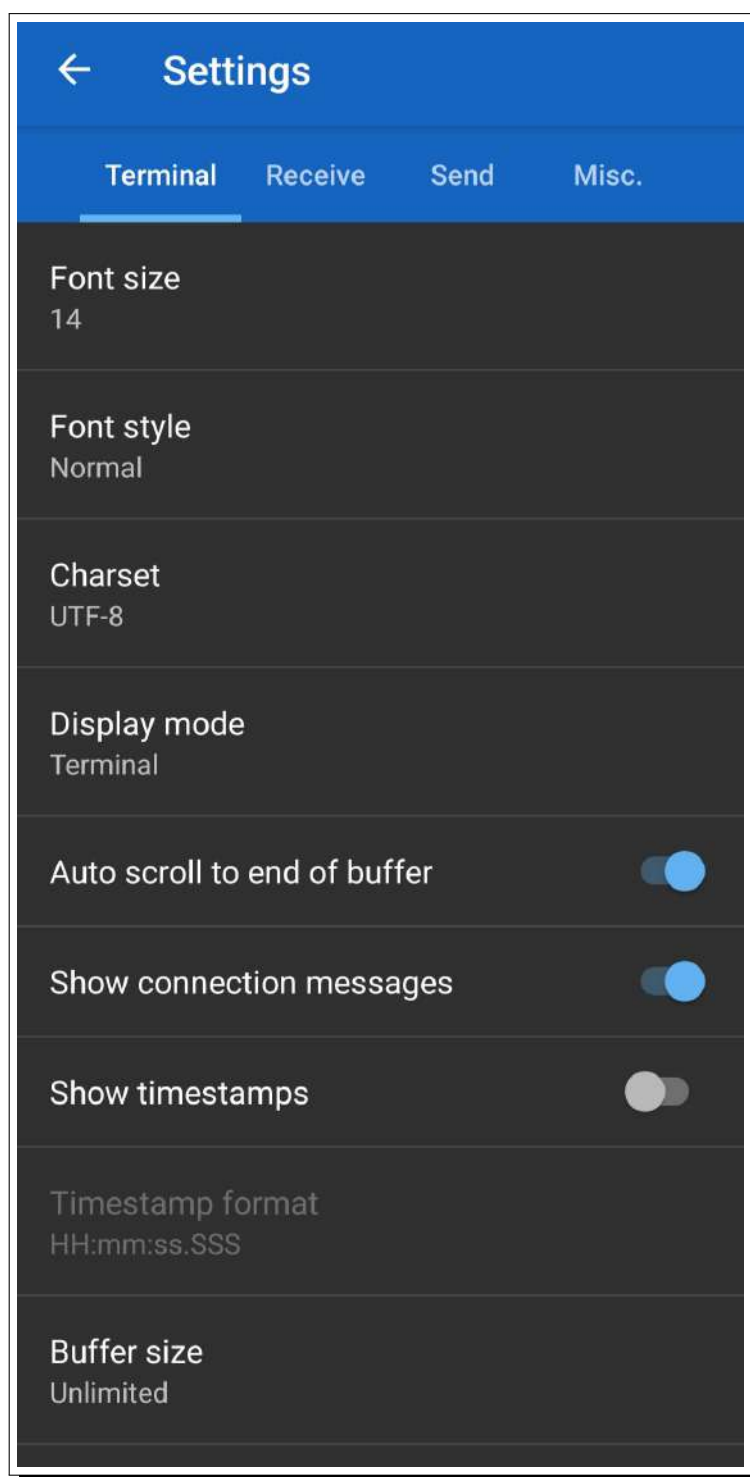


Figura C.2: Configuración de la pestaña Terminal.



C.1.2. Receive

Como puede verse en la Figura C.3 esta pestaña solo cuenta con una opción (Newline) que debe ser CR+LF.

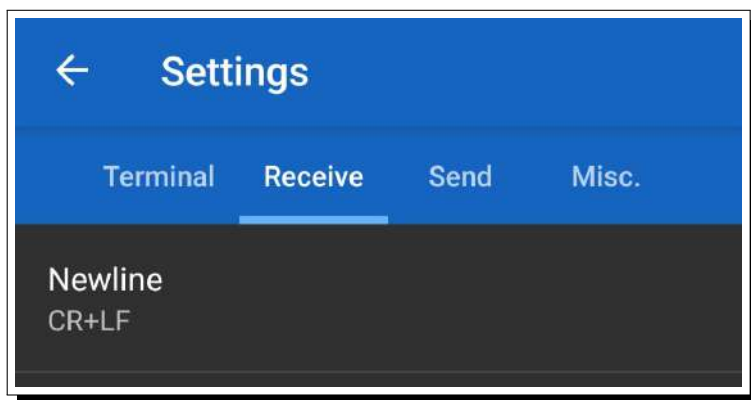


Figura C.3: Configuración de la pestaña Receive.

C.1.3. Send

Esta configuración puede verse en la Figura C.4.

- **Newline:** CR+LF.
- **Edit mode:** Text.
- **Line delay:** 0.
- **Character delay:** 0.
- **Local echo:** Debe estar activado.
- **Clear input on send:** Debe estar desactivado.

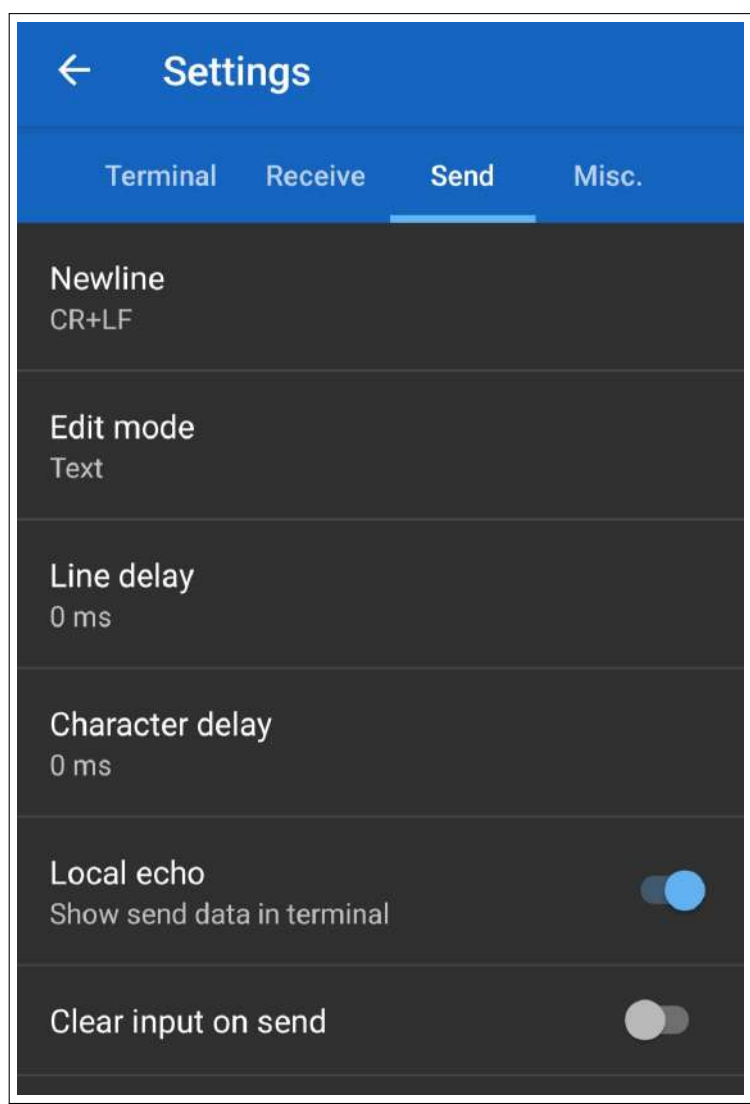


Figura C.4: Configuración de la pestaña Send.

C.1.4. Misc

Esta configuración puede verse en la Figura C.5.

- **Macro buttons:** 2 Rows.
- **Keep screen on when connected:** Debe estar activado.
- **Show notification when connected:** Opcional activarlo o no.
- **Save + log folder:** Indica la ruta donde guardar los archivos si quieres guardarlos en el móvil.



- **Hints:** No es configuración, es un boton para mostrar recomendaciones.

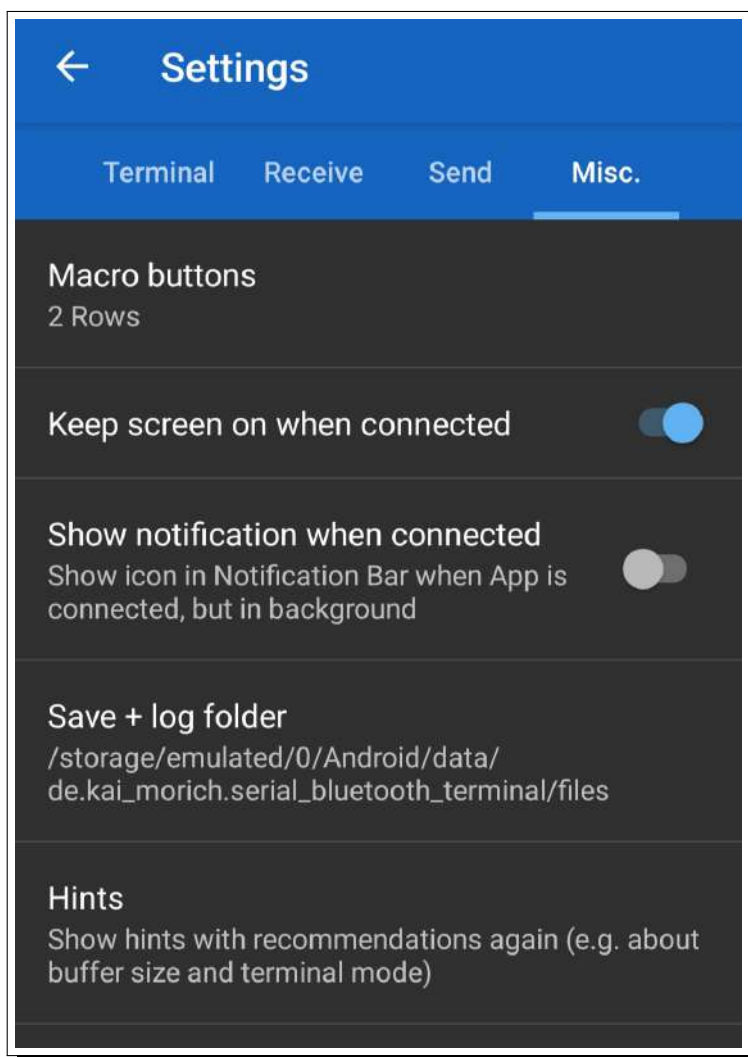


Figura C.5: Configuración de la pestaña Misc.

C.2. Conexión con la nariz electrónica

Para conectar la aplicación con la nariz electrónica, primero hay que volver a pulsar las tres rayas horizontales en la esquina superior izquierda de la pantalla principal de la aplicación. A continuación, se debe seleccionar la opción *Devices*. Aparecerá una lista con los dispositivos Bluetooth disponibles, como se ve en la Figura C.6. Además en la esquina superior derecha está el botón “SCAN” para volver a buscar dispositivos. Hay que seleccionar el dispositivo correspondiente a la nariz electrónica (en este caso *nosecrisE4*) y esperar a que se establezca la conexión. Una vez conectado, se puede volver a la pantalla principal de la aplicación para comenzar a recibir datos.

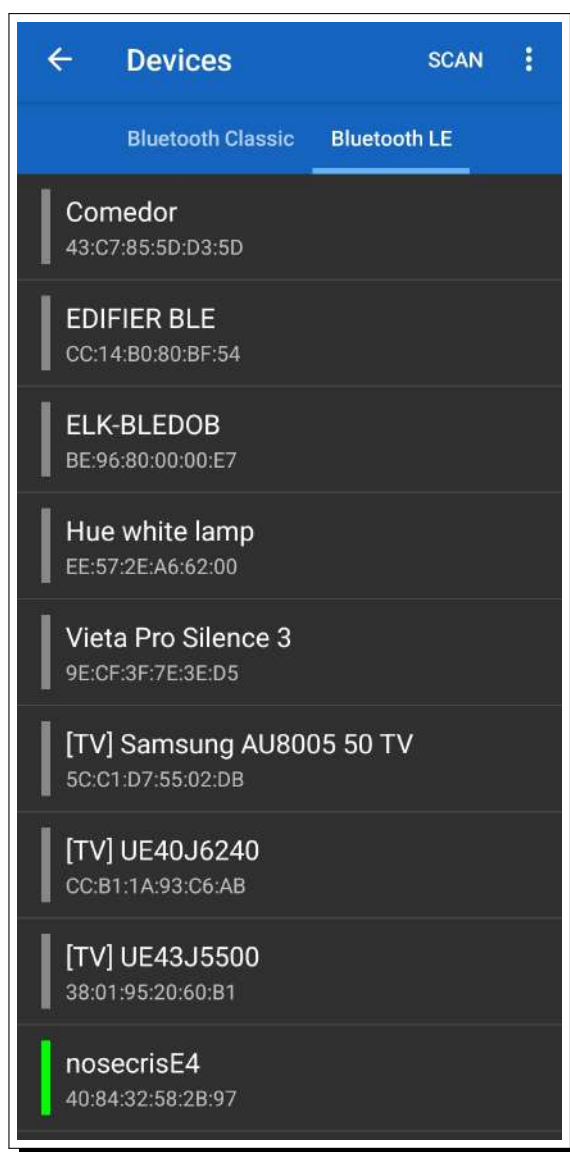


Figura C.6: Lista de dispositivos Bluetooth disponibles.



C.3. Configuración de botones

Para facilitar la toma de muestras, se pueden configurar los botones macros en la parte inferior de la pantalla principal de la aplicación. Para ello simplemente hay que mantener pulsado el botón que se desea configurar hasta que aparezca la pantalla de configuración del botón, como se ve en la Figura C.7.

Una vez en la pantalla de configuración, es posible encontrar varios parámetros:

- **Name:** Nombre del botón (por ejemplo, “Start” o “Stop”).
- **Value:** Comando que se enviará al dispositivo al pulsar el botón (por ejemplo, “Exper” o “STOP”).
- **Edit mode:** Tipo de envío, debe estar en “Text”.
- **Action:** Puede ser “Send”, para que el comando se envíe automáticamente, o “Insert”, para que el comando se inserte en el campo de texto sin enviarse (Este último se usa para enviar comandos que requiere que se escriba algún parámetro antes de enviarlo).

← Edit Macro ⓘ ✓

Name
Cycles

Value
#NSNC,

Edit mode
☒ Text ☐ HEX ☐ Multiline Text

Action
☐ Send ☒ Insert

☐ Repeat

Figura C.7: Pantalla de configuración de un botón macro.



Para un uso normal de la nariz electrónica, se recomienda configurar al menos los botones que se muestran en la Tabla C.2.

Botón	Value	Action	Descripción
AutoStop	#NSAS,2	Send	Activa la parada automática
Cycles	#NSNC,	Insert	Establece el número de ciclos
Settings	SETTINGS	Send	Permite ver los valores de la velocidad de los datos, el número de ciclos, el estado del autostop y los tiempos de adsorción y desorción
Exper	Exper,	Send	Inicia la toma de muestras
Stop	Stop	Send	Detiene la toma de muestras
Ta (s)	#NSTA,	Insert	Establece el tiempo de adsorción en segundos
Td (s)	#NSTD,	Insert	Establece el tiempo de desorción en segundos

Tabla C.2: Configuración recomendada de botones macros para la nariz electrónica.

Anexo D

Entrenamiento de modelos experimentales

En este anexo se detallan los procedimientos y configuraciones utilizadas para entrenar los modelos experimentales presentados en el capítulo 5. También es importante destacar que estos entrenamientos se llevaron a cabo usando los primeros 76 archivos de muestras obtenidos.

D.1. Entrenamiento

D.1.1. Keras

El entrenamiento del modelo en Keras se realizó mediante un notebook de Jupyter ejecutado en Google Colab. A continuación se describe el proceso detallando cada una de sus partes.

Importación de librerías: El notebook comienza importando las dependencias necesarias para el procesamiento de datos, entrenamiento y visualización:

```
1 import os
```



```
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.impute import SimpleImputer
8 from sklearn.metrics import confusion_matrix
9 import seaborn as sns
10 from tensorflow.keras.models import Sequential
11 from tensorflow.keras.layers import Dense, Dropout
12 from tensorflow.keras.callbacks import EarlyStopping
13 from google.colab import drive
14
15 print("Librerías cargadas correctamente.")
```

Código D.1: Importación de librerías para el entrenamiento en Keras.

Como se muestra en el Código D.1, se utilizan `numpy` y `pandas` para la manipulación de datos, `matplotlib` y `seaborn` para visualización, `sklearn` para el preprocesamiento (imputación, escalado y división del dataset), y `tensorflow.keras` para la construcción y entrenamiento de la red neuronal.

Carga y etiquetado de datos: Los datos se extraen de archivos de texto, convirtiendo las lecturas de los sensores en vectores numéricos aplanados:

```
1 drive.mount('/content/drive')
2
3 # Configuración de rutas
4 bueno_dir = '/content/drive/MyDrive/Colab Notebooks/BUENO'
5 malo_dir = '/content/drive/MyDrive/Colab Notebooks/MALO'
6
7 X_features = []
8 y_labels = []
```



```
9
10 def cargar_sustancias(ruta_carpeta, etiqueta):
11     conteo = 0
12     for archivo in os.listdir(ruta_carpeta):
13         if archivo.endswith(".txt"):
14             ruta_completa = os.path.join(ruta_carpeta, archivo)
15             try:
16                 # Carga con pandas, mas robusto para decimales ','
17                 datos_df = pd.read_csv(ruta_completa, sep=',',
18                                         decimal=',', skiprows=1, header=None)
19                 # Convertir a numerico, forzando errores a NaN
20                 datos_df = datos_df.apply(pd.to_numeric, errors='coerce')
21                 # Convertir a numpy y aplanar
22                 datos = datos_df.values.flatten()
23
24                 if np.all(np.isnan(datos)):
25                     print(f"Advertencia: '{archivo}' contiene solo NaNs.")
26                 else:
27                     X_features.append(datos)
28                     y_labels.append(etiqueta)
29                     conteo += 1
30             except Exception as e:
31                 print(f"Error en {archivo}: {e}")
32     return conteo
```

Código D.2: Función para cargar y etiquetar los datos de entrenamiento.

En el Código D.2, la función `cargar_sustancias` (líneas 10–32) procesa cada archivo `.txt` de la carpeta especificada. La línea 17 lee el archivo con `pandas`, interpretando la coma como separador decimal y omitiendo la primera fila de metadatos. La línea 20 convierte todos los valores a numérico, forzando los errores a `NaN`. La línea 22 aplanar la matriz a un vector unidimensional. Si el archivo contiene solo valores `NaN`, se ignora



(líneas 24–25); en caso contrario, se añade a las listas de características y etiquetas.

```
1  # Cargar datos
2  n_buenos = cargar_sustancias(bueno_dir, 1) # Clase Positiva
3  n_malos = cargar_sustancias(malo_dir, 0) # Clase Negativa
4
5  # Homogeneización: Padding con NaN hasta longitud máxima
6  if X_features:
7      max_len = max(len(x) for x in X_features)
8      padded_X_features = []
9      for x in X_features:
10         if len(x) < max_len:
11             padding = np.full(max_len - len(x), np.nan)
12             padded_X_features.append(np.concatenate((x, padding)))
13         else:
14             padded_X_features.append(x)
15     X = np.array(padded_X_features)
16 else:
17     X = np.array([])
18     print("Advertencia: No se cargaron datos.")
19
20 y = np.array(y_labels)
21 print(f"Forma de la matriz X: {X.shape}")
```

Código D.3: Homogeneización (Padding) de los datos.

Debido a que las capturas tienen duraciones distintas, el Código D.3 muestra el proceso de homogeneización (líneas 6–15). Se encuentra la longitud máxima entre todos los arrays (línea 7), que resulta ser 242.676 características. Los arrays más cortos se rellenan con valores NaN hasta alcanzar esta longitud máxima (líneas 10–12), garantizando una matriz de características constante.

División del dataset, imputación y escalado: Los datos se dividen y preparan



para el entrenamiento:

```
1  # Separar un 20% para el Test final
2  X_train_val, X_test, y_train_val, y_test = train_test_split(
3      X, y, test_size=0.20, random_state=42, stratify=y
4  )
5
6  # Imputar los valores NaN con estrategia constante (valor 0)
7  imputer = SimpleImputer(strategy='constant', fill_value=0)
8  X_train_val_imputed = imputer.fit_transform(X_train_val)
9  X_test_imputed = imputer.transform(X_test)
10
11 # Escalar los datos (Fundamental para una ANN)
12 scaler = StandardScaler()
13 X_train_val_scaled = scaler.fit_transform(X_train_val_imputed)
14 X_test_scaled = scaler.transform(X_test_imputed)
15
16 print(f"Entrenamiento + Validacion: {X_train_val_scaled.shape[0]} muestras")
17 print(f"Prueba final (Test): {X_test_scaled.shape[0]} muestras")
```

Código D.4: División del dataset, imputación y escalado.

En el Código D.4, la primera división (líneas 2–4) reserva el 20 % de los datos para el test final, utilizando estratificación para mantener la proporción de clases. El `SimpleImputer` (líneas 7–9) reemplaza los valores NaN generados por el padding con ceros, ya que las redes neuronales no pueden procesar valores faltantes. El `StandardScaler` (líneas 12–14) normaliza los datos para que tengan media 0 y varianza 1, un paso fundamental para que las funciones de activación de Keras operen correctamente. Del 80 % restante, se utilizará un 20 % para validación durante el entrenamiento (configurado en el método `fit`), resultando en una distribución final aproximada de 64 % para entrenamiento, 16 % para validación y 20 % para prueba.

Definición de la arquitectura de la red neuronal: Se ha implementado un modelo



de tipo Secuencial que permite un flujo de datos lineal a través de capas densas:

```
1  # Definir la estructura de la red
2  model = Sequential([
3      Dense(32, activation='relu', input_shape=(X_train_val_scaled.shape[1],)),
4      Dropout(0.2), # Ayuda a evitar el sobreajuste
5      Dense(16, activation='relu'),
6      Dense(1, activation='sigmoid') # Salida binaria (0 a 1)
7  ])
8
9  model.compile(optimizer='adam', loss='binary_crossentropy',
10               metrics=['accuracy'])
11
12 # Configurar parada temprana
13 early_stop = EarlyStopping(monitor='val_loss', patience=10,
14                             restore_best_weights=True)
```

Código D.5: Definición de la arquitectura de la red neuronal en Keras.

La arquitectura consta de:

- **Primera capa oculta** (línea 3): 32 neuronas con activación ReLU y entrada de tamaño 242.676 (determinado dinámicamente). Su función es realizar la primera abstracción de los datos del sensor.
- **Capa de Dropout** (línea 4): Con una tasa de 0.2 (20 %) para prevenir el sobreajuste durante el entrenamiento.
- **Segunda capa oculta** (línea 5): 16 neuronas con activación ReLU, que profundiza en la jerarquía de patrones detectados.
- **Capa de salida** (línea 6): Una única neurona con activación Sigmoide, que proyecta el resultado como una probabilidad entre 0 y 1.



El modelo se compila (línea 9) con el optimizador Adam y la función de pérdida *Binary Cross Entropy*, adecuada para clasificación binaria. El `EarlyStopping` (líneas 12–13) detiene el entrenamiento si la pérdida de validación no mejora durante 10 épocas consecutivas, restaurando los mejores pesos encontrados.

Entrenamiento del modelo: El entrenamiento se realiza por lotes con validación interna:

```
1 print("Iniciando entrenamiento...")
2
3 history = model.fit(
4     X_train_val_scaled, y_train_val,
5     epochs=50,
6     batch_size=5,
7     validation_split=0.2, # Divide internamente para validacion
8     callbacks=[early_stop],
9     verbose=1
10 )
11
12 print("Entrenamiento finalizado")
```

Código D.6: Entrenamiento del modelo con validación.

Como se muestra en el Código D.6, el entrenamiento se ejecuta durante un máximo de 50 épocas con un tamaño de lote de 5 muestras (líneas 5–6). Este tamaño pequeño permite una actualización más frecuente de los pesos, lo que puede ayudar a la convergencia del modelo con conjuntos de datos limitados. El parámetro `validation_split=0.2` (línea 7) divide internamente el conjunto de entrenamiento, reservando el 20 % para validación.

Evaluación y visualización de resultados: Una vez completado el entrenamiento, se evalúa el modelo y se visualizan las curvas de aprendizaje:

```
1 # Evaluar con el conjunto de Test
2 loss, accuracy = model.evaluate(X_test_scaled, y_test, verbose=0)
```




```
3
4 print(f"--- RESULTADOS FINALES ---")
5 print(f"Precision en Test (Accuracy): {accuracy*100:.2f}%")
6
7 # Grafico de la evolucion del entrenamiento
8 plt.figure(figsize=(12, 4))
9 plt.subplot(1, 2, 1)
10 plt.plot(history.history['accuracy'], label='Entrenamiento')
11 plt.plot(history.history['val_accuracy'], label='Validacion')
12 plt.title('Precision del Modelo')
13 plt.legend()
14
15 plt.subplot(1, 2, 2)
16 plt.plot(history.history['loss'], label='Entrenamiento')
17 plt.plot(history.history['val_loss'], label='Validacion')
18 plt.title('Perdida (Loss) del Modelo')
19 plt.legend()
20 plt.show()
```

Código D.7: Evaluación del modelo y visualización de curvas de aprendizaje.

En el Código D.7, la evaluación en el conjunto de test (línea 2) proporciona una estimación imparcial del rendimiento del modelo. Las gráficas de precisión y pérdida (líneas 8–20) permiten visualizar si el modelo sufre de sobreajuste.

```
1 # Hacer predicciones en el conjunto de prueba
2 y_pred_proba = model.predict(X_test_scaled)
3 y_pred = (y_pred_proba > 0.5).astype(int)
4
5 # Generar la matriz de confusion
6 cm = confusion_matrix(y_test, y_pred)
7
8 plt.figure(figsize=(6, 5))
```



```
9 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
10             xticklabels=['Malo (0)', 'Bueno (1)'],
11             yticklabels=['Malo (0)', 'Bueno (1)'])
12 plt.xlabel('Prediccion')
13 plt.ylabel('Valor Real')
14 plt.title('Matriz de Confusion')
15 plt.show()
```

Código D.8: Generación de la matriz de confusión.

En el Código D.8, la matriz de confusión (líneas 2–6) muestra el número de verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos, permitiendo evaluar el comportamiento del modelo en cada clase.

Exportación del modelo: El paso final consiste en guardar el modelo en formato nativo de Keras y convertirlo a TensorFlow Lite:

```
1 # Definir la ruta para guardar el modelo
2 model_save_path = '/content/drive/MyDrive/Colab Notebooks/modelo_keras.keras'
3
4 # Guardar el modelo
5 model.save(model_save_path)
6
7 print(f"Modelo guardado exitosamente en: {model_save_path}")
```

Código D.9: Guardado del modelo en formato Keras.

```
1 import tensorflow as tf
2
3 # Cargar el modelo Keras guardado
4 model = tf.keras.models.load_model(model_save_path)
5
6 # Inicializar el convertidor de TFLite
7 converter = tf.lite.TFLiteConverter.from_keras_model(model)
```



```
8
9  # Convertir el modelo
10 tflite_model = converter.convert()
11
12 # Guardar el modelo TFLite
13 tflite_model_save_path =
14     '/content/drive/MyDrive/Colab Notebooks/modelo.tflite'
15 with open(tflite_model_save_path, 'wb') as f:
16     f.write(tflite_model)
17
18 print(f"Modelo TFLite guardado en: {tflite_model_save_path}")
```

Código D.10: Conversión del modelo a TensorFlow Lite.

El Código D.9 muestra el guardado del modelo en formato nativo de Keras (`.keras`), almacenando tanto la arquitectura como los pesos optimizados. Posteriormente, como se observa en el Código D.10, se convierte a TensorFlow Lite (`.tflite`) mediante el `TFLiteConverter` (líneas 7–10), generando un archivo binario optimizado con menor huella de memoria y latencia reducida, preparado para realizar inferencias en dispositivos móviles.

Para el entrenamiento del “modelo plus” con el dataset aumentado, se siguió un procedimiento idéntico al descrito anteriormente, asegurando la consistencia en la comparación de resultados entre ambos modelos.

D.1.2. PyTorch

El entrenamiento del modelo en PyTorch se realizó mediante un notebook de Jupyter ejecutado en Google Colab. A continuación se describe el proceso detallando cada una de sus partes.

Importación de librerías: El notebook comienza importando las dependencias



necesarias para el procesamiento de datos, entrenamiento y visualización:

```
1 import os
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 from torch.utils.data import DataLoader, TensorDataset
6 import numpy as np
7 import pandas as pd
8 from sklearn.model_selection import train_test_split
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.impute import SimpleImputer
11 from google.colab import drive
12 import matplotlib.pyplot as plt
13 import seaborn as sns
14 from sklearn.metrics import confusion_matrix
15
16 print("Librerías cargadas correctamente.")
```

Código D.11: Importación de librerías para el entrenamiento en PyTorch.

Como se muestra en el Código D.11, se utilizan `torch` y sus módulos (`nn`, `optim`, `DataLoader`) para la construcción y entrenamiento de la red neuronal, `numpy` y `pandas` para la manipulación de datos, `sklearn` para el preprocesamiento, y `matplotlib` y `seaborn` para visualización.

Carga y etiquetado de datos: Los datos se extraen de archivos de texto de forma similar a Keras, convirtiendo las lecturas de los sensores en vectores numéricos aplanados:

```
1 drive.mount('/content/drive')
2
3 # Configuración de rutas
4 bueno_dir = '/content/drive/MyDrive/Colab Notebooks/BUENO'
5 malo_dir = '/content/drive/MyDrive/Colab Notebooks/MALO'
```



```
6
7 X_features = []
8 y_labels = []
9
10 def cargar_sustancias(ruta_carpeta, etiqueta):
11     conteo = 0
12     for archivo in os.listdir(ruta_carpeta):
13         if archivo.endswith(".txt"):
14             ruta_completa = os.path.join(ruta_carpeta, archivo)
15             try:
16                 datos_df = pd.read_csv(ruta_completa, sep=' ',
17                                         decimal=',', skiprows=1, header=None)
18                 datos_df = datos_df.apply(pd.to_numeric, errors='coerce')
19                 datos = datos_df.values.flatten()
20
21                 if np.all(np.isnan(datos)):
22                     print(f"Advertencia: '{archivo}' contiene solo NaNs.")
23                 else:
24                     X_features.append(datos)
25                     y_labels.append(etiqueta)
26                     conteo += 1
27             except Exception as e:
28                 print(f"Error en {archivo}: {e}")
29     return conteo
30
31 # Cargar y homogeneizar datos (Padding con NaN)
32 n_buenos = cargar_sustancias(bueno_dir, 1)
33 n_malos = cargar_sustancias(malo_dir, 0)
34
35 if X_features:
36     max_len = max(len(x) for x in X_features)
```



```
37 padded_X_features = []
38 for x in X_features:
39     if len(x) < max_len:
40         padding = np.full(max_len - len(x), np.nan)
41         padded_X_features.append(np.concatenate((x, padding)))
42     else:
43         padded_X_features.append(x)
44 X = np.array(padded_X_features)
45
46 y = np.array(y_labels)
47 print(f"Forma de la matriz X: {X.shape}")
```

Código D.12: Función para cargar y etiquetar los datos de entrenamiento.

Como se muestra en el Código D.12, el proceso de carga es idéntico al de Keras: se leen los archivos con `pandas` (líneas 16–17), se convierten a numérico (línea 18), se aplanan (línea 19) y se aplica padding con NaN hasta la longitud máxima de 242.676 características (líneas 35–44).

División del dataset, imputación y conversión a tensores: A diferencia de Keras, PyTorch requiere una división explícita en tres conjuntos y la conversión de los datos a tensores:

```
1  # Dividir en entrenamiento+validacion y prueba (80/20)
2  X_train_val, X_test, y_train_val, y_test = train_test_split(
3      X, y, test_size=0.20, random_state=42, stratify=y
4  )
5
6  # Dividir entrenamiento+validacion en entrenamiento y validacion
7  X_train, X_val, y_train, y_val = train_test_split(
8      X_train_val, y_train_val, test_size=0.25, random_state=42,
9      stratify=y_train_val
10 ) # 0.25 de 0.8 = 0.2 del total
```



```
11
12 # Imputacion y Escalado
13 imputer = SimpleImputer(strategy='constant', fill_value=0)
14 scaler = StandardScaler()
15
16 X_train = scaler.fit_transform(imputer.fit_transform(X_train))
17 X_val = scaler.transform(imputer.transform(X_val))
18 X_test = scaler.transform(imputer.transform(X_test))
19
20 # Convertir a Tensores de PyTorch
21 X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
22 y_train_tensor = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
23 X_val_tensor = torch.tensor(X_val, dtype=torch.float32)
24 y_val_tensor = torch.tensor(y_val, dtype=torch.float32).unsqueeze(1)
25 X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
26 y_test_tensor = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)
27
28 # Crear DataLoaders para entrenamiento y validacion
29 batch_size = 5
30 train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
31 train_loader = DataLoader(train_dataset, batch_size, shuffle=True)
32
33 val_dataset = TensorDataset(X_val_tensor, y_val_tensor)
34 val_loader = DataLoader(val_dataset, batch_size, shuffle=False)
35
36 print(f"Entrenamiento: {X_train.shape[0]} muestras")
37 print(f"Validacion: {X_val.shape[0]} muestras")
38 print(f"Prueba final (Test): {X_test.shape[0]} muestras")
```

Código D.13: División del dataset, imputación y conversión a tensores de PyTorch.

En el Código D.13, la división se realiza en dos pasos (líneas 2–10): primero se separa



el 20 % para test, y luego del 80 % restante se extrae el 25 % para validación ($0.25 \times 0.8 = 0.2$ del total), resultando en una distribución 60/20/20. Las líneas 21–26 convierten los arrays de NumPy a tensores de PyTorch, añadiendo una dimensión extra a las etiquetas con `unsqueeze(1)`. Los `DataLoader` (líneas 29–34) permiten iterar sobre los datos en lotes durante el entrenamiento.

Definición de la arquitectura de la red neuronal: Se define una clase que hereda de `nn.Module`, implementando una arquitectura MLP:

```
1 class OlfativaANN(nn.Module):
2     def __init__(self, input_dim):
3         super(OlfativaANN, self).__init__()
4         self.network = nn.Sequential(
5             nn.Linear(input_dim, 32),
6             nn.ReLU(),
7             nn.Dropout(0.2),
8             nn.Linear(32, 16),
9             nn.ReLU(),
10            nn.Linear(16, 1),
11            nn.Sigmoid()
12        )
13
14    def forward(self, x):
15        return self.network(x)
16
17    # Inicializar modelo, funcion de perdida y optimizador
18    input_features = X_train.shape[1]
19    model = OlfativaANN(input_features)
20    criterion = nn.BCELoss() # Entropia Cruzada Binaria
21    optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Código D.14: Definición de la clase OlfativaANN en PyTorch.



En el Código D.14, la clase `OlfativaANN` (líneas 1–15) define la arquitectura mediante `nn.Sequential`: capa lineal de 32 neuronas (línea 5), activación ReLU (línea 6), Dropout del 20 % (línea 7), capa de 16 neuronas (línea 8), ReLU (línea 9), capa de salida (línea 10) y Sigmoide (línea 11). El modelo se inicializa con la dimensión de entrada (línea 19), se configura la función de pérdida BCE (línea 20) y el optimizador Adam con tasa de aprendizaje 0.001 (línea 21).

Bucle de entrenamiento con Early Stopping: A diferencia de Keras, en PyTorch el bucle de entrenamiento se implementa manualmente:

```
1 epochs = 15
2 train_losses, val_losses = [], []
3 train_accuracies, val_accuracies = [], []
4
5 # Parametros para Early Stopping
6 early_stopping_patience = 10
7 best_val_loss = float('inf')
8 patience_counter = 0
9
10 for epoch in range(epochs):
11     # Fase de entrenamiento
12     model.train()
13     running_loss = 0.0
14     correct_train, total_train = 0, 0
15
16     for inputs, labels in train_loader:
17         optimizer.zero_grad()
18         outputs = model(inputs)
19         loss = criterion(outputs, labels)
20         loss.backward()
21         optimizer.step()
22         running_loss += loss.item()
```



```
23
24     predicted = (outputs > 0.5).float()
25     total_train += labels.size(0)
26     correct_train += (predicted == labels).sum().item()
27
28 train_accuracy = 100 * correct_train / total_train
29 train_losses.append(running_loss / len(train_loader))
30 train_accuracies.append(train_accuracy)
```

Código D.15: Bucle de entrenamiento con Early Stopping en PyTorch.

En el Código D.15, el bucle de entrenamiento (líneas 10–30) itera sobre cada época. En la fase de entrenamiento (líneas 12–30), se activa el modo de entrenamiento (línea 12), se itera sobre los lotes del `DataLoader` (línea 16), se calcula la pérdida (línea 19), se propaga hacia atrás (línea 20) y se actualizan los pesos (línea 21). Las líneas 24–26 calculan la precisión del entrenamiento.

```
1     # Fase de validacion
2     model.eval()
3     val_loss = 0.0
4     correct_val, total_val = 0, 0
5
6     with torch.no_grad():
7         for inputs, labels in val_loader:
8             outputs = model(inputs)
9             loss = criterion(outputs, labels)
10            val_loss += loss.item()
11
12            predicted = (outputs > 0.5).float()
13            total_val += labels.size(0)
14            correct_val += (predicted == labels).sum().item()
15
16 val_accuracy = 100 * correct_val / total_val
```



```
17     val_losses.append(val_loss / len(val_loader))
18     val_accuracies.append(val_accuracy)
19
20     # Logica de Early Stopping
21     current_val_loss = val_losses[-1]
22     if current_val_loss < best_val_loss:
23         best_val_loss = current_val_loss
24         patience_counter = 0
25     else:
26         patience_counter += 1
27         if patience_counter >= early_stopping_patience:
28             print(f"Early stopping en la epoca {epoch+1}.")
29             break
```

Código D.16: Fase de validación y lógica de Early Stopping.

En el Código D.16, la fase de validación (líneas 2–18) se ejecuta sin calcular gradientes (`torch.no_grad()`) para ahorrar memoria. La lógica de Early Stopping (líneas 21–29) verifica si la pérdida de validación mejora; si no mejora durante 10 épocas consecutivas, el entrenamiento se detiene.

Evaluación y visualización de resultados: Una vez completado el entrenamiento, se evalúa el modelo en el conjunto de test. El Código D.17 muestra la evaluación final, el Código D.18 presenta la visualización de las curvas de aprendizaje, y el Código D.19 genera la matriz de confusión:

```
1  # Evaluacion Final
2  model.eval()
3  with torch.no_grad():
4      predictions = model(X_test_tensor)
5      predicted_classes = (predictions > 0.5).float()
6      accuracy = (predicted_classes == y_test_tensor).float().mean()
7      print(f"\n--- RESULTADOS FINALES ---")
```



```
8 print(f"Precision en Test: {accuracy.item()*100:.2f}%")
```

Código D.17: Evaluación final del modelo en PyTorch.

```
1 epochs_range = range(1, len(train_accuracies) + 1)
2
3 plt.figure(figsize=(12, 5))
4
5 plt.subplot(1, 2, 1)
6 plt.plot(epochs_range, train_accuracies, label='Entrenamiento')
7 plt.plot(epochs_range, val_accuracies, label='Validacion')
8 plt.title('Precision del Modelo')
9 plt.xlabel('Epoca')
10 plt.ylabel('Precision (%)')
11 plt.legend()
12
13 plt.subplot(1, 2, 2)
14 plt.plot(epochs_range, train_losses, label='Entrenamiento')
15 plt.plot(epochs_range, val_losses, label='Validacion')
16 plt.title('Perdida (Loss) del Modelo')
17 plt.xlabel('Epoca')
18 plt.ylabel('Loss')
19 plt.legend()
20
21 plt.tight_layout()
22 plt.show()
```

Código D.18: Visualización de curvas de aprendizaje en PyTorch.

```
1 model.eval()
2 with torch.no_grad():
3     predictions = model(X_test_tensor)
4     predicted_classes = (predictions > 0.5).float().cpu().numpy()
```



```
5     true_classes = y_test_tensor.cpu().numpy()
6
7     cm = confusion_matrix(true_classes, predicted_classes)
8
9     plt.figure(figsize=(6, 5))
10    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
11                xticklabels=['Malo', 'Bueno'], yticklabels=['Malo', 'Bueno'])
12    plt.xlabel('Prediccion')
13    plt.ylabel('Valor Real')
14    plt.title('Matriz de Confusion')
15    plt.show()
```

Código D.19: Generación de la matriz de confusión en PyTorch.

Exportación del modelo: El modelo se guarda en formato nativo de PyTorch y posteriormente se convierte a TensorFlow Lite mediante ONNX como formato intermedio:

```
1 model_save_path = '/content/drive/MyDrive/Colab Notebooks/modelo_pytorch.pth'
2
3 # Guardar el diccionario de estado del modelo
4 torch.save(model.state_dict(), model_save_path)
5
6 print(f"Modelo guardado exitosamente en: {model_save_path}")
```

Código D.20: Guardado del modelo en formato PyTorch.

```
1 import onnx
2 import onnx_tf
3 import tensorflow as tf
4
5 # Crear input dummy con la forma esperada
6 dummy_input = torch.randn(1, input_features, requires_grad=True)
7 onnx_path = '/content/drive/MyDrive/Colab Notebooks/modelo_pytorch.onnx'
8
```



```
9  # Exportar a ONNX
10 torch.onnx.export(model,
11                    dummy_input,
12                    onnx_path,
13                    export_params=True,
14                    opset_version=11,
15                    do_constant_folding=True,
16                    input_names=['input'],
17                    output_names=['output'],
18                    dynamic_axes={'input': {0: 'batch_size'},
19                                'output': {0: 'batch_size'}})
20
21 print(f"Modelo exportado a ONNX en: {onnx_path}")
```

Código D.21: Exportación del modelo PyTorch a ONNX.

```
1  # Convertir ONNX a TensorFlow SavedModel
2  onnx_model = onnx.load(onnx_path)
3  tf_saved_model_path = '/content/drive/MyDrive/Colab Notebooks/modelo_pytorch'
4
5  tf_rep = onnx_tf.backend.prepare(onnx_model, device='CPU')
6  tf_rep.export_tf(tf_saved_model_path)
7
8  # Convertir TensorFlow SavedModel a TFLite
9  converter = tf.lite.TFLiteConverter.from_saved_model(tf_saved_model_path)
10 tflite_model = converter.convert()
11
12 tflite_model_path =
13     '/content/drive/MyDrive/Colab Notebooks/modelo_pytorch.tflite'
14 with open(tflite_model_path, 'wb') as f:
15     f.write(tflite_model)
16
```



```
17 print(f"Modelo TFLite guardado en: {tflite_model_path}")
```

Código D.22: Conversión de ONNX a TensorFlow Lite.

El Código D.20 muestra el guardado del modelo en formato PyTorch. El Código D.21 presenta la exportación a ONNX (líneas 10–19), y el Código D.22 detalla el proceso de conversión a TensorFlow Lite, que requiere dos pasos intermedios: primero se convierte ONNX a TensorFlow SavedModel (líneas 2–6), y finalmente se convierte a TensorFlow Lite (líneas 9–14). Este proceso es más complejo que en Keras debido a las diferencias entre los frameworks.

Para el entrenamiento del “modelo plus” con el dataset aumentado, se siguió un procedimiento casi idéntico al descrito anteriormente, asegurando la consistencia en la comparación de resultados entre ambos modelos. Los únicos cambios radicarón en el número de épocas, que se incrementó a 50 para permitir un aprendizaje más profundo, y la adaptación del tamaño del lote (batch size) a 10 muestras por iteración, optimizando así el proceso de entrenamiento con el nuevo conjunto de datos.

D.2. Resultados

Tras el entrenamiento, se evalúa el modelo para comprobar si los resultados sufren de sobreajuste pese a haber aplicado “Early Stopping”. El rendimiento se visualiza mediante curvas de aprendizaje Accuracy (Precisión) y Loss (Pérdida) y matrices de confusión.

En la Figura D.1 y la Figura D.2 se muestran las curvas de aprendizaje y la matriz de confusión del modelo entrenado en Keras, mientras que en la Figura D.3 y la Figura D.4 se presentan los resultados correspondientes al “modelo plus”.

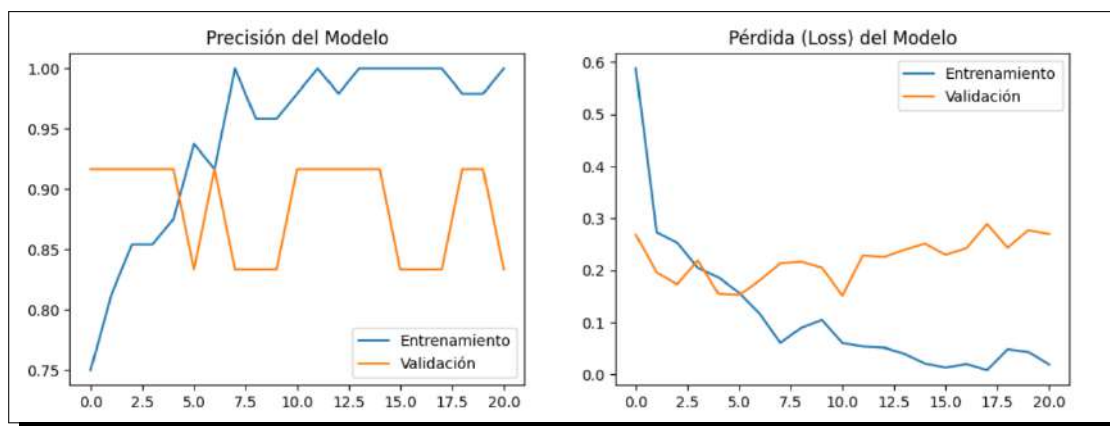


Figura D.1: Curvas de precisión (Accuracy) y pérdida (Loss) durante el entrenamiento y validación del modelo en Keras.

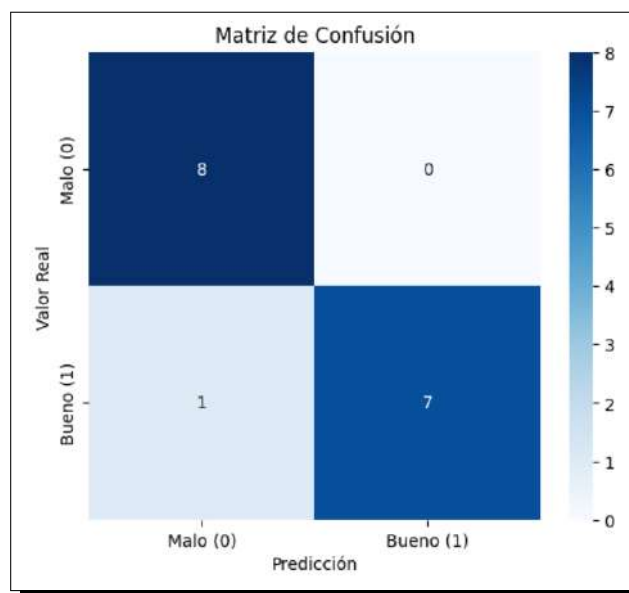


Figura D.2: Matriz de confusión del modelo entrenado en Keras.

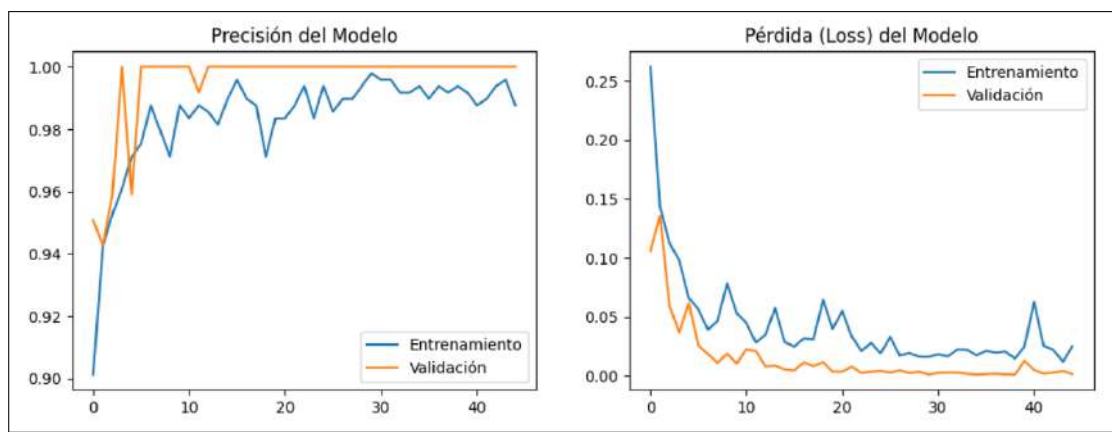


Figura D.3: Curvas de precisión (Accuracy) y pérdida (Loss) durante el entrenamiento y validación del modelo en Keras con datos ampliados.

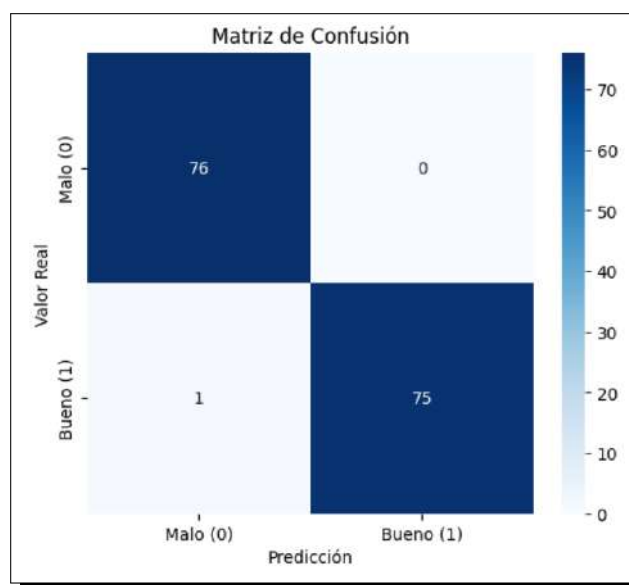


Figura D.4: Matriz de confusión del modelo entrenado en Keras con datos ampliados.

Observando las matrices de confusión, se puede apreciar que ambos modelos presentan una gran capacidad para clasificar correctamente las muestras, con un número muy reducido de falsos negativos y ningún falso positivo. Sin embargo, si se analizan las curvas de aprendizaje, pese a que ambos modelos presentan una precisión bastante elevada (93.75 % en el modelo normal y 99.34 % en el modelo aumentado), el modelo normal muestra signos de sobreajuste, con una brecha significativa entre las curvas de entrenamiento y validación. En contraste, el modelo aumentado exhibe curvas más

alineadas, indicando una mejor generalización. En el modelo normal se ve cómo la precisión de validación es bastante inferior a la de entrenamiento, además de que la pérdida de validación acaba siendo mayor que la de entrenamiento, lo que indica que el modelo ha memorizado patrones específicos del conjunto de entrenamiento que no se generalizan bien a datos nuevos. En el modelo aumentado, la curva de precisión de validación es bastante estable y superior a la del entrenamiento, además de que la pérdida de validación es menor que la de entrenamiento, lo que indica que el modelo ha aprendido patrones más generales y es capaz de adaptarse mejor a datos no vistos previamente.

Esto sugiere que el uso de datos ampliados ha contribuido a mejorar la robustez del modelo, permitiéndole aprender patrones más generales y reducir la dependencia de características específicas del conjunto de entrenamiento.

De manera similar, en la Figura D.5 y la Figura D.6 se muestran las curvas de aprendizaje y la matriz de confusión del modelo entrenado en PyTorch, mientras que en la Figura D.7 y la Figura D.8 se presentan los resultados correspondientes al “modelo plus”.

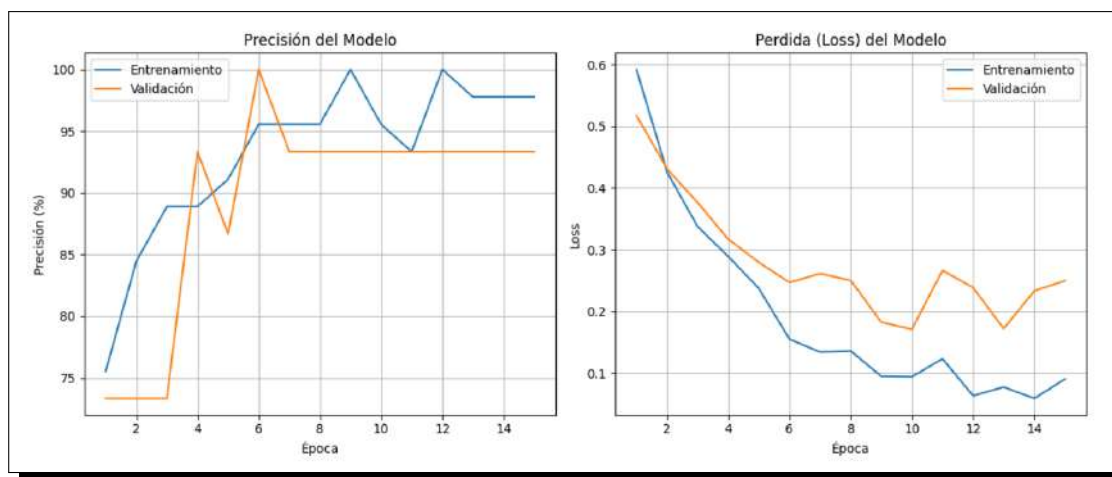


Figura D.5: Curvas de precisión (Accuracy) y pérdida (Loss) durante el entrenamiento y validación del modelo en PyTorch.

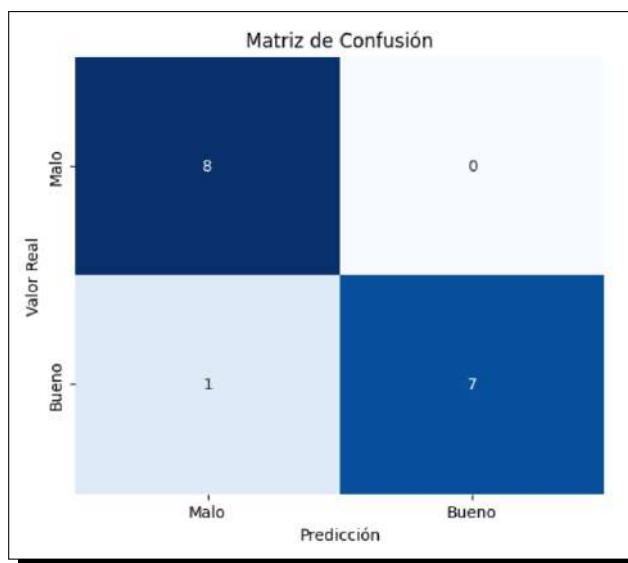


Figura D.6: Matriz de confusión del modelo entrenado en PyTorch.

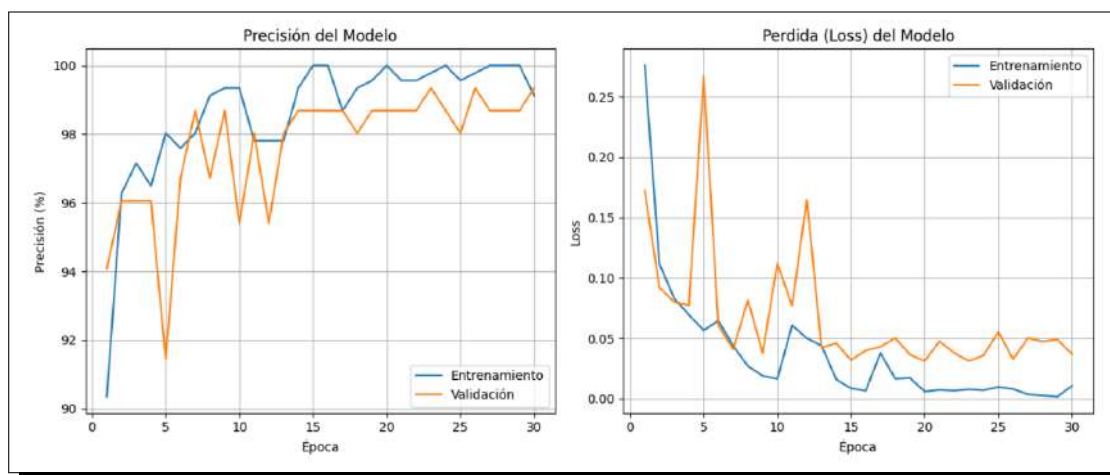


Figura D.7: Curvas de precisión (Accuracy) y pérdida (Loss) durante el entrenamiento y validación del modelo en PyTorch con datos ampliados.

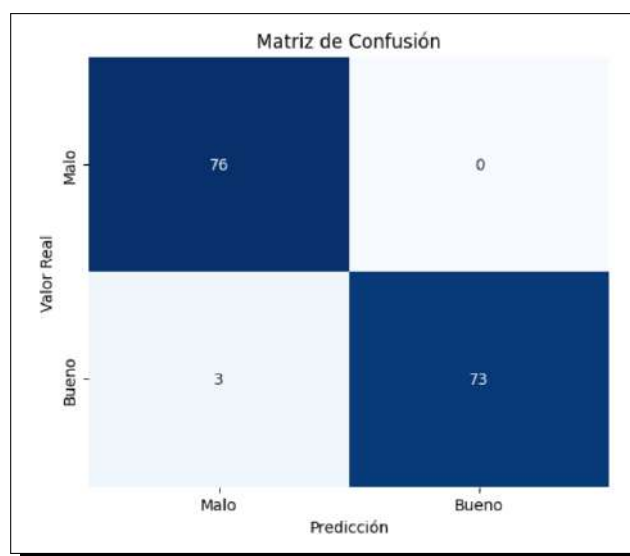


Figura D.8: Matriz de confusión del modelo entrenado en PyTorch con datos ampliados.

En este caso es posible observar que, a diferencia de Keras, los resultados del entrenamiento del modelo normal son algo mejores, pese a que la precisión sea la misma (93.75 %) y sigue habiendo una ligera diferencia entre las curvas de entrenamiento y validación, siendo esta última inferior a la otra, dicha diferencia es algo menor que con Keras. Lo mismo pasa en el caso de la gráfica de la pérdida. El resultado es parecido al de Keras, con una pérdida de validación mayor que la de entrenamiento (aunque menor que en el caso de Keras), lo que indica que el modelo ha memorizado patrones específicos del conjunto de entrenamiento que no se generalizan bien a datos nuevos. En el modelo aumentado, la curva de precisión de validación es algo inestable, además de que no supone una gran mejora respecto al modelo normal (con una precisión del 98.03 %), ya que la precisión de validación sigue siendo inferior a la de entrenamiento. En cuanto a la pérdida, esta sigue siendo mayor en el conjunto de validación que en el de entrenamiento, lo que indica que el modelo sigue memorizando patrones específicos del conjunto de entrenamiento.

Por otra parte, comparando ambos frameworks directamente, se puede observar que Keras presenta una mayor brecha entre las curvas de entrenamiento y validación en el modelo normal, lo que sugiere un mayor sobreajuste en comparación con los modelos de



PyTorch. En el caso del modelo aumentado, Keras muestra una mejor alineación entre las curvas, indicando una mejor generalización en comparación con PyTorch, donde hay indicios de sobreajuste en ambos modelos.

Tras analizar estos resultados, es posible concluir que el uso de datos ampliados ha mejorado la robustez y capacidad de generalización de los modelos en ambos frameworks, aunque Keras parece beneficiarse más de esta técnica en términos de reducción del sobreajuste, quedando el modelo normal de Keras como el que peor resultados presenta de los cuatro modelos entrenados seguido del modelo normal de PyTorch, el cual a su vez es superado por su versión aumentada, quedando el modelo de Keras con datos ampliados como el mejor de los cuatro modelos estandar.

D.3. Pseudocódigos

A continuación se presentan los pseudocódigos de los procedimientos de entrenamiento implementados en Keras y PyTorch. Es el mismo para ambos modelos normales y aumentados, con la única diferencia en el dataset utilizado para el entrenamiento, y las etapas y tamaño de lote mencionados anteriormente.

D.3.1. Keras

Algoritmo: Flujo de entrenamiento de la Red Neuronal para detección de sustancias (Keras).

- 1: **Entrada:** Archivos espectrales (.txt) en directorios *BUENO* y *MALO*
- 2: **Salida:** Modelo clasificador optimizado en formato .keras y .tflite
- 3: *Fase 1: Preprocesamiento de Datos*
- 4: Inicializar listas X (características) e y (etiquetas)
- 5: **for** cada archivo en directorios **do**
- 6: Leer datos numéricos omitiendo metadatos

```
7:   Aplanar matriz de datos a vector unidimensional
8:   if longitud <  $L_{\text{máx}}$  then
9:       Aplicar Padding con valores NaN hasta alcanzar  $L_{\text{máx}}$ 
10:   end if
11:   Asignar etiqueta (1 para Bueno, 0 para Malo)
12: end for
13: Imputar valores faltantes (NaN) utilizando estrategia de constante (0)
14: Dividir dataset: 80 % Entrenamiento/Validación, 20 % Prueba Final
15: Fase 2: Normalización
16: Calcular  $\mu, \sigma$  de media y desviación estándar del conjunto de entrenamiento
17:  $X_{\text{scaled}} \leftarrow \frac{X - \mu}{\sigma}$  (Escalado estándar para convergencia de ANN)
18: Fase 3: Construcción de la Red Neuronal Artificial
19: Definir arquitectura secuencial:
20:     1. Capa Densa (32 neuronas, activación ReLU, entrada de tamaño 242676)
21:     2. Capa Dropout (0.2) para evitar sobreajuste
22:     3. Capa Densa (16 neuronas, activación ReLU)
23:     4. Capa de Salida (1 neurona, activación Sigmoide)
24: Configurar optimizador Adam (tasa: 0.001) y función de pérdida Binary Crossentropy
25: Fase 4: Entrenamiento y Evaluación
26: Configurar Early Stopping monitoreando la pérdida de validación
27: Entrenar modelo por 50 épocas con tamaño de lote (batch) de 5
28: Evaluar precisión final en el conjunto de prueba (Test)
29: Fase 5: Exportación
30: Guardar modelo en formato nativo Keras (.keras)
31: Convertir modelo a TensorFlow Lite (.tflite) para despliegue móvil/embebido
```



D.3.2. PyTorch

Algoritmo: Flujo de entrenamiento de la Red Neuronal para detección de sustancias (PyTorch).

- 1: **Entrada:** Archivos espectrales (.txt) en directorios *BUENO* y *MALO*
- 2: **Salida:** Modelo clasificador optimizado en formato .pth y .tflite
- 3: *Fase 1: Preprocesamiento de Datos*
- 4: Inicializar listas X (características) e y (etiquetas)
- 5: **for** cada archivo en directorios **do**
- 6: Leer datos numéricos usando pandas
- 7: Convertir a numérico forzando errores a NaN
- 8: Aplanar matriz de datos a vector unidimensional
- 9: **if** todos los valores son NaN **then**
- 10: Ignorar archivo y mostrar advertencia
- 11: **else**
- 12: Añadir datos a X y etiqueta (1 para BUENO, 0 para MALO) a y
- 13: **end if**
- 14: **end for**
- 15: Encontrar longitud máxima entre todos los arrays
- 16: Rellenar (Padding) los arrays más cortos con valores NaN
- 17: *Fase 2: División del Dataset y Escalado*
- 18: Dividir datos: 80 % Entrenamiento/Validación, 20 % Prueba Final (stratificado)
- 19: Dividir 80 % en 75 % Entrenamiento, 25 % Validación (resultado: 60 %, 20 %, 20 %)
- 20: Imputar valores faltantes (NaN) utilizando estrategia de constante (valor 0)
- 21: Normalizar datos con *StandardScaler* (media 0, varianza 1)
- 22: Convertir datasets a tensores de PyTorch
- 23: *Fase 3: Construcción de la Red Neuronal Artificial*
- 24: Definir clase *OlfativaANN* heredando de *nn.Module*:
- 25: 1. Capa Lineal (entrada→32 neuronas)
- 26: 2. Activación ReLU

```
27:   3. Capa Dropout (0.2) para evitar sobreajuste
28:   4. Capa Lineal (32→16 neuronas)
29:   5. Activación ReLU
30:   6. Capa Lineal (16→1 neurona)
31:   7. Activación Sigmoide
32: Inicializar modelo con dimensión de entrada = tamaño de características
33: Configurar función de pérdida Binary Cross Entropy Loss
34: Configurar optimizador Adam (tasa de aprendizaje: 0.001)
35: Fase 4: Entrenamiento con Early Stopping
36: Configurar parámetros: épocas=15, tamaño de lote=5, paciencia=10 épocas
37: Crear DataLoader para entrenamiento y validación
38: for cada época hasta máximo de épocas do
39:     Modo entrenamiento: Forward pass, calcular pérdida, backward, actualizar pesos
40:     Modo evaluación: Forward pass en conjunto de validación, calcular pérdida
41:     if pérdida de validación mejora then
42:         Reiniciar contador de paciencia
43:         Guardar el mejor modelo (opcional)
44:     else
45:         Incrementar contador de paciencia
46:         if paciencia alcanzada then
47:             Detener entrenamiento (Early Stopping)
48:         end if
49:     end if
50: end for
51: Fase 5: Evaluación en Conjunto de Prueba
52: Evaluar modelo en datos de prueba
53: Calcular precisión y matriz de confusión
54: Visualizar curvas de aprendizaje (Accuracy y Loss)
55: Fase 6: Exportación del Modelo
```




- 56: Guardar modelo en formato PyTorch (.pth)
- 57: Exportar modelo a ONNX (.onnx) usando *torch.onnx.export*
- 58: Cargar modelo ONNX y convertir a TensorFlow SavedModel
- 59: Convertir TensorFlow SavedModel a TensorFlow Lite (.tflite)
- 60: Guardar modelo TFLite para despliegue móvil/embebido

Anexo E

Pseudocódigos de Sniffer

En este anexo se presentan los pseudocódigos de los archivos principales que componen la aplicación móvil Sniffer desarrollada para la detección de drogas mediante la nariz electrónica. Estos pseudocódigos describen la lógica y el flujo de datos dentro de la aplicación, facilitando la comprensión de su funcionamiento y estructura.

E.1. Pantalla principal

La pantalla principal de la aplicación es responsable de gestionar el escaneo de dispositivos Bluetooth disponibles, la solicitud de permisos necesarios y la navegación hacia la pantalla de análisis una vez seleccionado un dispositivo.

Algoritmo: Flujo de escaneo y selección de dispositivos Bluetooth (Pantalla Principal).

- 1: Entrada: Solicitud de búsqueda de dispositivos Bluetooth
- 2: Salida: Navegación a pantalla de análisis con dispositivo seleccionado
- 3: Fase 1: Solicitud y Validación de Permisos
- 4: Crear lista de permisos: bluetoothScan, bluetoothConnect, location
- 5: Solicitar permisos al sistema operativo



```
6: Obtener estado de cada permiso como booleano
7: if todos los permisos están otorgados then
8:     Proceder a fase de escaneo
9: else
10:     Mostrar notificación: permisos necesarios
11:     Detener flujo de inicialización
12: end if
13: Fase 2: Obtención de Dispositivos Conectados
14: Obtener lista de dispositivos conectados del sistema
15: for cada dispositivo conectado do
16:     Convertir dispositivo a objeto ScanResult
17:     Crear estructura de datos con nombre e ID
18:     Añadir a lista scanResults
19: end for
20: Fase 3: Iniciación del Escaneo Bluetooth
21: Detener escaneo previo para evitar conflictos
22: Iniciar nuevo escaneo con duración de 15 segundos
23: Configurar listener para eventos de descubrimiento
24: Fase 4: Procesamiento de Resultados del Escaneo
25: Crear conjunto de IDs de dispositivos mostrados
26: Monitorear stream de scanResults
27: for cada dispositivo descubierto do
28:     if nombre no vacío AND ID no en existingIds then
29:         Añadir dispositivo a lista scanResults
30:     end if
31: end for
32: Actualizar interfaz mediante setState()
33: Fase 5: Monitoreo del Estado de Escaneo
34: Crear listener para monitor de estado de escaneo
```



```
35: if isScanning es verdadero then
36:     Mostrar barra de progreso linear
37: else
38:     Ocultar barra de progreso
39: end if
40: Fase 6: Renderización de Lista de Dispositivos
41: for cada dispositivo en scanResults do
42:     Extraer nombre del dispositivo
43:     Crear listener para estado de conexión
44:     if nombre contiene NOSE OR ESP32 then
45:         Marcar como nariz electrónica
46:         Aplicar estilos especiales: borde azul
47:         Mostrar nombre en negrita
48:     else
49:         Aplicar estilos estándar
50:     end if
51:     Renderizar tarjeta de dispositivo
52: end for
53: Fase 7: Actualización de Estado de Conexión
54: for cada dispositivo mostrado do
55:     if estado = conectado then
56:         Mostrar: CONECTADO
57:     else
58:         Mostrar: ID remoto del dispositivo
59:     end if
60: end for
61: Fase 8: Selección de Dispositivo por Usuario
62: if dispositivo NO está conectado then
63:     Detener escaneo Bluetooth
```



```
64:     Establecer conexión con dispositivo
65: end if
66: Esperar confirmación de conexión
67: Navegar a pantalla de análisis
68: Fase 9: Limpieza Automática al Retornar
69: for cada dispositivo en lista de conectados do
70:     Desconectar dispositivo
71: end for
72: Limpiar lista de scanResults
73: Reiniciar escaneo
```

E.2. Pantalla de análisis

La pantalla de análisis es el núcleo funcional de la aplicación, donde se realiza la comunicación continua con la nariz electrónica, se capturan datos de sensores en tiempo real y se ejecutan modelos de inteligencia artificial para la detección de sustancias. La pantalla combina procesamiento de datos, normalización estadística e inferencia neural para clasificar las muestras.

Algoritmo: Flujo de análisis y detección de sustancias (Pantalla de Análisis).

- 1: Entrada: Dispositivo Bluetooth conectado con servicios UART disponibles
- 2: Salida: Clasificación de muestra con confianza probabilística
- 3: Fase 1: Carga de Configuración de Modelos
- 4: Cargar archivo JSON models-config.json desde assets
- 5: Parsear estructura JSON extrayendo lista de modelos
- 6: **for** cada modelo en lista **do**
- 7: Extraer parámetros: id, displayName, fileName
- 8: Extraer vectores: medias, desviaciones
- 9: Crear instancia ModelConfig con parámetros



```
10:     Añadir a lista availableConfigs
11: end for
12: if lista de modelos no vacía then
13:     Establecer primer modelo como defecto
14:     Cargar archivo TFLite del modelo inicial
15: end if
16: Fase 2: Establecimiento de Conexión Bluetooth
17: Esperar confirmación de conexión del dispositivo
18: Aplicar retardo de 500 ms para estabilización
19: Descubrir servicios disponibles en el dispositivo
20: Fase 3: Identificación de Servicios UART
21: for cada servicio descubierto do
22:     if UUID del servicio = servicio UART transparente then
23:         for cada característica en servicio do
24:             if UUID característica contiene 9616 then
25:                 Almacenar característica como uartChar
26:                 Habilitar notificaciones en característica
27:                 Configurar listener para datos recibidos
28:             end if
29:         end for
30:     end if
31: end for
32: Fase 4: Procesamiento de Datos en Tiempo Real
33: Inicializar buffer de string vacío
34: Cuando se reciben datos desde dispositivo:
35: Convertir bytes a string y acumular en buffer
36: if buffer contiene salto de línea then
37:     Dividir buffer por saltos de línea
38:     Preservar última línea incompleta en buffer
```



```
39:   for cada línea completa do
40:       Eliminar espacios en blanco
41:   if línea = STOP OR contiene AutoStop Mode then
42:       Detener análisis
43:       Mostrar diálogo: Experimento Finalizado
44:   end if
45:   if línea comienza con # then
46:       Ignorar línea (eco de comando)
47:   end if
48:   if línea = OK then
49:       Añadir mensaje: OK — Comando aceptado
50:   else
51:       Procesar línea como trama de sensores
52:       if validación exitosa AND análisis en progreso then
53:           Pasar datos a red neuronal
54:       end if
55:   end if
56: end for
57: end if
58: Fase 5: Procesamiento y Extracción de Datos
59: Dividir trama por comas para extraer tokens
60: if número de tokens mayor o igual a 21 then
61:     Definir índices de sensores: 1,4,5,6,7,8,9,10,12,14,15,16,20
62:   for cada índice de sensor do
63:       Extraer token en posición índice
64:       Reemplazar comas por puntos
65:       Convertir a double
66:       Almacenar en lista sensoresLimpios
67:   end for
```



```
68:     Extraer fase de muestreo del último token
69:     if fase = 1.0 (Muestra) then
70:         Retornar lista de sensores procesados
71:     else
72:         Descartar datos
73:         Retornar null
74:     end if
75: else
76:     Retornar null (trama inválida)
77: end if
78: Fase 6: Normalización de Datos
79: Obtener configuración de modelo: medias y desviaciones
80: for cada valor i en datos de entrada do
81:     Obtener media  $\mu_i$  y desviación  $\sigma_i$ 
82:     if  $\sigma_i = 0$  then
83:         Usar  $\sigma_i = 1.0$  para evitar división por cero
84:     end if
85:     Aplicar normalización:  $x\text{-scaled}[i] = (x[i] - \mu_i) / \sigma_i$ 
86: end for
87: Reshape vector a forma [1, 13] para entrada del modelo
88: Fase 7: Inferencia en Red Neuronal TFLite
89: Validar que intérprete TFLite esté cargado
90: Validar que configuración de modelo esté seleccionada
91: Crear tensor de salida con forma [1, 1]
92: Ejecutar: interpreter.run con entrada normalizada
93: Extraer probabilidad de salida:  $P = \text{output}[0][0]$ 
94: Fase 8: Actualización de Resultado y Retroalimentación
95: Actualizar resultadoIA con nueva probabilidad
96: if P mayor o igual a 0.7 AND vibración habilitada then
```




```
97:    Verificar disponibilidad de vibrador
98:    if vibrador disponible then
99:        Ejecutar vibración de 500 ms como alerta
100:    end if
101: end if
102: Actualizar interfaz con nuevo resultado
103: Mostrar sustancia detectada o aire limpio
104: Mostrar porcentaje de confianza
105: Fase 9: Envío de Comandos al Dispositivo
106: if tiempo desde último comando mayor o igual a 500 ms then
107:    Obtener comando: Exper (iniciar) o Stop (detener)
108:    Formatear comando con retorno de carro y salto de línea
109:    Enviar bytes codificados a característica UART
110:    Registrar hora de comando
111:    Mostrar en consola: enviando comando
112: else
113:    Descartar comando (anticebo)
114: end if
115: Fase 10: Selección Dinámica de Modelo
116: if usuario selecciona modelo diferente then
117:    Obtener nueva configuración del modelo
118:    Actualizar currentConfig
119:    Resetear resultadoIA a 0.0
120:    Cargar archivo TFLite del nuevo modelo
121: end if
122: Fase 11: Control de Historial de Consola
123: for cada nuevo mensaje do
124:    Insertar mensaje al inicio de lista
125:    Etiquetar como comando o dato recibido
```



```
126:   if tamaño lista mayor a 100 then  
127:       Eliminar mensaje más antiguo  
128:   end if  
129: end for  
130:  Mostrar historial en orden inverso
```

E.3. Pantalla de ajustes

La pantalla de ajustes permite al usuario modificar los parámetros del funcionamiento de la nariz electrónica, sincronizando la configuración con el dispositivo mediante comandos UART. Incluye opciones para ajustar ciclos, tiempos de muestreo y preferencias de la aplicación.

Algoritmo: Flujo de configuración y sincronización de parámetros (Pantalla de Ajustes).

- 1: Entrada: Dispositivo Bluetooth conectado con característica UART activa
- 2: Salida: Configuración sincronizada con el dispositivo y preferencias de usuario
- 3: Fase 1: Inicialización de Pantalla de Ajustes
- 4: Mostrar interfaz con campos ajustables
- 5: Inicializar variables: numCiclos, tiempoAdsorcion, tiempoDesorcion, autoStop, vibracionActiva
- 6: Invocar lectura de configuración actual del dispositivo
- 7: Fase 2: Lectura de Configuración Actual
- 8: Inicializar buffer para acumular respuesta
- 9: Crear comando: SETTINGS
- 10: Formatear comando: agregar retorno de carro y salto de línea
- 11: Enviar bytes codificados a característica UART
- 12: Mostrar indicador de carga o espera
- 13: Fase 3: Recepción de Datos de Configuración



```
14: Escuchar stream de datos desde dispositivo
15: Cuando se reciben datos:
16: Convertir bytes a string
17: Acumular en buffer de configuración
18: if buffer contiene indicador de completitud then
19:     Procesar respuesta y extraer parámetros
20:     Actualizar interfaz con valores leídos
21:     Limpiar buffer
22: end if
23: Fase 4: Extracción de Parámetros desde Respuesta
24: Buscar cada parámetro en la cadena de respuesta:
25: Extraer tiempo de desorción: buscar Desorption time:
26: Extraer tiempo de adsorción: buscar Adsorption time:
27: Extraer número de ciclos: buscar Number of cycles in AutoStop Mode:
28: Extraer estado de AutoStop: buscar AutoStop Mode:
29: for cada parámetro encontrado do
30:     Validar que el valor sea un número entero válido
31:     Almacenar en variable correspondiente
32: end for
33: Fase 5: Actualización de Interfaz Gráfica
34: Mostrar todos los parámetros con valores actuales:
35: Número de ciclos: mostrar campo numérico con controles +/-
36: Tiempo de adsorción: mostrar campo numérico con controles +/-
37: Tiempo de desorción: mostrar campo numérico con controles +/-
38: Swtich de AutoStop inteligente: mostrar toggle activado/desactivado
39: Switch de Vibración de Alerta: mostrar toggle activado/desactivado
40: Fase 6: Modificación de Parámetros por Usuario
41: Para cada parámetro numérico:
42: Mostrar botón de decremento y incremento
```



```
43: if usuario presiona botón + then
44:     Incrementar valor en paso especificado
45:     Validar que no supere límite máximo
46:     Actualizar pantalla con nuevo valor
47: end if
48: if usuario presiona botón - then
49:     Decrementar valor en paso especificado
50:     Validar que no sea inferior a límite mínimo
51:     Actualizar pantalla con nuevo valor
52: end if
53: Fase 7: Envío de Comandos de Configuración
54: Cuando usuario presiona CONFIRMAR CAMBIOS
55: Enviar comando número de ciclos: #NSNC,[valor]
56: Aplicar retardo de 150 ms
57: Enviar comando tiempo de adsorción: #NSTA,[valor]
58: Aplicar retardo de 150 ms
59: Enviar comando tiempo de desorción: #NSTD,[valor]
60: Aplicar retardo de 150 ms
61: Enviar comando estado AutoStop: #NSAS,[0 o 1]
62: Aplicar retardo de 150 ms
63: Fase 8: Notificación de Cambios
64: Para cada comando enviado:
65: if callback onMessageSent está disponible then
66:     Invocar callback con texto del comando y tipo (true = comando)
67:     Propagar información a pantalla anterior
68: end if
69: Fase 9: Confirmación Visual de Aplicación
70: Mostrar SnackBar flotante con mensaje: Configuración aplicada
71: Configurar color de fondo verde para indicar éxito
```



- 72: Duración: 2 segundos
- 73: Mostrar en parte superior de pantalla
- 74: Fase 10: Retorno a Pantalla Anterior
- 75: Capturar estado de vibración (vibracionActiva)
- 76: Navegar de vuelta a pantalla anterior
- 77: Pasar estado de vibración como valor de retorno
- 78: Pantalla anterior recibe booleano y actualiza su estado

E.4. Barra superior

La barra superior personalizada (CustomSearchAppBar) es un widget redondeado que se adapta dinámicamente según la pantalla actual. Proporciona navegación mediante botones laterales (ajustes o búsqueda) y muestra el título de la pantalla centrado, manteniendo una interfaz consistente en toda la aplicación.

Algoritmo: Flujo de construcción y adaptación de barra superior.

- 1: Entrada: Parámetros de título, callbacks opcionales e indicador de pantalla
- 2: Salida: Widget AppBar personalizado redondeado con botones contextuales
- 3: Fase 1: Inicialización de Parámetros
- 4: Recibir parámetro: title (texto visible en barra)
- 5: Recibir parámetro opcional: onSettingsPressed (callback botón ajustes)
- 6: Recibir parámetro opcional: onSearchPressed (callback botón búsqueda)
- 7: Recibir parámetro: isSettingsScreen (booleano indicador de contexto)
- 8: Recibir parámetro opcional: onBackAction (callback botón retroceso)
- 9: Fase 2: Cálculo de Tamaño Preferido
- 10: Establecer preferredSize altura = 100 píxeles
- 11: Establecer ancho = pantalla completa
- 12: Fase 3: Construcción del Contenedor Exterior
- 13: Crear Container con padding superior de 40 píxeles



```
14: Establecer padding lateral de 15 píxeles (izquierda y derecha)
15: Crear Container interno con altura de 60 píxeles
16: Fase 4: Decoración del Contenedor
17: Establecer color de fondo: azul oscuro (Colors.blue.shade800)
18: Configurar borde redondeado: 30 píxeles de radio
19: Añadir sombra:
20:   Color sombra: negro con 26 % de opacidad
21:   Desenfoque (blur): 8 píxeles
22:   Desplazamiento vertical: 4 píxeles hacia abajo
23: Fase 5: Construcción de Fila de Contenido
24: Crear fila (Row) con disposición horizontal
25: Distribuir elementos: izquierda, centro, derecha
26: Fase 6: Botón Izquierdo Contextual
27: if isSettingsScreen es verdadero then
28:   Mostrar botón flecha retroceso
29:   Configurar icono: Icons.arrow_back
30:   Configurar color: blanco
31:   if onBackAction está disponible then
32:     Conectar callback a onBackAction
33:   else
34:     Conectar callback a Navigator.pop
35:   end if
36: else
37:   if onSettingsPressed está disponible then
38:     Mostrar botón configuración (settings)
39:     Configurar icono: Icons.settings
40:     Configurar color: blanco
41:     Conectar callback a onSettingsPressed
42:   else
```



```
43:         Reservar espacio vacío de 48 píxeles
44:     end if
45: end if
46: Fase 7: Título Centrado
47: Expandir elemento para ocupar espacio disponible
48: Convertir título a mayúsculas
49: Aplicar estilo de texto:
50: Color: blanco
51: Peso de fuente: negrita (FontWeight.bold)
52: Tamaño de fuente: 13 píxeles
53: Espaciado de letras: 1.1
54: Fase 8: Botón Derecho de Búsqueda
55: if onSearchPressed está disponible then
56:     Mostrar botón búsqueda
57:     Configurar icono: Icons.search
58:     Configurar color: blanco
59:     Conectar callback a onSearchPressed
60: else
61:     Reservar espacio vacío de 48 píxeles
62: end if
63: Fase 9: Renderización Final
64: Compilar estructura completa de widget
65: Aplicar restricciones de tamaño preferido
66: Mostrar AppBar con diseño visual aplicado
```

E.5. Navegación

La clase utilitaria AppNavigation centraliza toda la lógica de transiciones entre pantallas de la aplicación. Proporciona métodos estáticos para navegar hacia la pantalla



de ajustes (con retorno de valores booleanos) y hacia la pantalla principal (limpiando el historial de navegación). Esto asegura una navegación consistente y controlada en toda la aplicación.

Algoritmo: Flujo de gestión y navegación entre pantallas.

- 1: Entrada: Contexto BuildContext y parámetros de navegación específicos según destino
- 2: Salida: Transición a nueva pantalla con parámetros sincronizados
- 3: Fase 1: Invocación de Navegación a Ajustes
- 4: Usuario presiona botón de configuración en pantalla actual
- 5: Invocar método: AppNavigation.goToSettings()
- 6: Pasar contexto BuildContext
- 7: Pasar dispositivo BluetoothDevice conectado
- 8: Pasar característica UART uartChar (nullable)
- 9: Pasar callback: onMessageSent(String, bool) para recibir mensajes
- 10: Fase 2: Creación de MaterialPageRoute
- 11: Instanciar MaterialPageRoute genérica con tipo bool
- 12: Definir builder con función lambda
- 13: Construir instancia de SettingsScreen dentro de builder
- 14: Fase 3: Paso de Parámetros a SettingsScreen
- 15: Pasar device al constructor de SettingsScreen
- 16: Pasar uartChar (puede ser null) al constructor
- 17: Pasar callback onMessageSent al constructor
- 18: Verificar que todos los parámetros requeridos estén disponibles
- 19: Fase 4: Ejecución de Navigator.push
- 20: Ejecutar Navigator.push<bool>() con el contexto y ruta
- 21: Especificar tipo genérico <bool> para capturar retorno
- 22: Iniciar transición de pantalla hacia SettingsScreen
- 23: Mantener historial de navegación activo
- 24: Fase 5: Espera de Confirmación



- 25: SettingsScreen se muestra en primer plano
- 26: Usuario modifica parámetros de configuración
- 27: Usuario presiona botón CONFIRMAR o ATRÁS
- 28: SettingsScreen retorna valor booleano: vibracionActiva
- 29: Fase 6: Retorno de Valor Booleano
- 30: Capturar estado de vibración modificado
- 31: Ejecutar Navigator.pop(context, vibracionActiva)
- 32: Devolver booleano a pantalla anterior mediante Future
- 33: Fase 7: Invocación de Navegación a Pantalla Principal
- 34: Usuario cierra sesión o requiere retorno al inicio
- 35: Invocar método: AppNavigation.goToHome()
- 36: Pasar contexto BuildContext
- 37: Fase 8: Limpieza de Historial de Navegación
- 38: Ejecutar Navigator.pushAndRemoveUntil()
- 39: Crear MaterialPageRoute hacia MainScreen
- 40: Definir predicado de ruta: (route) =>false
- 41: El predicado false elimina TODAS las rutas previas
- 42: Fase 9: Construcción de MainScreen
- 43: Instanciar const MainScreen() sin parámetros
- 44: Aplicar transición de pantalla
- 45: Establecer MainScreen como único punto en la pila de navegación
- 46: Fase 10: Garantía de Punto de Entrada Único
- 47: Usuario no puede presionar tecla atrás desde MainScreen
- 48: Historial completamente limpio sin rutas pendientes
- 49: Solo MainScreen es accesible tras goToHome()
- 50: Previene navegación accidental a pantallas cerradas



E.6. Inicialización de la aplicación

El archivo `main.dart` es el punto de entrada de la aplicación Sniffer. Realiza la inicialización de los bindings de Flutter, configura el tema visual, y establece la pantalla principal como punto de inicio. Utiliza Material Design 3 con una paleta de colores personalizada basada en azul oscuro.

Algoritmo: Flujo de inicialización y configuración de la aplicación (main).

- 1: Entrada: Comando de ejecución de la aplicación desde el sistema operativo
- 2: Salida: Aplicación Flutter ejecutándose con MyApp como widget raíz
- 3: Fase 1: Ejecución de Función main
- 4: Sistema operativo invoca función `main()`
- 5: Inicio de punto de entrada de la aplicación
- 6: Fase 2: Inicialización de Bindings
- 7: Llamar a `WidgetsFlutterBinding.ensureInitialized()`
- 8: Garantizar que el sistema de binding de Flutter esté preparado
- 9: Permitir operaciones específicas antes de `runApp()`
- 10: Establecer comunicación entre Flutter y la plataforma nativa
- 11: Fase 3: Ejecución de MyApp
- 12: Invocar `runApp()` con instancia `const MyApp()`
- 13: Pasar widget raíz a Flutter engine
- 14: Flutter comienza a renderizar la aplicación
- 15: Fase 4: Construcción de MaterialApp
- 16: `MyApp.build()` es invocado con contexto `BuildContext`
- 17: Retornar `MaterialApp` con configuración personalizada
- 18: Establecer nombre de la aplicación: *Nariz Electrónica UEx*
- 19: Desactivar banner de depuración: `debugShowCheckedModeBanner = false`
- 20: Fase 5: Configuración del Tema
- 21: Crear instancia de `ThemeData`
- 22: Generar `ColorScheme` desde seed color: `Colors.blue.shade900`



- 23: El seed color azul oscuro proporciona paleta coherente
- 24: Activar Material Design 3: `useMaterial3 = true`
- 25: Configurar fondo de Scaffold: `Colors.grey.shade50` (gris muy claro)
- 26: Fase 6: Generación de Paleta de Colores
- 27: `ColorScheme.fromSeed()` genera colores derivados del seed
- 28: Genera colores primarios, secundarios y terciarios automáticamente
- 29: Crear colores para: background, surface, error, inversión
- 30: Asegurar contraste y accesibilidad según Material Design 3
- 31: Fase 7: Establecimiento de Pantalla Inicial
- 32: Asignar home: `const MainScreen()`
- 33: `MainScreen` es el widget inicial que se renderiza
- 34: Cuando app inicia, `MainScreen` aparece en primer plano
- 35: `MainScreen` es la base de la pila de navegación
- 36: Fase 8: Renderización y Ejecución
- 37: Flutter renderiza el árbol de widgets completo
- 38: Aplica tema configurado a toda la aplicación
- 39: `MainScreen` se muestra en pantalla
- 40: Aplicación está lista para interacción del usuario
- 41: Sistema de eventos y listeners comienza a funcionar

Glosario

Agente Canino: Perro entrenado específicamente para realizar tareas especializadas de seguridad, rescate o detección de sustancias. Los agentes caninos destacan por su excepcional sentido del olfato, pudiendo detectar y localizar drogas, explosivos, personas desaparecidas y otras sustancias de interés.

Biomímesis: Ciencia que estudia la naturaleza como fuente de inspiración para el desarrollo de tecnologías y sistemas que resuelvan problemas humanos mediante la imitación de modelos, sistemas o elementos biológicos.

Compuestos Orgánicos Volátiles (COVs): Grupo de sustancias químicas orgánicas cuya presión de vapor es lo suficientemente alta en condiciones ambientales normales como para evaporarse significativamente y entrar en la atmósfera. Son las moléculas clave en la detección olfativa.

Epitelio Olfativo: Tejido sensorial especializado ubicado en la cavidad nasal que contiene los receptores olfativos (neuronas sensoriales olfativas). Es responsable de detectar moléculas odorantes volátiles y convertirlas en señales nerviosas que se transmiten al cerebro para la percepción del olfato.

Compuestos de Azufre Volátiles (VSCs): Sustancias químicas que contienen azufre y presentan una alta volatilidad. Estos compuestos suelen tener olores fuertes y característicos, y son de gran interés en estudios de detección olfativa y análisis ambiental debido a su bajo umbral de detección por sistemas biológicos y sensores.



Quimiometría: Disciplina científica que utiliza métodos matemáticos y estadísticos para diseñar o seleccionar procedimientos de medida óptimos y proporcionar la máxima información química relevante mediante el análisis de datos químicos.

Análisis de Componentes Principales (PCA): Método estadístico de reducción de la dimensionalidad que transforma un conjunto de variables correlacionadas en un número menor de variables no correlacionadas llamadas componentes principales, facilitando la identificación de patrones y agrupamientos.

Análisis de Función de Discriminantes (DFA): Técnica estadística utilizada para determinar qué variables permiten distinguir mejor entre dos o más grupos de muestras previamente definidos, con el fin de crear un modelo predictivo de clasificación.

Análisis de Clusters (CA): Técnica exploratoria de análisis de datos que agrupa objetos o muestras de tal manera que la similitud entre los miembros de un mismo grupo sea máxima y la similitud con miembros de otros grupos sea mínima.

Artificial Neuronal Network (ANN): Redes neuronales artificiales en español, son modelos computacionales inspirados en el funcionamiento del sistema nervioso central biológico. Están compuestos por capas de nodos interconectados que aprenden a reconocer patrones complejos y relaciones no lineales en los datos.

Máquinas Vectoriales de Soporte (SVM): Algoritmos de aprendizaje automático supervisado capaces de resolver problemas de clasificación y regresión mediante la búsqueda de un hiperplano óptimo que maximice la separación entre diferentes clases en un espacio multidimensional.

Servicio cinológico: Unidad o equipo especializado en el adiestramiento, cuidado y empleo de perros con fines de seguridad, rescate o detección de sustancias ilícitas y especímenes protegidos.

Tráfico de marfil: Comercio ilegal de colmillos de elefantes y otros animales que poseen



defensas de marfil. Es una de las principales amenazas para la conservación de la fauna silvestre a nivel global.

Sobreajuste (Overfitting): Fenómeno que ocurre cuando un modelo de aprendizaje automático se ajusta demasiado a los datos de entrenamiento, capturando ruido y detalles irrelevantes, lo que reduce su capacidad de generalizar y rendir correctamente ante nuevos datos no vistos.

Dropout: Técnica de regularización utilizada en el entrenamiento de redes neuronales artificiales para prevenir el sobreajuste. Consiste en desactivar aleatoriamente un porcentaje de neuronas en una capa durante cada iteración del entrenamiento, forzando a la red a aprender características más robustas.

ReLU (Rectified Linear Unit): Función de activación utilizada en redes neuronales que transforma los valores de entrada anulando los negativos (convirtiéndolos en cero) y manteniendo los positivos. Es eficiente computacionalmente y ayuda a mitigar el problema del desvanecimiento del gradiente.

Sigmoide: Función matemática de activación que tiene una curva en forma de “S”. Transforma cualquier valor real de entrada en un valor comprendido en el rango entre 0 y 1, por lo que es ampliamente utilizada en la capa de salida de modelos de clasificación binaria para representar probabilidades.

Bluetooth Low Energy (BLE): Tecnología de red inalámbrica de área personal diseñada para aplicaciones de bajo consumo energético y corto alcance. A diferencia del Bluetooth clásico, está optimizada para dispositivos que requieren operar durante largos periodos con baterías limitadas, transmitiendo pequeñas cantidades de datos de forma periódica.

Indoor Air Quality (IAQ): Término que hace referencia a la calidad del aire dentro y alrededor de edificios y estructuras. En el contexto de narices electrónicas y sensores de gas, se utiliza un índice IAQ para cuantificar la presencia de Compuestos



Orgánicos Volátiles (COVs) y otros contaminantes, sirviendo como referencia para calibrar o validar las condiciones ambientales durante la toma de muestras.

Bibliografía

- [1] L. Buck and R. Axel. A novel multigene family may encode odorant receptors: A molecular basis for odor recognition. *Cell*, 65:175–187, 1991. [doi:10.1016/0092-8674\(91\)90418-X](https://doi.org/10.1016/0092-8674(91)90418-X).
- [2] K. C. Persaud and G. Dodd. Analysis of discrimination mechanisms in the mammalian olfactory system using a model nose. *Nature*, 299(5881):352–355, 1982. [doi:10.1038/299352a0](https://doi.org/10.1038/299352a0).
- [3] J. W. Gardner and P. N. Bartlett. A brief history of electronic noses. *Sensors and Actuators B: Chemical*, 18(1-3):210–211, 1994. [doi:10.1016/0925-4005\(94\)87085-3](https://doi.org/10.1016/0925-4005(94)87085-3).
- [4] D. Karakaya, O. Ulucan, and M. Turkan. Electronic nose and its applications: A survey. *International Journal of Automation and Computing*, 17:179–209, 2020. [doi:10.1007/s11633-019-1212-9](https://doi.org/10.1007/s11633-019-1212-9).
- [5] K. G. Furton and L. J. Myers. The scientific foundation and efficacy of the use of canines as chemical detectors for explosives. *Talanta*, 54(3):487–500, 2001. [doi:10.1016/S0039-9140\(00\)00546-4](https://doi.org/10.1016/S0039-9140(00)00546-4).
- [6] N. Lorenzo, T. Wan, R. J. Harper, Y.-L. Hsu, M. Chow, S. Rose, and K. G. Furton. Laboratory and field experiments used to identify *Canis lupus var. familiaris* active odor signature chemicals from drugs, explosives, and humans. *Analytical and Bioanalytical Chemistry*, 376(8):1212–1224, 2003. [doi:10.1007/s00216-003-2018-7](https://doi.org/10.1007/s00216-003-2018-7).

- [7] E. A. Baldwin, J. Bai, A. Plotto, and S. Dea. Electronic noses and tongues: Applications for the food and pharmaceutical industries. *Sensors*, 11(5):4744–4766, 2011. [doi:10.3390/s110504744](https://doi.org/10.3390/s110504744).
- [8] L. Capelli, S. Sironi, and R. Del Rosso. Electronic noses for environmental monitoring applications. *Sensors*, 14(11):19979–20007, 2014. [doi:10.3390/s141119979](https://doi.org/10.3390/s141119979).
- [9] T. Seiyama, A. Kato, K. Fujiishi, and M. Nagatani. A new detector for gaseous components using semiconductive thin films. *Analytical Chemistry*, 34(11):1502–1503, 1962. [doi:10.1021/ac60191a001](https://doi.org/10.1021/ac60191a001).
- [10] N. Yamazoe and K. Shimano. Theory of power laws for semiconductor gas sensors. *Sensors and Actuators B: Chemical*, 128(2):566–573, 2008. [doi:10.1016/j.snb.2007.07.036](https://doi.org/10.1016/j.snb.2007.07.036).
- [11] H. Ulmer, J. Mitrovics, G. Noetzel, U. Weimar, and W. Göpel. Odours and flavours identified with hybrid modular sensor systems. *Sensors and Actuators B: Chemical*, 43(1-3):24–33, 1997. [doi:10.1016/S0925-4005\(97\)00161-5](https://doi.org/10.1016/S0925-4005(97)00161-5).
- [12] I. T. Jolliffe and J. Cadima. Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A*, 374(2065):20150202, 2016. [doi:10.1098/rsta.2015.0202](https://doi.org/10.1098/rsta.2015.0202).
- [13] M. A. Akbar, A. Tariq, N. Sarfraz, and S. A. A. Shah. Electronic nose: Pattern recognition analysis for gas identification using PCA and LDA. *International Journal of Advanced Computer Science and Applications*, 7(8):276–281, 2016. [doi:10.14569/IJACSA.2016.070837](https://doi.org/10.14569/IJACSA.2016.070837).
- [14] E. L. Hines, E. Llobet, and J. W. Gardner. Electronic noses: A review of signal processing techniques. *IEE Proceedings – Circuits, Devices and Systems*, 146(6):297–310, 1999. [doi:10.1049/ip-cds:19990670](https://doi.org/10.1049/ip-cds:19990670).
- [15] I. Lundström, C. Svensson, A. Spetz, H. Sundgren, and F. Winqvist. From hydrogen sensors to olfactory images—twenty years with catalytic field-effect devices. *Sensors*

- and Actuators B: Chemical*, 13(1-3):16–23, 1993. doi:[10.1016/0925-4005\(93\)85313-Y](https://doi.org/10.1016/0925-4005(93)85313-Y).
- [16] H. Sundgren, I. Lundström, F. Winqvist, I. Lukkari, R. Carlsson, and S. Wold. Evaluation of a multiple gas mixture with a simple MOSFET gas sensor array and pattern recognition. *Sensors and Actuators B: Chemical*, 2(2):115–123, 1990. doi:[10.1016/0925-4005\(90\)80020-Z](https://doi.org/10.1016/0925-4005(90)80020-Z).
- [17] M. S. Freund and N. S. Lewis. A chemically diverse conducting polymer-based “electronic nose”. *Proceedings of the National Academy of Sciences*, 92(7):2652–2656, 1995. doi:[10.1073/pnas.92.7.2652](https://doi.org/10.1073/pnas.92.7.2652).
- [18] J. V. Hatfield, P. Neaves, P. J. Hicks, K. Persaud, and P. Travers. Towards an integrated electronic nose using conducting polymer sensors. *Sensors and Actuators B: Chemical*, 18(1-3):221–228, 1994. doi:[10.1016/0925-4005\(94\)87088-1](https://doi.org/10.1016/0925-4005(94)87088-1).
- [19] T.-Z. Wu. A piezoelectric biosensor as an olfactory receptor for odour detection: Electronic nose. *Biosensors and Bioelectronics*, 14(1):9–18, 1999. doi:[10.1016/S0956-5663\(98\)00086-4](https://doi.org/10.1016/S0956-5663(98)00086-4).
- [20] T. A. Kuchmenko and L. B. Lvova. A perspective on recent advances in piezoelectric chemical sensors for environmental monitoring and foodstuffs analysis. *Chemosensors*, 7(3):39, 2019. doi:[10.3390/chemosensors7030039](https://doi.org/10.3390/chemosensors7030039).
- [21] C. Wongchoosuk, A. Wisitsoraat, A. Tuantranont, and T. Kerdcharoen. Portable electronic nose based on carbon nanotube-SnO₂ gas sensors and its application for detection of methanol contamination in whiskeys. *Sensors and Actuators B: Chemical*, 147(2):392–399, 2010. doi:[10.1016/j.snb.2010.03.072](https://doi.org/10.1016/j.snb.2010.03.072).
- [22] S. Chatterjee, M. Castro, and J. F. Feller. An e-nose made of carbon nanotube based quantum resistive sensors for the detection of eighteen polar/nonpolar voc biomarkers of lung cancer. *Journal of Materials Chemistry B*, 1(36):4563–4575, 2013. doi:[10.1039/c3tb20819b](https://doi.org/10.1039/c3tb20819b).

- [23] G. Chen, T. M. Paronyan, and A. R. Harutyunyan. Sub-ppt gas detection with pristine graphene. *Applied Physics Letters*, 101(5):053119, 2012. doi:[10.1063/1.4742327](https://doi.org/10.1063/1.4742327).
- [24] A. Khorramifar, H. Karami, L. Lvova, A. Kolouri, E. Łazuka, M. Piłat-Rożek, G. Łagód, J. Ramos, J. Lozano, M. Kaveh, and Y. Darvishi. Environmental engineering applications of electronic nose systems based on MOX gas sensors. *Sensors*, 23(12):5716, 2023. doi:[10.3390/s23125716](https://doi.org/10.3390/s23125716).
- [25] S. Marco and A. Gutiérrez-Gálvez. Signal and data processing for machine olfaction and chemical sensing: A review. *IEEE Sensors Journal*, 12(11):3189–3214, 2012. doi:[10.1109/JSEN.2012.2192920](https://doi.org/10.1109/JSEN.2012.2192920).
- [26] M. Pardo and G. Sberveglieri. Classification of electronic nose data with support vector machines. *Sensors and Actuators B: Chemical*, 107(2):730–737, 2005. doi:[10.1016/j.snb.2004.12.005](https://doi.org/10.1016/j.snb.2004.12.005).
- [27] L. Zhang, F. Tian, H. Nie, L. Dang, G. Li, Q. Ye, and C. Kadri. Classification of multiple indoor air contaminants by an electronic nose and a hybrid support vector machine. *Sensors and Actuators B: Chemical*, 174:114–125, 2012. doi:[10.1016/j.snb.2012.07.021](https://doi.org/10.1016/j.snb.2012.07.021).
- [28] A. Rosales, M. I. Talaverano, J. Lozano, C. Sánchez-Vicente, Ó. Santamaría, C. García-Latorre, and S. Rodrigo. Craft beer vs industrial beer: Chemical and sensory differences. *British Food Journal*, 123(12):4332–4352, 2021. doi:[10.1108/BFJ-01-2021-0074](https://doi.org/10.1108/BFJ-01-2021-0074).
- [29] A. Martínez, A. Hernández, P. Arroyo, J. Lozano, A. Martín, and M. de G. Córdoba. Early detection of *Monilinia laxa* in nectarine using electronic nose technology: A non-destructive diagnostic approach. *Chemosensors*, 13(11):391, 2025. doi:[10.3390/chemosensors13110391](https://doi.org/10.3390/chemosensors13110391).

- [30] P. Arroyo, F. Meléndez, J. I. Suárez, J. L. Herrero, S. Rodríguez, and J. Lozano. Electronic nose with digital gas sensors connected via Bluetooth to a smartphone for air quality measurements. *Sensors*, 20(3):786, 2020. doi:[10.3390/s20030786](https://doi.org/10.3390/s20030786).
- [31] A.-C. Romain and J. Nicolas. Long term stability of metal oxide-based gas sensors for e-nose environmental applications: An overview. *Sensors and Actuators B: Chemical*, 146(2):502–506, 2010. doi:[10.1016/j.snb.2009.12.027](https://doi.org/10.1016/j.snb.2009.12.027).
- [32] A. Rudnitskaya. Calibration update and drift correction for electronic noses and tongues. *Frontiers in Chemistry*, 6:433, 2018. doi:[10.3389/fchem.2018.00433](https://doi.org/10.3389/fchem.2018.00433).
- [33] Q. Chen, J. Song, J. Bi, X. Meng, and X. Wu. Characterization of volatile profile from ten different varieties of Chinese jujubes by HS-SPME/GC–MS coupled with E-nose. *Food Research International*, 105:605–615, 2018. doi:[10.1016/j.foodres.2017.11.054](https://doi.org/10.1016/j.foodres.2017.11.054).
- [34] Guardia Civil. Centro de adiestramiento de perros, 2025. [Online]. Available: https://web.guardiacivil.es/es/institucional/conocenos/centros_docentes/perros/. [Accessed: Jan. 23, 2026].
- [35] L. Lazarowski and D. C. Dorman. Explosives detection by military working dogs: Olfactory generalization from components to mixtures. *Applied Animal Behaviour Science*, 151:84–93, 2014. doi:[10.1016/j.applanim.2013.11.010](https://doi.org/10.1016/j.applanim.2013.11.010).
- [36] R. J. Harper, J. R. Almirall, and K. G. Furton. Identification of dominant odor chemicals emanating from explosives for use in developing optimal training aid combinations and mimics for canine detection. *Talanta*, 67(2):313–327, 2005. doi:[10.1016/j.talanta.2005.05.019](https://doi.org/10.1016/j.talanta.2005.05.019).
- [37] I. Gazit and J. Terkel. Domination of olfaction over vision in explosives detection by dogs. *Applied Animal Behaviour Science*, 82(1):65–73, 2003. doi:[10.1016/S0168-1591\(03\)00051-0](https://doi.org/10.1016/S0168-1591(03)00051-0).

- [38] L. Lit, J. B. Schweitzer, and A. M. Oberbauer. Handler beliefs affect scent detection dog outcomes. *Animal Cognition*, 14(3):387–394, 2011. [doi:10.1007/s10071-010-0373-2](https://doi.org/10.1007/s10071-010-0373-2).

Licencia

Estudio de viabilidad de una Nariz Electrónica como herramienta de apoyo para los Agentes Caninos © 2026 by Raúl Martín-Romo Sánchez is licensed under Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International.

To view a copy of this license, visit:

<https://creativecommons.org/licenses/by-nc-sa/4.0/>